

Aula 01

*DataPrev (Perfil 3: Desenvolvimento de
Software) Banco de Dados - 2026
(Pós-Edital)*

Autor:

Lucas Fargoni Di Ianni

09 de Julho de 2026

Índice

1) 2.0 Modelos Lógicos - Teoria	3
2) 2.0 Modelos Lógicos - Resumo	54
3) 2.0 Modelos Lógicos - Questões Comentadas - FGV	64
4) 2.0 Modelos Lógicos - Lista de Questões - FGV	88
5) 2.1 IDEF1X - Teoria	102
6) 2.1 IDEF1X - Questões Comentadas - MULTIBANCAS	109
7) 2.1 IDEF1X - Lista de Questões - MULTIBANCAS	123
8) Modelo Lógico - Bibliografia	132

MODELOS DE DADOS LÓGICOS

Introdução

Pessoal, agora vamos estudar os **modelos de dados lógicos**, aqueles que estão a um **nível intermediário de abstração entre o conceitual** (o mais abstrato, que descreve o quê o sistema precisa armazenar, usando entidades e relacionamentos, sem se preocupar com tecnologia) **e o físico** (o mais concreto, que descreve como os dados serão armazenados em determinado produto comercial).

Assim sendo, o **modelo lógico** consegue traduzir o modelo conceitual para uma estrutura compatível com o paradigma de banco de dados a ser utilizado, porém sem adentrar em especificações físicas de armazenamento.

É nesse momento, então, a depender da natureza dos dados e das necessidades do negócio, que decidiremos se os dados serão organizados como tabelas (modelo lógico relacional), como documentos (modelo lógico documental), como grafos, como objetos, etc.

Consequentemente, também é nesse momento que definimos o tipo de SGBD a ser utilizado. Por exemplo, ao modelarmos nossos dados com o modelo relacional, escolhemos, automaticamente, um banco de dados relacional, composto por tabelas, atributos etc. De outro modo, se modelarmos nossos dados com um modelo de dados orientado a objetos, nosso banco de dados seguirá esse paradigma, sendo composto por classes, atributos, associações etc.

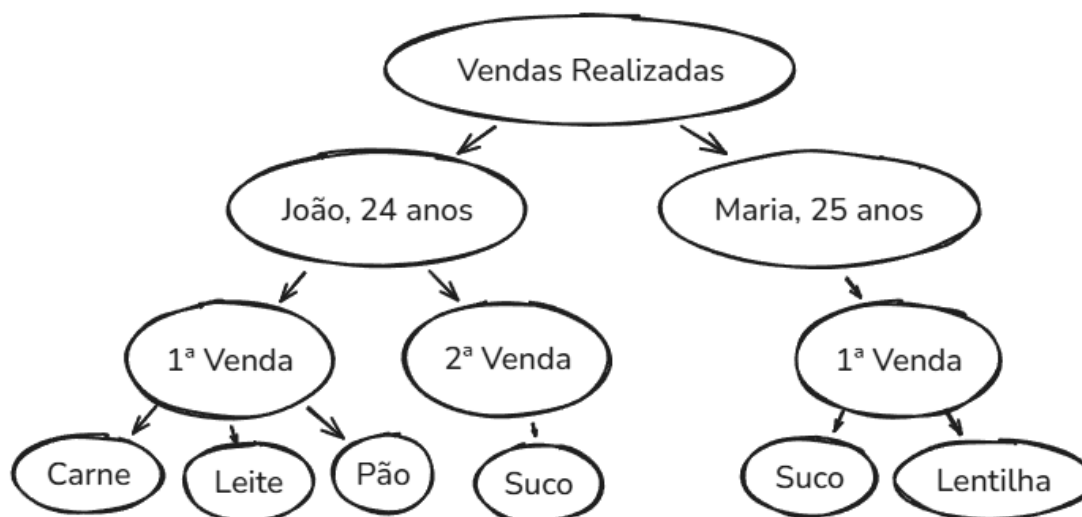
Percebam que aqui ainda estamos agnósticos ao produto a ser utilizado na prática, decisão a ser tomada na modelagem física. Por exemplo, se definirmos na modelagem lógica que nossos dados exigem um modelo relacional, será apenas com o modelo físico que escolheremos qual produto comercial utilizaremos, como MySQL, PostgreSQL ou Oracle, por exemplo.

Tipos de Modelos de Dados Lógicos

Vamos começar nosso estudo dos tipos de modelos de dados lógicos na década de 60, momento em que a IBM apresentou ao mundo o **modelo de dados hierárquico**. Nesse tipo de modelo, **organizava-se os dados em uma estrutura de árvore**.

Essa árvore é a mesmíssima estrutura que vemos em outros cursos de computação: temos um nó do topo (a raiz) e, a partir dele, os dados se ramificam para baixo. Cada dado é um nó, e as conexões entre os dados são as arestas que ligam um nó ao outro. A navegação sempre segue essa estrutura de cima para baixo, isto é, do pai para os filhos.

Por exemplo, em um banco de dados hierárquico de um e-commerce, poderíamos ter registros de consumidores, de pedidos do consumidor e de itens de cada pedido.



Percebam algo interessante: **no modelo hierárquico, necessariamente temos**, para cada nó (exceto a raiz), 1 nó pai e possíveis vários nós filhos. Em termos mais técnicos, o modelo hierárquico está restrito a modelar relacionamentos de razão de **cardinalidade 1:N**.

Bom, a maior limitação do modelo hierárquico é que o mundo real nem sempre é uma árvore, já que, por vezes, podemos ter outros tipos de relacionamentos, exigindo que um nó tenha mais de um pai. Por exemplo, sabemos que um pedido possui vários itens, mas também é possível que um mesmo item esteja em mais de um pedido, configurando um relacionamento M:N.

Em uma árvore, isso é impossível de representar diretamente, pois cada nó só pode ter um pai. Para contornar essa limitação, precisamos duplicar dados, com o mesmo item aparecendo repetido sob cada pedido que o contém. Isso gera redundância e todos os problemas decorrentes dela.

Para conseguir superar esse tipo de limitação, o consórcio CODASYL (Conference on Data Systems Languages) formalizou um novo padrão de modelo de dados lógico no ano de 1969, o **modelo em rede**. Esse tipo de modelo, **estende as capacidades do modelo hierárquico admitindo que um nó tenha múltiplos nós pais**.



Em 1970, Edgar F. Codd propôs o **modelo relacional**, no clássico artigo "A Relational Model of Data for Large Shared Data Banks". O grande diferencial desse modelo foi propor uma **nova forma de se obter os registros de dados** que desejamos.

Antes dele, nos modelos hierárquicos ou em rede, cabia ao próprio software encontrar os dados seguindo os links (ponteiros) de um registro para outro. Ou seja, era necessário que o programador soubesse realizar a navegação "passo a passo" dentro do armazenamento físico do banco de dados para conseguir obter as informações que desejava. Saibam que esse tipo de navegação é chamada de procedural.

O modelo relacional trata de mudar esse paradigma. Agora **quem deseja os dados precisa apenas descrever quais dados deseja, cabendo ao próprio banco de dados as achar**. Assim sendo, para obter dados de compras de determinado cliente, basta enviar uma declaração ao banco, como "desejo todas as compras do consumidor 'João'", que ele mesmo trata de encontrar o dado armazenado fisicamente e devolver tal informação. Essa é a chamada **navegação declarativa**.



Na prática, **implementou-se esse paradigma de navegação declarativa** em bancos de dados relacionais através de uma **linguagem** chamada **SQL**.

Além disso, foi o primeiro modelo de dados a ter fundamentação matemática, **baseando-se** em **Lógica de Primeira Ordem** (ou Lógica de Predicados) e **teoria dos conjuntos**, o que garante a criação de estruturas de dados bem formatadas e confiáveis. Graças a isso, também conseguimos representar a estrutura dos dados de forma muito simples: através de tabelas.

Ao longo dos anos, o modelo relacional se tornou o padrão na indústria, multiplicando-se os bancos de dados que obedecem a esse modelo (chamados de bancos de dados relacionais).

Nos anos 80 entramos na área de modelos "pós relacionais", momento em que foi proposto o **modelo de dados orientado a objeto**. Esse modelo lógico visava combater uma dor comum dos programadores: **alinhar a forma de programar das linguagens orientadas a objetos** (Java, C#, Python, etc.) **com a forma que bancos de dados relacionais guardam os dados**.

Como não há um mapeamento óbvio dos elementos clássicos de uma linguagem orientada a objetos (como herança, objetos aninhados, polimorfismo etc.) e os elementos de um banco de dados relacional (basicamente suas tabelas e relacionamentos), faz-se necessário uma camada para "traduzir" o código em declarações ao banco. Os bancos orientados a objetos vieram

justamente para reduzir esse atrito na interface entre “software orientado a objetos” e “banco de dados”.

No final dos anos 2000, com o crescimento da internet, das redes sociais e dos grandes volumes de dados, surgiram necessidades que o modelo relacional não atendia de forma eficiente. Aplicações como Google, Facebook e Amazon lidavam com quantidades massivas de dados, distribuídos em muitos servidores, com exigências de velocidade e flexibilidade que os bancos relacionais tradicionais tinham dificuldade em oferecer. Os bancos NoSQL surgiram como resposta a essas demandas.

Mas cuidado, porque “**NoSQL**” (Not Only SQL) **não é um modelo de dados em si**. Na realidade, trata-se de uma **categoria ampla que agrupa tudo o que não segue estritamente o modelo relacional**. Na prática, o movimento NoSQL não convergiu para um único modelo lógico, mas se diversificou em vários paradigmas, cada um adequado a um tipo diferente de problema. Vejamos alguns dos principais.

- **Chave-valor:** o modelo mais simples. Cada dado é armazenado como um par de chave e valor. Extremamente rápido para buscas diretas por chave, mas limitado para consultas complexas;
- **Orientado a documentos:** armazena dados como documentos (geralmente JSON). Cada documento pode ter estrutura diferente dos demais, oferecendo flexibilidade para dados que não se encaixam bem em tabelas rígidas;
- **Colunar:** organiza dados por colunas em vez de por linhas. Otimizado para consultas analíticas que precisam ler grandes volumes de dados de poucos atributos;
- **Orientado a grafos:** representa dados como nós e arestas, otimizado para travessias e consultas que exploram relacionamentos (ex.: redes sociais, sistemas de recomendação, detecção de fraudes).



(CEBRASPE (CESPE)/TDP (DPE RO)/DPE RO/Técnico em Informática/2022) Dentre as formas de representação lógica de um banco de dados, aquela que apresenta os dados por meio de tabelas, colunas e restrições de integridade é denominada modelo

- a) conceitual.
- b) entidade relacionamento.
- c) relacional.
- d) hierárquico.

e) de rede.

Comentários:

a) **Errada.** O modelo conceitual opera em um nível de abstração alto, representando entidades e relacionamentos do mundo real de forma independente de qualquer tecnologia, sem definir tabelas ou restrições de integridade.

b) **Errada.** O modelo entidade-relacionamento é uma forma de modelagem conceitual, utilizado na fase de projeto para representar visualmente entidades, atributos e relacionamentos, não organizando dados em tabelas.

c) **Correta.** O modelo relacional é precisamente aquele que representa os dados por meio de tabelas compostas por linhas e colunas, e que utiliza restrições de integridade para garantir a validade e consistência dos dados armazenados.

d) **Errada.** O modelo hierárquico organiza os dados em estrutura de árvore, com nós e arestas, permitindo apenas relacionamentos 1:N entre pai e filhos, sem uso de tabelas.

e) **Errada.** O modelo em rede é uma extensão do hierárquico que permite múltiplos pais por nó, também organizando os dados em grafos com ponteiros e nós, e não em tabelas com restrições de integridade.

Gabarito: C

(CEBRASPE (CESPE)/MPO/Tecnologia da Informação/Desenvolvimento de Sistemas Orçamentários/2024) Por serem uma evolução dos modelos de bancos de dados hierárquicos e em redes, os bancos de dados relacionais dispõem de caminhos predefinidos por ligações e nós para viabilizar o acesso aos dados de forma sistemática e ordenada.

Comentários:

A afirmativa atribui ao modelo relacional uma característica que pertence justamente aos modelos que ele veio superar. Nos modelos hierárquico e em rede, o acesso aos dados era feito por navegação procedural, ou seja, o programador precisava seguir caminhos predefinidos por ligações e nós para localizar os registros desejados. O modelo relacional rompeu exatamente com esse paradigma ao adotar a navegação declarativa: o usuário simplesmente descreve quais dados deseja, cabendo ao próprio banco localizá-los. Portanto, caminhos predefinidos por nós e ligações são características dos modelos anteriores, não do relacional.

Gabarito: Errado

Modelo de Dados Relacional

Conceitos

A peça fundamental do modelo relacional é o objeto matemático chamado de “**relação**”, que, no fim das contas, pode ser **visualizada como uma tabela**.

Assim sendo, uma relação (tabela) possui duas dimensões (bidimensionais): o **cabeçalho**, que define quais **atributos sobre os dados** existem (as **colunas da tabela**), e o **corpo**, que contém as instâncias da relação, **chamadas de tuplas** (as **linhas da tabela**).



Tecnicamente, o modelo relacional utiliza os termos relação, tupla e atributo e os bancos de dados relacionais (isto é, que seguem o modelo relacional) utilizam os conceitos de tabela, registro e campo.

Percebam então que hierarquia básica da informação seria:

Banco de Dados Relacional → Tabela → Registro → Campo (menor unidade de informação do banco).

De todo modo, neste curso utilizaremos cada termo de forma intercambiável, porque acredito que o ganho pedagógico é maior que esse preciosismo técnico.

Por exemplo, podemos ter a relação "Professor" com os atributos "nome" e "idade" e as instâncias "João" e "35", "Maria" e "36" e "Alex" e "34".



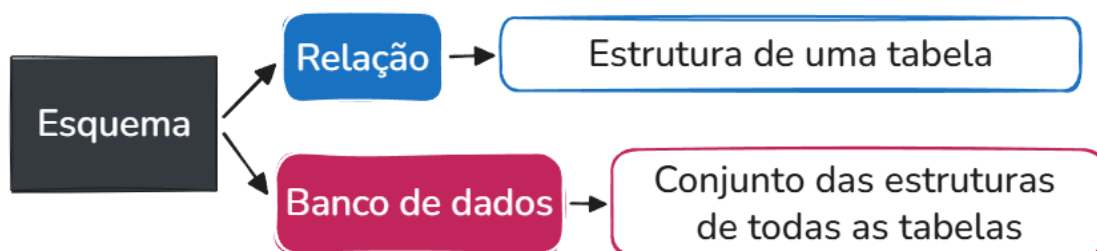
Além da forma gráfica, também podemos visualizar uma relação textualmente com o nome da relação seguido dos seus atributos em parênteses. Por exemplo, para o nosso caso teríamos a relação: Professor(nome, idade).

Continuando, se unirmos o **nome da relação com seus atributos** obtemos o **esquema** (schema) **da relação**. Percebam então que o esquema descreve a estrutura da relação, mas não seus dados (ou seja, só a "forma" da tabela).

Esquema da relação →

Professor	
nome	idade
João	35
Maria	36
Alex	34

Na prática, em banco de dados relacionais precisamos de vários esquemas de relação definidos, porque teremos várias tabelas. Esse conjunto de esquemas das relações de um banco é chamado de esquema do banco (database schema).



Saibam que a **quantidade de colunas da uma relação** representa seu **grau** (ou **aridade**) e a **quantidade de linhas** sua **cardinalidade**. Assim sendo, a relação "Professor" a seguir possui grau 2 e cardinalidade 3.

Cardinalidade = 2

Professor	
nome	idade
João	35
Maria	36
Alex	34

Aridade = 3

Pois bem, agora precisamos aprofundar um pouco mais nesses conceitos.

Saibam que, além dos nomes dos atributos, o **cabeçalho também contém os tipos de dados** que cada um deles aceita, ou seja, quais valores que de fato podem aparecer ali (como "inteiro entre 0 e 199" para "idade").



Também chamamos o tipo de dado, ou seja, o conjunto de valores permitidos para cada atributo (coluna), de **domínio**.



(CEBRASPE (CESPE)/MCom/Tecnologia da Informação e de Engenharia Senior/2022) No modelo relacional, uma tabela é chamada, formalmente, de relação, enquanto uma linha é chamada de tupla.

Comentários:

No modelo relacional, cada conceito possui uma terminologia formal própria: a tabela é chamada de relação, a linha é chamada de tupla e a coluna é chamada de atributo. Na prática dos SGBDs relacionais, esses termos são frequentemente usados de forma intercambiável com tabela, registro e campo, respectivamente, sem prejuízo de sentido.

Gabarito: Certo

(CEBRASPE (CESPE)/TCE AC/Gestão de Dados/2024) A modelagem lógica de um banco de dados inclui a determinação das tabelas necessárias e suas respectivas colunas, além da especificação dos tipos de dados para cada coluna.

Comentários:

A modelagem lógica de um banco de dados consiste justamente em definir a estrutura das tabelas necessárias para representar os dados, o que inclui determinar quais tabelas existem, quais colunas cada tabela possui e qual o tipo de dado aceito em cada coluna. Esse conjunto de definições forma o esquema lógico do banco, que descreve sua estrutura sem se preocupar com detalhes de armazenamento físico, que pertencem ao nível físico do projeto.

Gabarito: Certo

(VUNESP/CIJUN/Tecnologia da Informação/2023) Considere que em uma tabela de um banco de dados relacional exista um atributo denominado Nível. Deseja-se especificar o domínio desse atributo, o que é feito pelo estabelecimento

- a) do algoritmo de compactação de dados a ser utilizado para tal atributo em específico.
- b) do conjunto de valores possíveis que possam ser atribuídos a tal atributo.
- c) do algoritmo de criptografia a ser utilizado para codificar tal atributo.
- d) da forma normal a ser assumida pela tabela que contém tal atributo.

e) do número máximo de registros que poderão ser criados na tabela que contém tal atributo.

Comentários:

a) **Errada.** Algoritmo de compactação diz respeito ao armazenamento físico dos dados, não tendo relação com a definição do domínio de um atributo.

b) **Correta.** O domínio de um atributo é exatamente o conjunto de valores permitidos para aquela coluna. Ao especificar o domínio do atributo "Nível", estamos definindo quais valores são válidos para ele, como, por exemplo, inteiros de 1 a 5 ou um conjunto fixo de textos como "Básico", "Intermediário" e "Avançado".

c) **Errada.** Algoritmo de criptografia é um mecanismo de segurança para proteger os dados armazenados, não tendo relação com a definição do domínio de um atributo.

d) **Errada.** Forma normal é um conceito de normalização que trata da organização estrutural das tabelas para eliminar redundâncias.

e) **Errada.** O número máximo de registros de uma tabela é uma característica de capacidade de armazenamento, sem qualquer relação com o domínio de um atributo individual.

Gabarito: B

Restrições no modelo relacional

Vamos discutir um pouco sobre os tipos de restrições que podemos encontrar quando estamos projetando um banco de dados.

Primeiramente podemos enfrentar as **restrições implícitas**, que são aquelas **inerentes ao próprio modelo de dados escolhido** (por isso também são chamadas de **restrições inerentes baseadas no modelo**). Em outras palavras, restrições implícitas são as regras que não dependem do problema real enfrentado para serem criadas, pois vêm "de fábrica" do próprio formalismo do modelo escolhido.

Consequentemente, não precisamos escrever nenhum código ou comando para impor essas regras. Ou seja, não precisamos as "declarar" expressamente, visto que foram pressupostas para o modelo poder existir.

Já as **restrições explícitas** são aquelas que, **embora exigidas pelo modelo, precisamos declarar explicitamente ao criar a estrutura do nosso banco de dados para funcionarem**. Assim, precisamos "escrever código" para que o banco de dados saiba como essas regras funcionam no nosso caso específico. Ou seja, dependem do projeto e precisamos as declarar explicitamente.

No modelo relacional, declaramos expressamente essas restrições diretamente no esquema da base, por isso também são chamadas restrições de baseadas em esquema.

Um exemplo seria atribuir a um atributo "idade" como sendo um "número inteiro", ou um atributo "nome" como sendo um "conjunto de até 10 caracteres". Sabemos que o modelo relacional exige que todos os atributos tenham um tipo de dado (domínio) associado, mas, por

ser uma restrição explícita, esse tipo tem de ser declarado expressamente por nós na criação da estrutura do banco.

Eu sei que não se trata de uma aula de SQL, mas vou apresentar a seguir como declararíamos essa restrição usando uma das suas sublinguagens, a DDL¹, quando da criação da tabela "Professor".

```
-- Criando a tabela Professor
CREATE TABLE Professor (
  Nome VARCHAR(100),    -- VARCHAR(100) = atributo do tipo texto, até 100 caracteres
  Idade INT              -- INT = atributo do tipo inteiro (restrição de domínio explícita)
);
```

Há também aquelas **restrições que não são exigidas do modelo de dados utilizado e**, além disso, nós **também não conseguimos expressar de forma alguma na estrutura** (isto é, no esquema) do banco. Isso porque envolvem alguma lógica complexa do mundo real, precisando ser implementadas no código da aplicação (o programa que acessa o banco) ou escritas em algum tipo de código "extra" armazenado no banco (como triggers e procedimentos armazenados). Por isso, chamamos estas de **restrições baseadas na aplicação** ou **restrições semânticas** ou **regras de negócios**.

Por exemplo, a regra "professores que atrasam o início da aula em 5 minutos perdem o ponto" nada tem a ver com o modelo relacional e devem ser garantidas externamente ao modelo e ao esquema do banco.

Para finalizar, saibam que **cabe ao SGBD garantir as verificações de integridade do modelo**. Assim, qualquer restrição de integridade violada faz o SGBD não permitir a operação, a rejeitando (gera erro) ou aplicando alguma ação prevista (quando existir).



(CEBRASPE (CESPE)/APF/PF/2021) Em um sistema gerenciado de banco de dados, as restrições de integridade garantem que os dados possam ser armazenados, consultados e utilizados com confiabilidade.

Comentários:

As restrições de integridade existem justamente para assegurar que os dados armazenados no banco sejam válidos, consistentes e confiáveis em todas as operações. Seja impedindo valores

¹ Veremos no momento certo que o SQL é composto por sublinguagens, cada uma responsável por um aspecto diferente do banco de dados, sendo que cabe ao DDL (Data Definition Language) definir e alterar a estrutura do banco.

fora do domínio definido, tuplas duplicadas, referências inexistentes ou chaves primárias nulas, cada tipo de restrição atua como uma barreira que filtra dados inválidos antes que comprometam a base. É o SGBD o responsável por verificar e fazer cumprir essas restrições, rejeitando qualquer operação que as viole.

Gabarito: Certo

(FEPESE/AFRM/Pref Criciúma/2022) Sobre restrições no modelo relacional, associe o tipo de restrição à sua respectiva descrição.

Coluna 1 Tipo de restrição

1. Restrições implícitas
2. Restrições explícitas
3. Restrições semânticas

Coluna 2 Descrição

- () Não podem ser expressas diretamente nos esquemas do modelo de dados.
- () São restrições inerentes ao modelo de dados e baseadas nele.
- () Definidas pela DDL e expressas nos esquemas do modelo de dados.

Assinale a alternativa que indica a sequência correta, de cima para baixo.

- a) 1 • 2 • 3
- b) 1 • 3 • 2
- c) 2 • 1 • 3
- d) 3 • 1 • 2
- e) 3 • 2 • 1

Comentários:

A primeira descrição "não podem ser expressas diretamente nos esquemas do modelo de dados" corresponde às restrições semânticas (3), pois envolvem lógica do mundo real que precisa ser implementada na aplicação ou em código extra como triggers.

A segunda "são restrições inerentes ao modelo de dados e baseadas nele" corresponde às restrições implícitas (1), que vêm de fábrica com o próprio modelo e não precisam ser declaradas.

A terceira "definidas pela DDL e expressas nos esquemas do modelo de dados" corresponde às restrições explícitas (2), que precisam ser declaradas expressamente na criação da estrutura do banco, como a definição de tipos de dados.

Gabarito: D

Agora partiremos para o estudo das espécies de restrições implícitas e explícitas que existem.

Porém, de antemão, saibam que todas as restrições que estudaremos são chamadas de **restrições de estado**. Isso porque elas **definem as regras que os dados do banco devem obedecer em qualquer momento**. É como se a todo instante de tempo o banco de dados congelasse todas suas ações e verificasse se tudo está seguindo essas regras. Se todas as regras

estão sendo respeitadas naquele instante, o estado é válido. Não importa o que aconteceu antes nem o que vai acontecer depois.

Saibam que existem também as **restrições de transição**, que **se preocupam com a mudança de um estado para outro**. Elas não olham para um único instante, mas sim para as relações antes e depois de uma modificação. A pergunta aqui não é "os dados estão corretos agora?", como nas restrições de estado, mas sim "a passagem do estado anterior para o estado atual foi válida?".

Restrições implícitas do modelo relacional

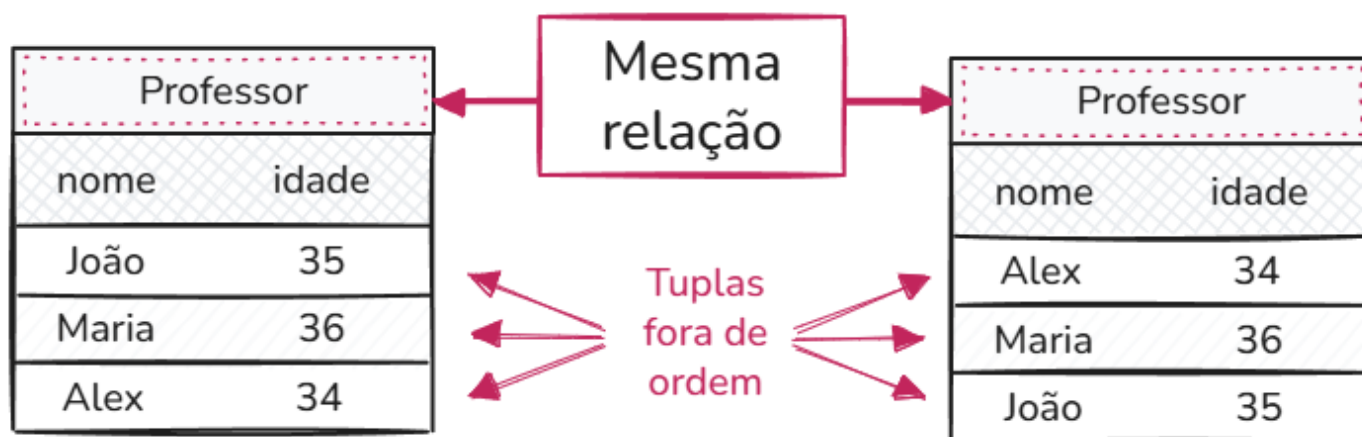
Agora estudaremos as restrições que o modelo relacional impõe por si mesmo, ou seja, as regras que nossos dados precisam seguir para estar conformes com esse modelo.

Se os dados violarem qualquer uma dessas regras, então eles simplesmente não formam uma relação válida, e não podem ser considerados descritos em um modelo relacional.

Restrição:

As tuplas de uma relação não possuem ordem.

Assim sendo, diferente de uma tabela de Excel em que temos a primeira linha, a segunda linha etc. na relação não há esse tipo de ordenação das tuplas.



O fato de não haver ordem das tuplas no modelo relacional teórico não significa que não há ordem nos registros de um banco de dados relacional.

Isso porque, na prática, os dados precisam ser gravados em algum lugar (em disco, como no HD, ou em memória, como na RAM). E qualquer meio de

armazenamento é sequencial por natureza: um byte vem antes de outro, um bloco de disco vem antes de outro.

Ou seja, não existe como armazenar dados fisicamente sem que haja uma ordem entre eles. Mesmo que o modelo teórico diga que a ordem não importa, o armazenamento físico inevitavelmente impõe uma.

Restrição:

Os atributos de uma tupla possuem ordem.

Assim como toda tupla na matemática, as do modelo relacional também são ordenadas, ou seja, seus atributos possuem ordem. Na prática, isso significa que ao definirmos os atributos de uma relação, a tupla criada obedecerá exatamente a ordem deles.

Por exemplo, se temos uma relação "Pessoa" com atributos "nome", "tamanho da camisa" e "idade", precisamos definir uma tupla seguindo essa ordem estrita: (Jorge, M, 32).

Professor	
nome	idade
Alex	34
Maria	36
João	35

Atributos
possuem
ordem



Ocorre que o próprio Codd, em seu artigo original, reconheceu que essa dependência da ordem das colunas era uma limitação ao modelo relacional.

Então, nesse mesmo artigo, ele propôs uma definição alternativa para essa restrição em que a ordem das colunas não importa, desde que identifiquemos cada coluna pelo seu nome de atributo.

Assim, nessa outra definição conseguiríamos definir os atributos das tuplas fora de ordem, desde que especificando os seus nomes. Por exemplo, {"idade":32, "nome": Jorge, "tamanho da camisa ": M}.

Restrição:

Cada valor de atributo deve ser atômico.

A definição do modelo relacional exige que cada valor de atributo deve ser atômico, isto é, indivisível. Em outras palavras, cada coluna guarda um único valor por linha.

Então não podemos ter um atributo "telefones" aceitando múltiplos valores como "98888-1111, 99999-2222" ou um atributo "endereço" composto por valores de vários outros atributos como "Rua das Flores, 123, 65000-000" em que há três informações distintas "endereço", "número" e "CEP", por exemplo.

Restrição:

Um valor desconhecido, inexistente, indisponível, não aplicável ou indefinido deve ser representado por NULL.

O NULL é a forma padronizada que o modelo relacional oferece para representar a ausência de valor. Não importa o motivo pelo qual o valor não existe, o modelo usa NULL para todos os casos.

Por exemplo, se criarmos uma relação "Professor" com um atributo "Identificação de Estrangeiro" para armazenar o número do documento de um professor não brasileiro, teremos que esse campo vale NULL para professores brasileiros.

Professor		
nome	idade	id estrangeiro
Alex	34	NULL
Maria	36	15A147
João	35	NULL

Valor ausente:
Alex e João são
brasileiros

Restrição:

Não há tuplas duplicadas.

Na matemática, se temos um conjunto de elementos, então sabemos, pela própria definição de conjunto, que não há elementos repetidos. No modelo relacional, como definimos uma relação (tabela) como um conjunto de tuplas, então sabemos que não pode haver tuplas repetidas.

Restrição:

Uma relação representa fatos sobre entidades e sobre seus relacionamentos.

Quando criamos o esquema de uma relação (ou seja, definimos seu nome e seus atributos), estamos basicamente estruturando uma declaração sobre o mundo real.

Por exemplo, ao criar a relação "Professor" com os atributos "nome" e "idade" estamos basicamente falando que "Existem, no mundo real, professores com nome ... e idade ...".

Percebam que essa estrutura não é capaz de ser avaliada, já que há lacunas a serem preenchidas. Assim, podemos criar tuplas para essa relação para declararmos de fato algo concreto.

Por exemplo, ao registrarmos "Lucas" e "18" para a relação "Professor", estamos afirmando um fato: *"Existe, no mundo real, um professor com nome Lucas e idade 18"*.



Consequentemente, como um banco de dados se trata de uma coleção de esquemas com suas próprias instâncias, então, o banco de dados inteiro se trata de uma coleção de fatos sobre o mundo real.

Acontece que nem toda relação representa uma entidade do mundo, como um "Professor", já que também podem representar relacionamentos entre essas entidades.

Por exemplo, se definirmos a relação "Matrícula" com os atributos "nome do aluno" e "nome do curso", podemos instanciar tuplas que representam um aluno matriculado em um determinado curso.

Pois bem, para essa explicação pouco importa se a tabela trata-se de uma entidade ou um relacionamento, já que em ambos os casos teremos afirmações sobre o mundo real: ao criar "Joana" e "Ciência de Dados" em "Matrícula", estamos afirmando que *"Joana está matriculada em Ciência de Dados"*.



(CEBRASPE (CESPE)/Ana (MPE RO)/MPE RO/Sistemas/2023) Na modelagem de dados, o objeto que representa uma entidade do mundo real e contém atributos para descrever suas características é

- a) a função.
- b) a chave estrangeira.

- c) o procedimento.
- d) a visão.
- e) a tabela.

Comentários:

A questão descreve exatamente o papel de uma tabela (relação) no modelo relacional. Ao criar uma relação com seu nome e seus atributos, estamos estruturando uma declaração sobre o mundo real, representando entidades e suas características. Cada atributo descreve uma propriedade da entidade, e cada tupla registra um fato concreto sobre ela.

As demais alternativas estão descartadas por definição: funções e procedimentos são estruturas de código executável, não representações de entidades; a chave estrangeira é um atributo que estabelece ligação entre tabelas; e a visão é uma tabela virtual que não armazena dados próprios, sendo derivada de outras tabelas.

Gabarito: E

Restrições explícitas do modelo relacional

Agora vamos avançar pelas restrições que dependem das nossas decisões como projetistas de um modelo relacional. Ou seja, aquelas restrições exigidas pelo modelo relacional, porém que dependem de nós para sua aplicação concreta um esquema de banco de dados específico.

Restrições de domínio

Sabemos que um domínio é simplesmente um conjunto de valores permitidos para um atributo, como, por exemplo, "ano" sendo um "número inteiro de 0 a 9999" ou "sexo" sendo "M" ou "F".

Pois bem, a **restrição de domínio** (também chamada restrição de integridade de domínio) **exige que cada atributo de uma relação tenha um domínio e que os valores inseridos pertençam a ele.**

Assim sendo, graças à integridade de domínio, cada coluna de uma tabela tem necessariamente um conjunto pré-definido de valores aceitáveis atribuídos. Ou seja, todo valor inserido naquela coluna deve pertencer a esse conjunto.

É justamente aqui que entra nosso exemplo de que "idade" deve ser um "número inteiro", já apresentado anteriormente.

```
-- Criando a tabela Professor
CREATE TABLE Professor (
  Nome VARCHAR(100),    -- VARCHAR(100) = atributo do tipo texto, até 100 caracteres
  Idade INT              -- INT = atributo do tipo inteiro (restrição de domínio explícita)
);
```

Com isso, **restringimos que dados sem sentido entrem no banco**, como uma idade negativa, por exemplo.

Somado-se a essa restrição a restrição implícita "cada valor de atributo deve ser atômico", temos que podemos definir como domínio de nossos dados apenas coisas elementares, como um inteiro (p.ex.: 25), um texto simples (p.ex.: "São Luís"), uma data (p.ex.: 2024-01-15), um valor lógico (p.ex.: true) etc., e nunca uma lista (p.ex.: ["João", "Maria", "Pedro"]) ou uma estrutura complexa (p.ex.: {rua: "Rua A", numero: 5}).



(CEBRASPE (CESPE)/Ana TI (FUB)/FUB/2023) De acordo com a restrição de domínio de uma relação, em uma chave estrangeira, o valor do campo tem que coincidir com uma chave primária de uma tabela referenciada.

Comentários:

A exigência de que o valor de uma chave estrangeira corresponda a uma chave primária da tabela referenciada é a definição de integridade referencial, e não de restrição de domínio. A restrição de domínio tem um escopo bem mais simples e restrito: ela exige apenas que o valor inserido em um atributo pertença ao conjunto de valores previamente definido para aquele atributo, como exigir que o campo "idade" receba apenas números inteiros. Cada restrição opera em um nível diferente, a de domínio atua sobre os valores individuais de cada atributo, enquanto a referencial atua sobre a consistência entre tabelas.

Gabarito: Errado

Restrições de chave

Já sabemos que uma relação é um conjunto de tuplas, e que o modelo relacional não aceita existir duas tuplas idênticas. Ou seja, se olharmos para todos os atributos de duas tuplas, pelo menos um dos seus valores deve ser diferente, sempre.

Pois bem, para garantir isso, podemos estabelecer o primeiro conceito importante dessa seção, o de superchave.

**Superchave se trata de um conjunto de atributos
que garante que a tupla seja única.**

Por exemplo, em uma relação "Professor" com atributos "CPF", "RG", "nome" e "CEP", podemos estabelecer a superchave mais óbvia possível (chamada de **superchave trivial**): o **conjunto de todos os atributos**.

Isso porque, necessariamente, nunca teremos dois professores com o mesmo CPF, RG, nome e CEP, ao mesmo tempo.

Professor			
CPF	RG	nome	CEP
123	321	Alex	11111
456	654	Maria	22222
789	987	João	33333

Superchave

Porém não paramos por aí. Temos subconjuntos desses atributos que também são superchaves, já que também garantem que a tupla seja única. Por exemplo, o conjunto de atributos "CPF", "RG" e "nome" se trata de outra superchave. Como não poderia deixar de ser, "CPF" e "RG" também formam outra superchave.

Agora percebam algo interessante: "CPF" e "RG" sozinhos também são superchaves! Afinal de contas, são dois conjuntos de atributos (com 1 elemento apenas cada um) que garantem que cada registro seja único, uma vez que é impossível termos dois registros na tabela com "CPF" ou "RG" repetidos.

Professor			
CPF	RG	nome	CEP
123	321	Alex	11111
456	654	Maria	22222
789	987	João	33333

Superchaves

Superchave

Superchave

Mas "nome" e "CEP" formam uma superchave? **Não!** Isso porque é possível que duas pessoas tenham mesmo nome e mesmo CEP.

Agora percebam que nessa explicação tivemos muita redundância de informação, já que quaisquer atributos em conjunto com "CPF" ou "RG" formam uma superchave. Então podemos simplificar isso estabelecendo o conceito de chave, também chamada de superchave mínima.

Uma chave é uma superchave que não pode ser reduzida pela remoção de um atributo.

Ou seja, a **chave nada mais é que uma superchave da qual não podemos remover nenhum atributo sem perder a propriedade de unicidade das tuplas.**

No nosso exemplo, nossas chaves para "Professor" seriam ou o "CPF" ou o "RG", porque se removermos o único atributo que cada uma delas contém (um o CPF, a outra o RG), sobra um conjunto vazio que obviamente não identifica nada.

Professor			
CPF	RG	nome	CEP
123	321	Alex	11111
456	654	Maria	22222
789	987	João	33333

Chaves

Percebam que o conjunto "CPF" e "nome" não pode ser uma chave, apesar de ser uma superchave. E por quê? Porque podemos remover "nome" e o conjunto restante (só "CPF") continua sendo superchave. Isso significa que "nome" era redundante, e "CPF" e "nome" não é mínimo.



Muitas vezes um único atributo não é suficiente para garantir a unicidade da tupla, o que nos obriga a unirmos dois ou mais atributos para formar a chave.

Por exemplo, na relação "Apartamento" com os atributos "número", "bloco" e "andar", "número" sozinho não garante unicidade, já que o apartamento "101" pode existir tanto no bloco "A" quanto no bloco "B". Da mesma forma, "bloco" sozinho não serve, pois cada bloco tem vários apartamentos. Porém, a combinação de "bloco" e "número" identifica cada apartamento de forma única: só há um apartamento "101" no bloco "A".

Nesses casos, em que **apenas a combinação de dois ou mais atributos consegue identificar as tuplas**, dizemos que estamos trabalhando com uma **chave composta**.

Percebam então que nossa relação "Professor" possui duas chaves, então as chamamos de **chaves candidatas**. Isso porque, para criarmos nosso modelo lógico, **precisamos escolher**

uma das chaves (candidatas) para de fato exercer o papel de identificação unívoca² de cada tupla da relação.

Professor			
CPF	RG	nome	CEP
123	321	Alex	11111
456	654	Maria	22222
789	987	João	33333

Chaves candidatas

Ao escolhermos uma chave candidata para ser a chave "oficial" usada para identificar as linhas da tabela, algumas coisas acontecem:

1. Essa chave candidata automaticamente é **promovida** para ser a **chave primária**;
2. As **outras chaves candidatas** se tornam as **chaves alternativas** (também chamadas de chaves únicas ou secundárias);
3. Mudanças visuais na apresentação da **chave primária**, já que são **apresentadas no esquema sublinhadas**.

Assim sendo, se escolhermos a chave candidata "CPF" como chave primária, então a chave candidata "RG" se torna uma chave alternativa (ou única).

Professor			
<u>CPF</u>	RG	nome	CEP
123	321	Alex	11111
456	654	Maria	22222
789	987	João	33333

Chave primária

Chave alternativa (única)

Agora vamos apresentar uma definição formal de chave primária.

Uma chave primária é o conjunto de atributos (colunas) efetivamente escolhido para identificar e distinguir de forma confiável cada tupla (registro) individual em uma relação (tabela) no banco de dados relacional.

² Unívoco é aquilo que admite apenas uma interpretação, há somente um único significado; sem teor ambíguo; inequívoco.

Na prática, para definição da chave primária, **o projetista do banco de dados acaba escolhendo um atributo que já existe naturalmente nos dados e que já possui a propriedade de unicidade**. Ou seja, escolhe um atributo que naturalmente possui valores únicos devido ao próprio problema real, como "CPF" quando estamos querendo identificar pessoas ou "chassi" para carros. Esse tipo de atributo recebe o nome pela literatura de **chave natural** (natural key).

No entanto, usar uma chave natural pode trazer problemas, porque nem sempre temos controle sobre como essa informação é definida.

Vejam o exemplo do CNPJ, que identifica de forma única cada empresa no Brasil. Durante décadas, o CNPJ foi um número de exatamente 14 dígitos. Com base nisso, muitos sistemas foram projetados armazenando esse campo como um número inteiro de 14 posições.

O problema é que, a partir de julho de 2026, o governo, que é quem define as regras do CNPJ, decidiu que ele passaria a ser alfanumérico, ou seja, agora pode conter tanto letras quanto números.

Então, todos sistemas que assumiam que o CNPJ seria "sempre" numérico precisaram ser adaptados para lidar com essa mudança.

Por essas e outras, o **projetista pode preferir criar um atributo novo na tabela que não tem nenhum significado no mundo real, mas que sempre será único, para ser a chave primária**. Assim, está sob controle interno da organização, nunca precisando mudar e existindo exclusivamente para servir como identificador no banco de dados. Nesse caso, dizemos que a tabela possui uma **chave substituta** (surrogate key). Na prática, esse tipo de atributo geralmente é chamado (ou iniciado) "id" (de identificador).

Professor				
<u>id_professor</u>	CPF	RG	nome	CEP
1	123	321	Alex	11111
2	456	654	Maria	22222
3	789	987	João	33333

Chave substituta





(CEBRASPE (CESPE)/AJ (PGDF)/PG DF/Analista de Sistemas/Suporte e Infraestrutura/2021) Em banco de dados, uma superchave se caracteriza por um conjunto de um ou mais atributos que permitem identificar uma única entidade em um conjunto de entidades.

Comentários:

Uma superchave é exatamente um conjunto de um ou mais atributos capaz de identificar de forma única cada tupla em uma relação. O conjunto pode ter um único atributo, como o CPF sozinho, ou múltiplos atributos combinados. O que define a superchave não é o tamanho do conjunto, mas sim a propriedade de garantir a unicidade da identificação.

Gabarito: Certo

(CEBRASPE (CESPE)/AFRE (SEFAZ RS)/2019) No modelo relacional, a afirmação "Duas tuplas distintas, em qualquer estado da relação, não podem ter valores idênticos para os atributos na chave" é

- a) falsa.
- b) uma restrição de domínio do modelo.
- c) uma propriedade exclusiva do modelo objeto-relacional.
- d) uma condição que deverá estar explícita na representação dos atributos de uma tupla.
- e) uma propriedade de chave do modelo.

Comentários:

A afirmativa descreve precisamente a restrição de chave do modelo relacional. Ela não se trata de uma restrição de domínio, que diz respeito aos valores válidos para cada atributo individualmente, nem é exclusiva do modelo objeto-relacional. É uma propriedade fundamental do próprio modelo relacional, decorrente do fato de que uma relação é um conjunto de tuplas, e conjuntos não admitem elementos repetidos. Portanto, por definição, nenhuma chave pode ter seus valores duplicados em duas tuplas distintas.

Gabarito: E

(CEBRASPE (CESPE)/INPI/Desenvolvimento e Manutenção de Sistemas/2024) O fato de haver um campo de chave estrangeira com o valor NULL não viola a restrição de chave estrangeira.

Comentários:

A restrição de integridade referencial exige que o valor de uma chave estrangeira corresponda a um valor existente na chave primária da tabela referenciada ou seja nulo. O valor nulo indica simplesmente que aquela tupla não está associada a nenhum registro da tabela pai, o que é uma situação válida e não constitui uma referência inválida. A violação ocorreria apenas se o valor fosse não nulo e não existisse na tabela referenciada.

Gabarito: Certo

Restrições de integridade de entidade

A **restrição de integridade de entidade** exige que a **chave primária não admita valores nulos**, isto é, seu valor em cada tupla não pode ficar vazio.



Percebam então que, embora as **chaves únicas (alternativas)** também garantam que não haja valores duplicados, elas **permitem valores nulos**.

Para implementarmos a restrição de integridade de entidade na criação da tabela basta utilizarmos a sintaxe "PRIMARY KEY" do SQL, que já restringe ao atributo ser único (ou seja, não haverá dois registros com o mesmo valor) e não nulo (isto é, "not null", aquele campo nunca pode ficar vazio).

Vamos aproveitar o momento para apresentar a sintaxe "UNIQUE", que nos permite definirmos as chaves únicas.

```
-- Criando a tabela Professor
CREATE TABLE Professor (
  CPF CHAR(11) PRIMARY KEY, -- Chave primária (candidata escolhida).
  RG VARCHAR(20) UNIQUE, -- Chave alternativa (candidata não escolhida)
  Nome VARCHAR(100), -- VARCHAR(100) = texto de até 100 dígitos
  CEP CHAR(8) -- CHAR(8) = texto de exatos 8 dígitos
);
```

Propriedade	Chave primária	Chave única (alternativa)
Valores duplicados	Não permite	Não permite
Valores nulos	Não permite	Permite
Quantidade por tabela	Apenas uma	Várias
Papel	Identificador oficial da tupla	Garantia adicional de unicidade



(CEBRASPE (CESPE)/ERAC (ANAC)/ANAC/Qualquer Área de Formação/2024) Em bancos de dados relacionais, a restrição de integridade de entidade estabelece-se entre duas relações e é usada para manter a consistência entre as tuplas nessas duas relações.

Comentários:

A afirmativa descreve a restrição de integridade referencial, não a de entidade. A integridade referencial é aquela que se estabelece entre duas relações, garantindo a consistência entre tuplas de tabelas diferentes por meio de chaves estrangeiras. A integridade de entidade, por sua vez, opera dentro de uma única tabela e tem um propósito bem mais simples e direto: exigir que a chave primária de cada tupla nunca seja nula, garantindo que todo registro possa ser identificado de forma única.

Gabarito: Errado

(CEBRASPE (CESPE)/TCE MS/Tecnologia da Informação/2025) Em um SGBD relacional, a restrição que garante que todos os valores em uma coluna sejam distintos entre si e que permite valor NULL é denominada

- a) cardinalidade.
- b) chave primária.
- c) índice.
- d) chave única.
- e) chave estrangeira.

Comentários:

A descrição aponta dois requisitos simultâneos: garantir que todos os valores de uma coluna sejam distintos e, ao mesmo tempo, permitir valores nulos. Esse conjunto de características é exclusivo da chave única. A chave primária fica descartada porque, embora também impeça duplicatas, não permite valores nulos. As demais alternativas, cardinalidade, índice e chave estrangeira, não têm como função primária garantir a unicidade de valores em uma coluna.

Gabarito: D

Restrições de integridade referencial

Para explicar o conceito de restrição de integridade referencial, antes precisamos aprender do que se trata uma chave estrangeira.

Pois bem, em algumas situações faz-se necessário estabelecer uma ligação entre duas tabelas de um banco de dados. Por exemplo, vamos supor que, além da relação "Professor", também temos a relação "Disciplina" para armazenar os assuntos passíveis de serem lecionados.

Professor			
<u>CPF</u>	RG	nome	CEP
123	321	Alex	11111
456	654	Maria	22222
789	987	João	33333

Disciplina	
<u>id_disciplina</u>	nome
1	Matemática
2	Português
3	Física

Ocorre que cada professor pode lecionar uma disciplina e cada disciplina é sempre lecionada por um professor. Ou seja, precisamos criar um tributo que servirá como um “elo” entre determinado professor e a disciplina que leciona. Para isso, podemos adicionar na tabela “Disciplina” um campo “RG_professor”.

Assim sendo, sempre quando encontrarmos um número de RG no atributo “RG_professor” saberemos exatamente qual professor está lecionando aquela disciplina. Esse campo “RG_professor” é justamente o que chamamos de chave estrangeira.

Disciplina		
<u>id_disciplina</u>	nome	rg_professor*
1	Matemática	654
2	Português	987
3	Física	NULL

Chave
estrangeira

→ Maria leciona matemática

→ João leciona português

→ Ninguém leciona física



Percebam que indicamos o atributo “rg_professor” como chave estrangeira inserindo um asterisco *) ao lado do atributo. Também podemos colocar ao lado dele um *hashtag* (#) ou escrever “FK” (de *foreign key*).

Entendida a explicação, vamos para nossa definição formal de chave estrangeira.

Uma chave estrangeira é um conjunto de atributos em uma tabela que se refere a uma chave de outra tabela, ligando essas duas tabelas.



Originalmente Codd instituiu que uma chave estrangeira deveria possuir os mesmos valores da chave **primária de outra tabela.**

Assim sendo, bancas como o CESPE, cobram que a chave estrangeira de uma relação se referencia à chave primária de outra. Tomem cuidado.

Vale saber que a literatura chama a **tabela** que **contém a chave estrangeira** de **relação que referencia** (ou filha) e a que **contém a chave** de **relação referenciada** (ou pai).

Relação referenciada

Professor			
<u>CPF</u>	RG	nome	CEP
123	321	Alex	11111
456	654	Maria	22222
789	987	João	33333

Relação que referencia

Disciplina		
<u>id_disciplina</u>	nome	rg_professor*
1	Matemática	654
2	Português	987
3	Física	NULL

Pois bem, feita toda essa explicação, finalmente vamos aprender a **restrição de integridade referencial**. Segundo ela, exige-se que **a chave estrangeira ou corresponda a um valor que já existe como chave na tabela referenciada, ou seja nula**, indicando que aquela tupla simplesmente não está associada a nenhuma tupla da outra tabela.

Assim sendo, é impossível haver no banco de dados um valor de chave estrangeira que não exista no atributo chave da tabela pai.

Finalmente, para implementarmos a restrição de integridade referencial na criação da tabela basta utilizamos a sintaxe "FOREIGN KEY" informando a coluna na tabela atual (a tabela filha) seguido de "REFERENCES" apontando para a coluna da outra tabela (a tabela pai Professor), que normalmente é PRIMARY KEY ou UNIQUE.

```
-- Criando a tabela Disciplina
CREATE TABLE disciplina (
  id_disciplina INT PRIMARY KEY,
  nome VARCHAR(100),
  rg_professor VARCHAR(20),
  -- Definição da chave estrangeira a seguir: garante que apenas valores que existem no atributo "RG" da
  tabela "Professor" possam ser adicionados em "rg_professor"
  -- Isso cria o relacionamento entre disciplina e professor
  FOREIGN KEY (rg_professor) REFERENCES professor(RG)
);
```



(CEBRASPE (CESPE)/TCE AC/Gestão de Dados/2024) A integridade referencial é uma propriedade que garante que os valores de uma chave estrangeira em uma tabela correspondem a valores existentes na chave primária da tabela referenciada.

Comentários:

A restrição de integridade referencial exige que todo valor presente em uma chave estrangeira corresponda a um valor já existente na chave primária da tabela referenciada, impedindo que existam no banco referências a registros inexistentes. Vale lembrar que a única exceção permitida é o valor nulo na chave estrangeira, indicando que aquela tupla simplesmente não está associada a nenhum registro da tabela pai, situação que não viola a restrição, pois não constitui uma referência inválida.

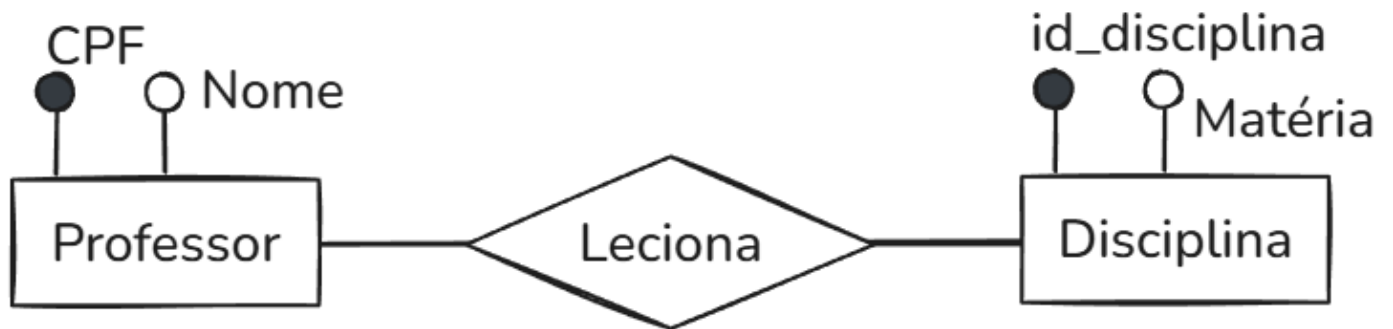
Notem a CESPE considerando que a chave estrangeira precisa se referenciar a uma chave primária, conforme Codd orientou.

Gabarito: Certo

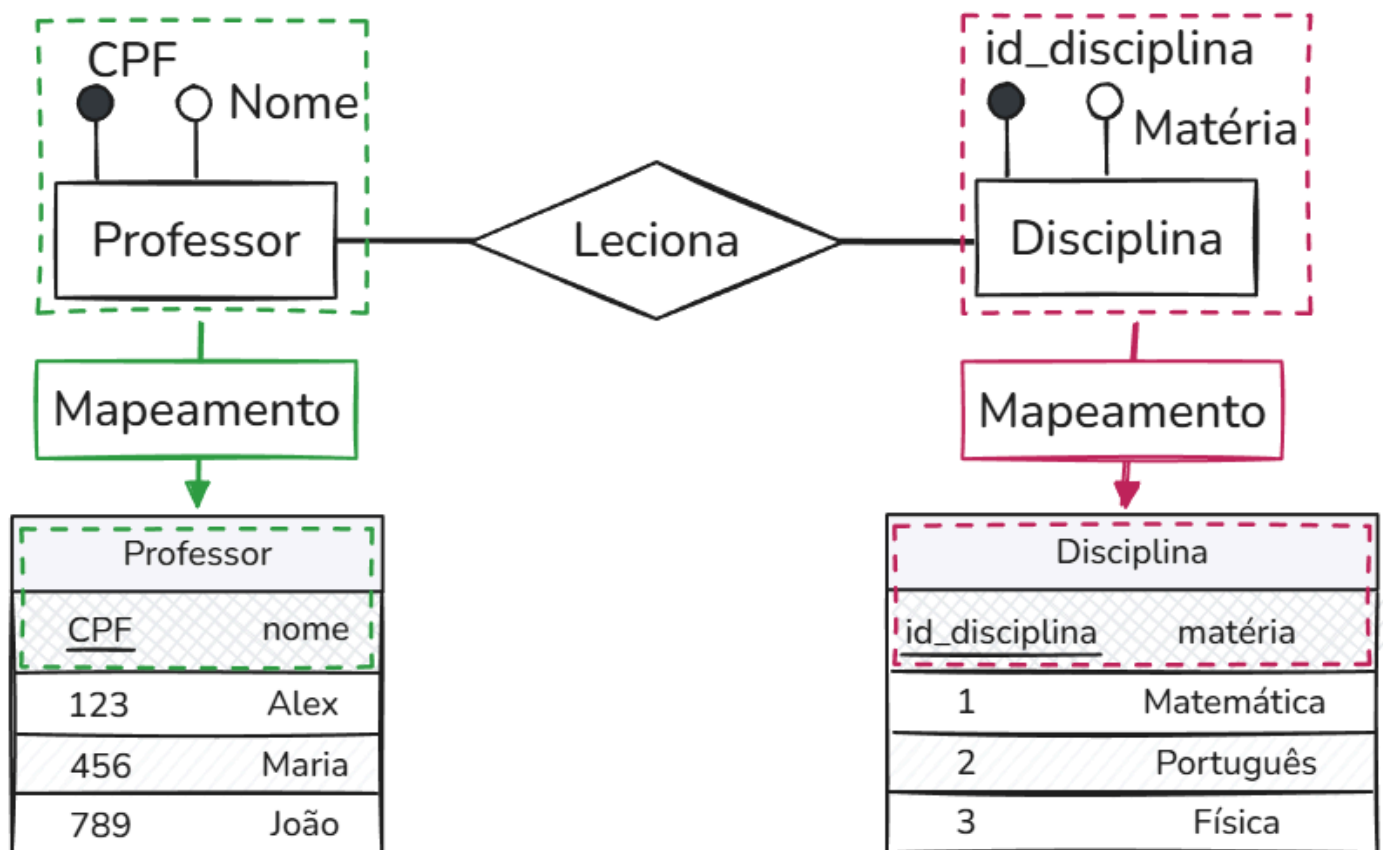
Mapeamento do Modelo ER para o Modelo Relacional

Quando projetamos um banco de dados, geralmente **começamos com um modelo Entidade-Relacionamento (ER)**, que é mais visual e conceitual. **Depois, precisamos transformá-lo em tabelas no modelo relacional**. Esse processo é chamado de **mapeamento**.

Antes de nos aprofundarmos, vamos usar como base o diagrama ER a seguir. Aqui temos as entidades "Professor", com atributos "CPF" e "nome", e "Disciplina", com atributos "id_disciplina" e "assunto", ligadas pelo relacionamento "Leciona".



Percebam que mapear as entidades do diagrama ER acaba sendo uma tarefa bem simples, visto, regra geral, que cada uma delas vira uma tabela no modelo relacional, e seus atributos viram colunas. Vejamos.



Mas e "Leciona", professor? Bom, as formas de tradução desse relacionamento serão justamente o que estudaremos nesta seção, visto que cada tipo de mapeamento depende da razão de cardinalidade entre as entidades.



(CEBRASPE (CESPE)/EMBRAPA/Gestão da Informação/2025) Durante o projeto lógico de banco de dados, os modelos entidade-relacionamento são convertidos em esquemas de tabelas, que podem ser implementados em sistemas de gerenciamento de banco de dados.

Comentários:

A afirmativa descreve corretamente o processo de mapeamento do modelo ER para o modelo relacional. O modelo ER é um modelo conceitual, visual e abstrato, utilizado na fase de projeto para representar as entidades e seus relacionamentos. O resultado desse mapeamento são esquemas de tabelas, que constituem o projeto lógico do banco de dados e podem então ser implementados em um SGBD. A sequência conceitual → lógico → implementação é justamente a ordem natural do projeto de banco de dados.

Gabarito: Certo

(CEBRASPE (CESPE)/TJ TRF6/TRF 6/Apoio Especializado/Suporte Técnico/2025) O modelo entidade-relacionamento é derivado do modelo lógico e apresenta detalhes sobre o armazenamento interno dos dados.

Comentários:

A afirmativa erra em ambos os seus elementos. Primeiro, a relação de derivação está invertida: o modelo ER não é derivado do modelo lógico, pelo contrário, é o modelo lógico que é derivado do modelo ER, pois o ER é o ponto de partida conceitual e o modelo relacional (lógico) é seu produto após o mapeamento. Segundo, detalhes sobre o armazenamento interno dos dados pertencem ao nível físico do banco de dados, não ao modelo ER, que opera no nível conceitual e é justamente caracterizado pela abstração e independência de como os dados são fisicamente armazenados.

Gabarito: Errado

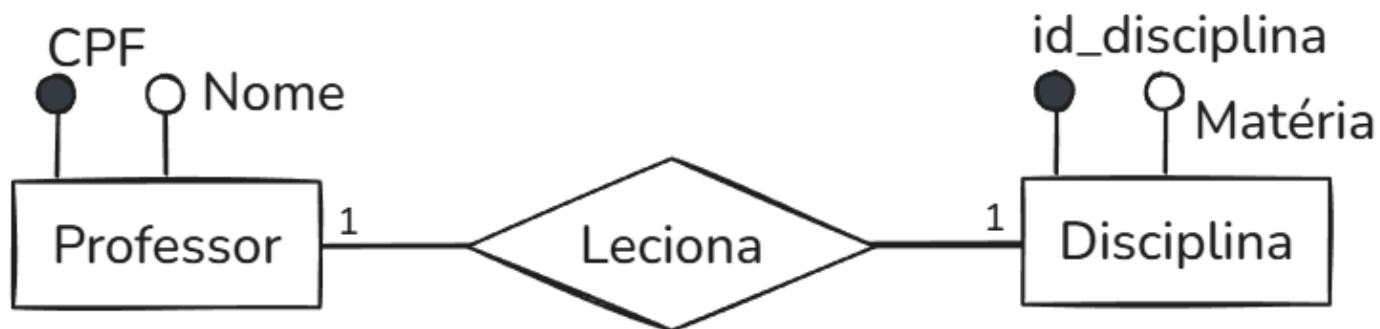
Mapeando de relacionamentos binários 1:1

Vamos estudar agora como mapear relacionamentos onde cada entidade de um lado se associa a, no máximo, uma entidade do outro lado. Ou seja, possuem razão de cardinalidade de 1:1.



Na razão de cardinalidade temos apenas as cardinalidades máximas do relacionamento, ou seja, se na razão de cardinalidade temos "1", então a cardinalidade pode ser (0,1) ou (1,1).

Vamos supor então que temos duas entidades "Professor", com atributos "CPF" e "nome", e "Disciplina", com atributos "id_disciplina" e "assunto", sendo que cada professor pode lecionar no máximo 1 disciplina e cada disciplina pode ser lecionada por no máximo 1 professor.

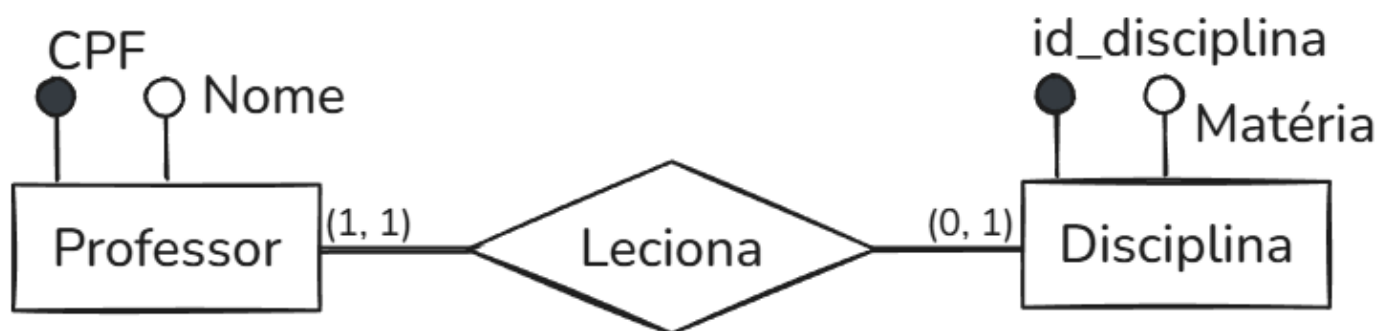


Bom, segundo Navathe, para garantirmos a existência do relacionamento entre as entidades podemos utilizar a **técnica de chave estrangeira**: **basta pegar a chave de uma tabela e colocá-la como chave estrangeira na outra**.

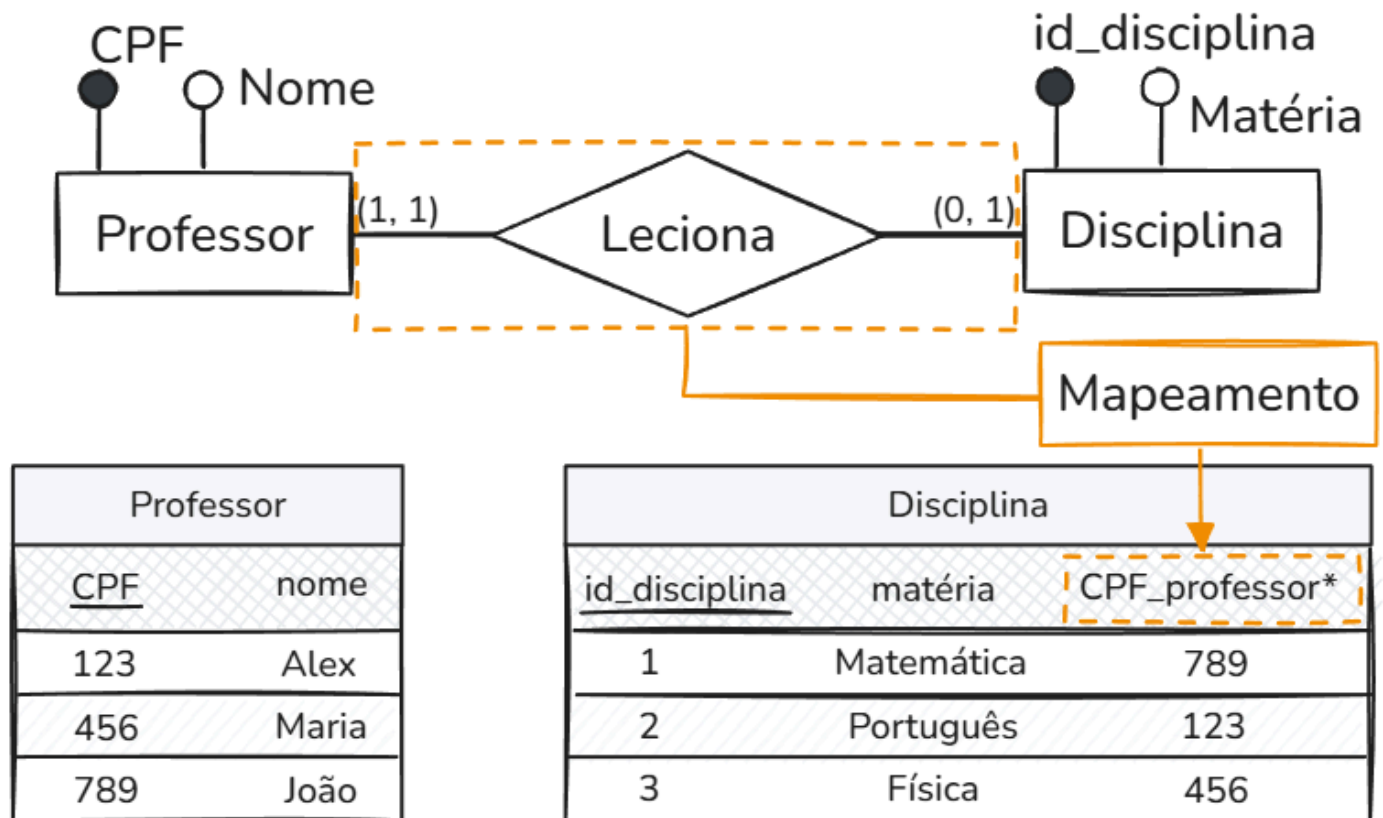
Não é preciso criar nada além disso, já que um único atributo adicional já resolve a associação.

Porém, segundo o autor, devemos **preferir inserir o novo atributo na tabela** que não pode existir se não estiver ligada ao menos uma vez pelo relacionamento, ou seja, a **que tenha participação mínima igual a 1** na outra (a chamada participação total).

Por exemplo, se supormos que “cada professor pode não lecionar nenhuma disciplina” e que “uma disciplina precisa ser lecionada por pelo menos 1 professor”, temos que a cardinalidade se torna:



Assim sendo, a recomendação é colocar a chave estrangeira na tabela “Disciplina” porque, se toda tupla está associada a pelo menos uma tupla de “Professor”, então a chave estrangeira sempre terá um valor preenchido. Isso evita valores nulos, o que é desejável em um bom projeto de banco de dados.



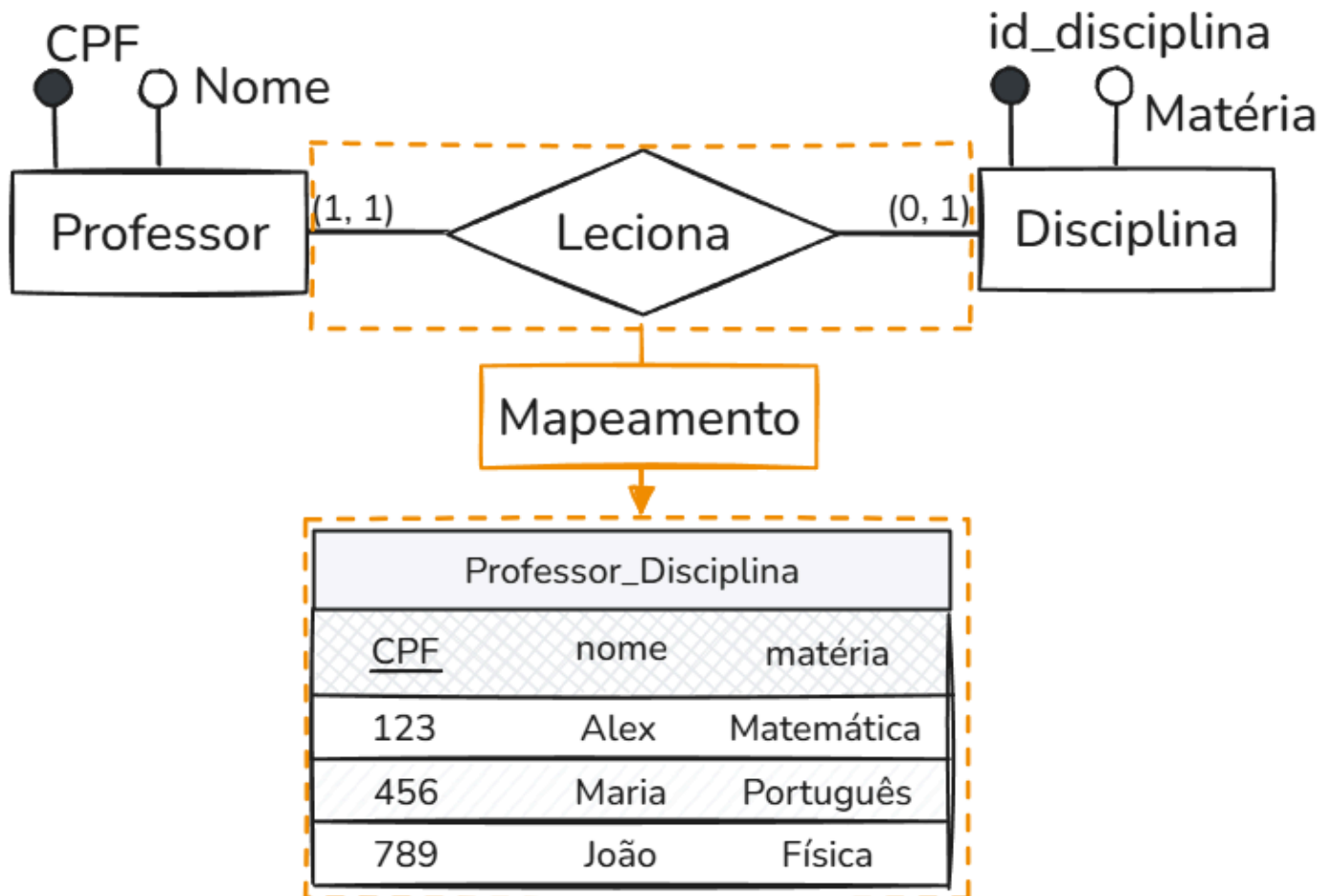
O próprio Navathe reconhece que a escolha de qual tabela recebe a chave estrangeira não é obrigatória, visto que tecnicamente funciona em qualquer direção. Ocorre que, se a chave estrangeira for colocada na tabela com relacionamento de cardinalidade mínima igual a 0 (participação parcial), algumas tuplas podem não ter chave estrangeira, deixando esse atributo vazio. Isso desperdiça espaço e torna os dados menos limpos.



Se o relacionamento possui atributos próprios, eles são colocados na mesma tabela que recebeu a chave estrangeira.

Outra forma de solucionarmos esse problema é através da **técnica de relação mesclada: unimos todos atributos em uma tabela só.**

Nesse caso, a associação entre as entidades deixa de ser representada por uma chave externa e passa a ser representada pela própria coexistência dos dados na mesma tupla, que contém todos relacionamentos (dos atributos e do relacionamento, se for o caso).

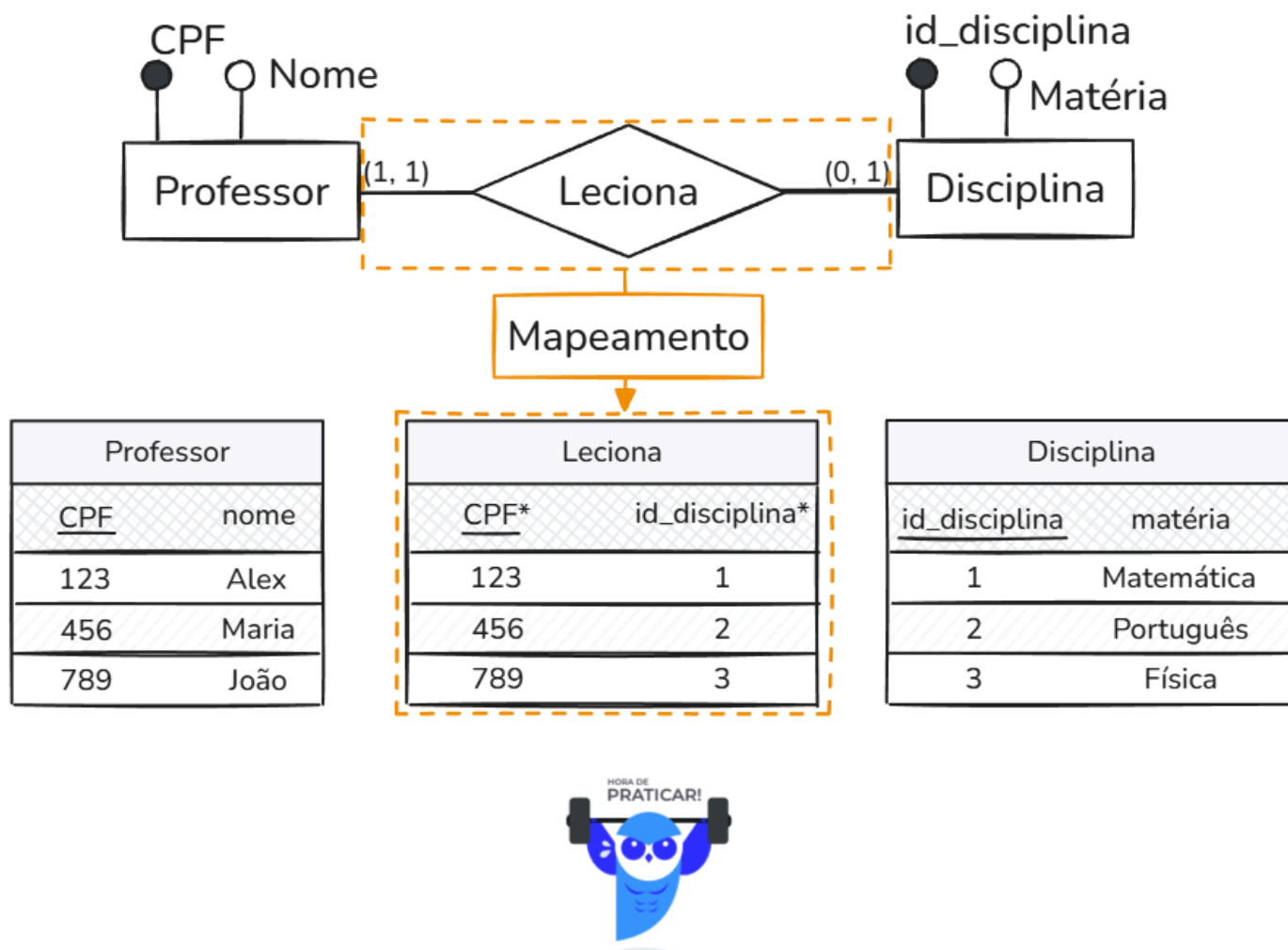


Percebam que, como não há motivo para manter a *surrogate key* "id_disciplina" na tabela mesclada, já que se trata de uma chave artificial criada apenas para fins de identificação interna, sem significado no negócio, ela foi descartada na tabela final, sem perda de informação.

Ainda existe uma terceira forma de resolvermos esse problema, porém **desencorajada** pelo autor: a **técnica de relação de referência cruzada** (ou técnica de relação de relacionamento).

Aqui **criamos uma tabela completamente nova para representar o relacionamento**, ou seja, uma tabela cuja única função é registrar as associações entre as tuplas de "Professor" e "Disciplina".

Para isso, essa tabela intermediária (chamada também de tabela de pesquisa ou relação de relacionamento) é composta pelas chaves primárias das duas tabelas originais, ambas atuando como chaves estrangeiras. Isso garante a integridade referencial nos dois sentidos.



(FUNDATEC/UFCSPA/Tecnologia da Informação/2024) No mapeamento de um diagrama entidade-relacionamento para um modelo relacional, quando um relacionamento é de tipo 1:1 (um-para-um) e ambas as entidades têm participação obrigatória no relacionamento (a cardinalidade mínima de ambas entidades é um), a regra de implementação mais indicada é a:

- Criação de uma tabela própria para cada entidade envolvida no relacionamento.
- Remoção dos atributos de uma das tabelas referentes às entidades envolvidas no relacionamento.
- Criação de uma tabela associativa, além das tabelas referentes às entidades envolvidas no relacionamento.
- Adição de colunas em cada uma das tabelas referentes às entidades envolvidas do relacionamento.
- Fusão das tabelas referentes às entidades envolvidas no relacionamento.

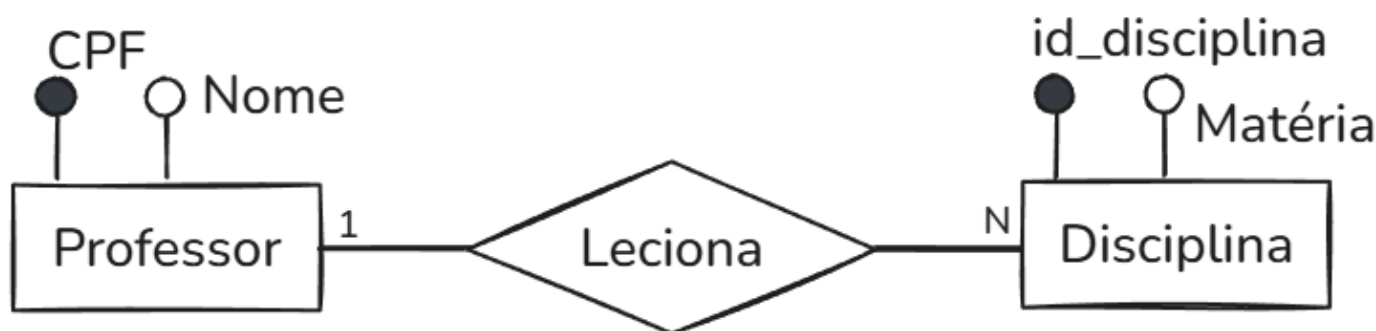
Comentários:

- a) **Errada.** Criar uma tabela para cada entidade sem nenhum mecanismo de associação não representa o relacionamento entre elas, sendo insuficiente como solução de mapeamento.
- b) **Errada.** Remover atributos de uma das tabelas causaria perda de informação, o que não é o objetivo do mapeamento.
- c) **Errada.** A criação de uma tabela associativa é a técnica de relação de referência cruzada, considerada a opção menos recomendada por Navathe para relacionamentos 1:1.
- d) **Errada.** Adicionar colunas em cada uma das tabelas corresponderia a colocar chaves estrangeiras nos dois lados simultaneamente, o que é redundante e desnecessário para um relacionamento 1:1.
- e) **Correta.** Quando ambas as entidades possuem participação obrigatória no relacionamento 1:1, alguns autores entendem que a técnica mais indicada é a fusão das tabelas em uma só, a relação mesclada. Como toda tupla de um lado estará sempre associada a uma tupla do outro, não haverá valores nulos e nenhuma informação será perdida. A coexistência dos dados na mesma tupla passa a representar a própria associação entre as entidades, resultando em um projeto mais limpo e eficiente.

Gabarito: E

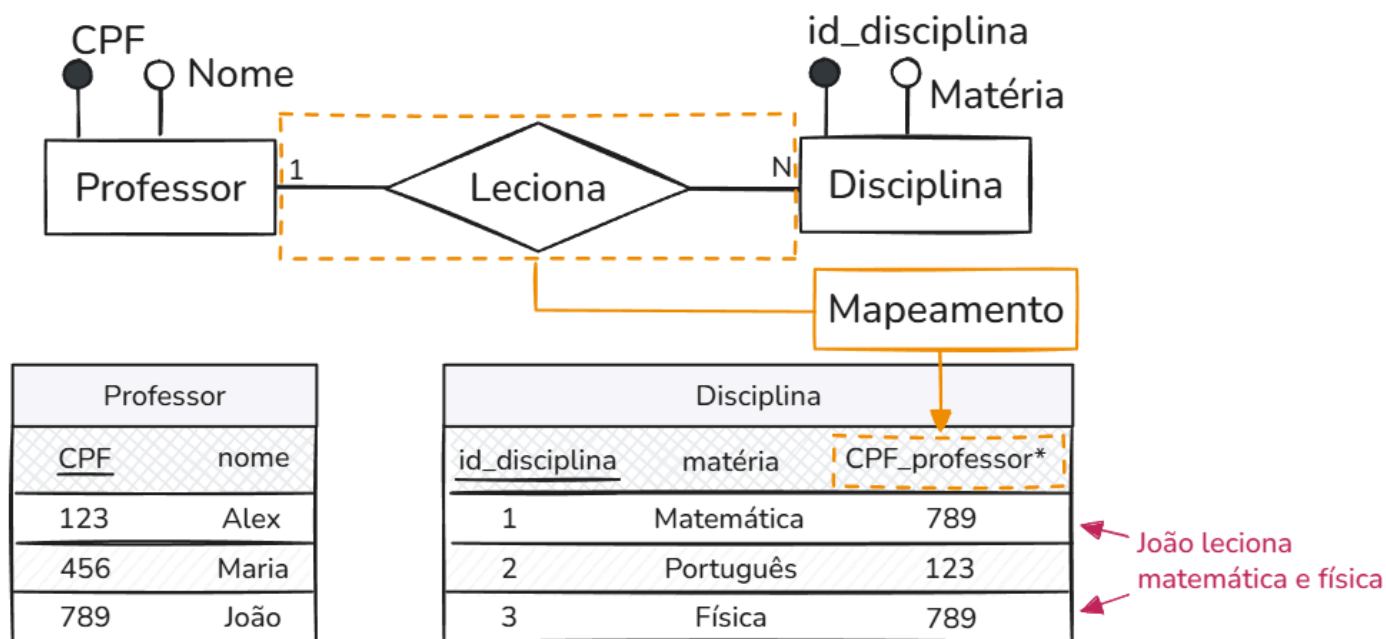
Mapeando de relacionamentos binários 1:N

Nessa seção precisamos supor que a razão de cardinalidade entre as entidades seja de 1:N, ou seja, vamos supor que cada professor pode lecionar várias disciplinas, enquanto cada disciplina ainda pode ser lecionada por no máximo 1 professor.



Segundo Navathe, nossa **predileção** de mapeamento em relacionamentos 1:N deve ser utilizar a **técnica de chave estrangeira**. Só que aqui precisamos identificar qual tabela representa o lado N, pois é nela que a chave estrangeira será colocada.

Vejam que temos uma regra mais restrita do que no caso 1:1, já que aqui não há escolha: **a chave estrangeira obrigatoriamente vai para o lado N**. Ou seja, a chave primária da tabela do lado 1 é adicionada como chave estrangeira na tabela do lado N para mapear o relacionamento "Leciona".

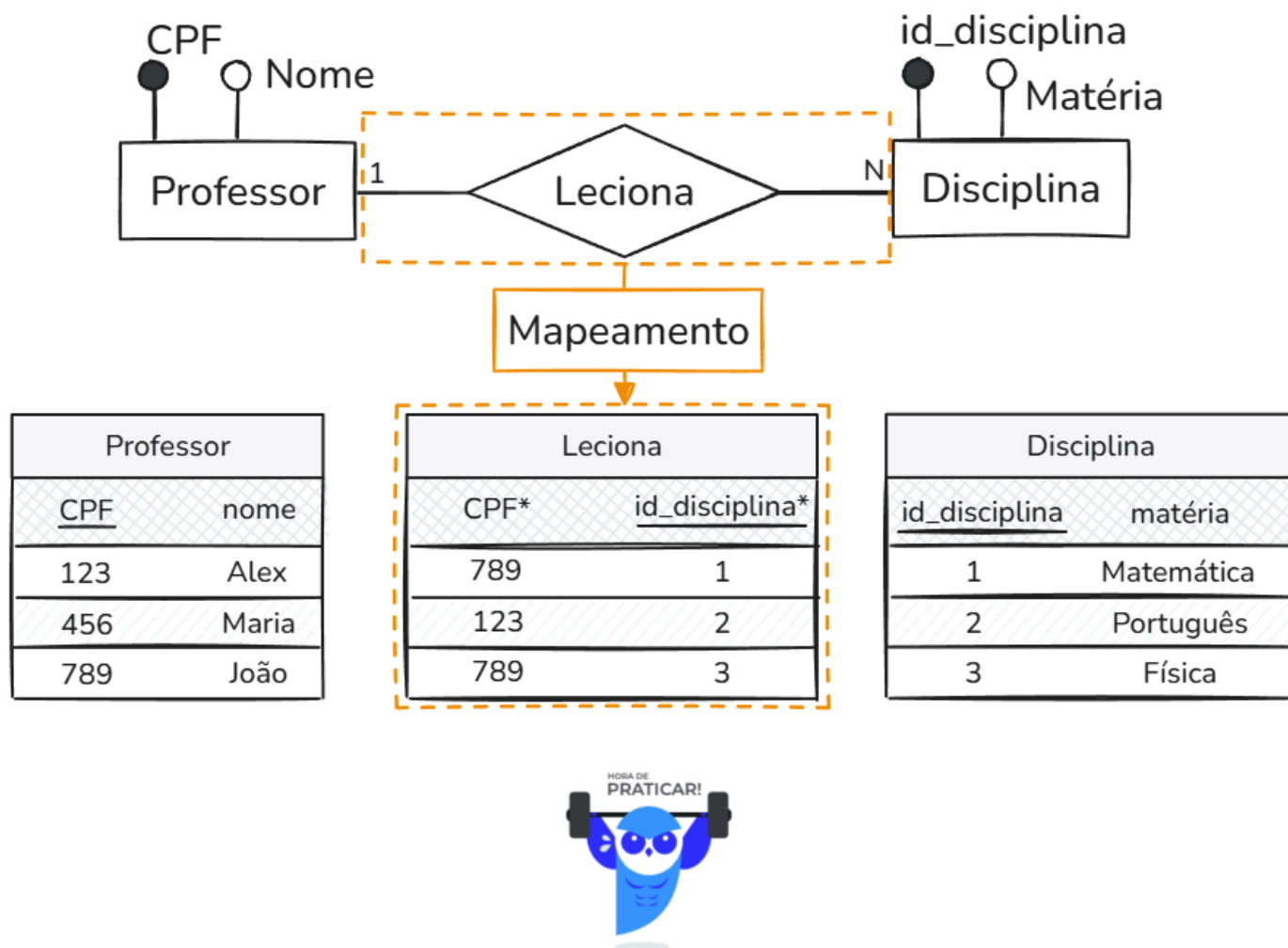


Assim como no caso 1:1, se o relacionamento possui atributos próprios, eles são colocados na mesma tabela que recebeu a chave estrangeira.

Percebam que seria impossível fazer o contrário disso, isto é, colocar a chave estrangeira no lado 1, mesmo se quiséssemos. Isso porque, como uma tupla do lado 1 pode estar associada a várias tuplas do lado N, um único atributo não conseguiria armazenar múltiplas referências.

Continuando, a solução alternativa para esse mapeamento seria pela técnica de relação de relacionamento, em que criamos uma nova tabela exclusivamente para representar o relacionamento.

A sua estrutura é a mesma da referência cruzada que vimos na seção anterior: uma tabela nova contendo apenas as chaves primárias das duas tabelas originais, ambas funcionando como chaves estrangeiras, sendo que só estarão registrados na tabela intermediária as chaves das tuplas que efetivamente participam da associação.



(CEBRASPE (CESPE)/MPO/Tecnologia da Informação/Gestão de Infraestrutura de TI/2024) Em geral, sempre que existir um relacionamento com cardinalidade de um-para-muitos, a referência lógica estará colocada na entidade que possuir o lado um da cardinalidade.

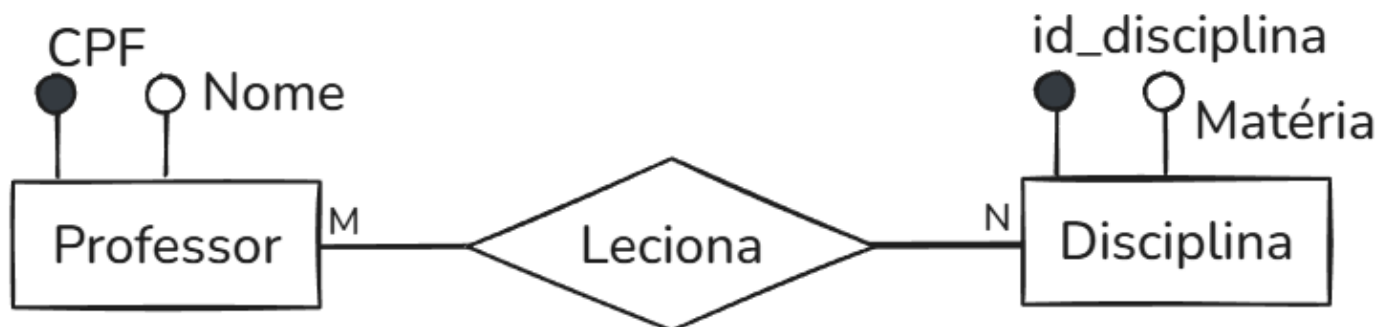
Comentários:

A afirmativa inverte a lógica do mapeamento de relacionamentos 1:N. A chave estrangeira deve ser colocada obrigatoriamente na entidade do lado N, e não no lado 1. Isso ocorre porque a entidade do lado 1 pode se associar a múltiplos registros do lado N, tornando impossível armazenar várias referências em um único atributo. Ao colocar a chave estrangeira no lado N, cada registro dessa entidade armazena a referência ao único registro do lado 1 ao qual está associado, respeitando a atomicidade dos valores exigida pelo modelo relacional.

Gabarito: Errado

Mapeando de relacionamentos binários M:N

Para essa seção, atualizaremos nosso diagrama ER para que tenha uma razão de cardinalidade de M:N. Assim, além de cada professor poder lecionar várias disciplinas, também temos que cada disciplina ainda pode ser lecionada por vários professores.

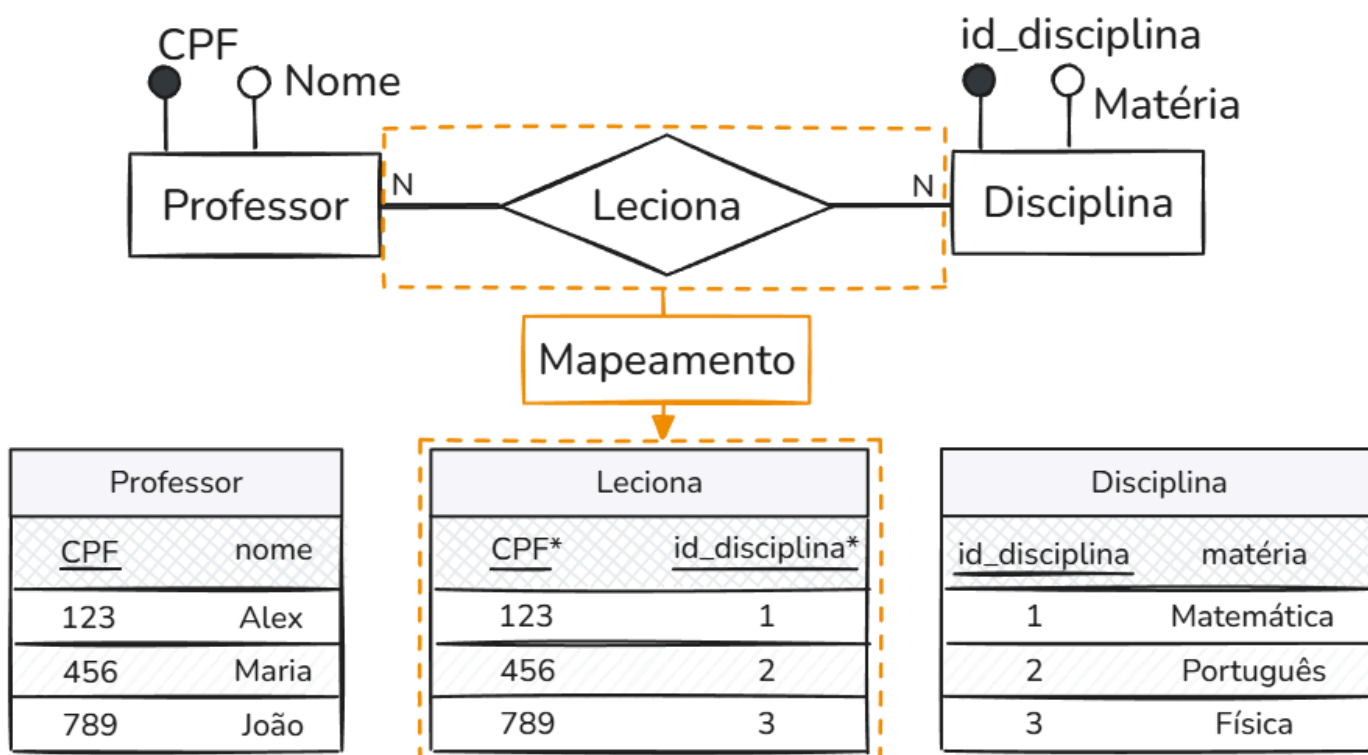


Talvez esse seja o tipo de relacionamento mais fácil de estudar, porque **aqui a criação de uma tabela intermediária** se trata da única forma de fazer o mapeamento, ou seja, **é obrigatória**.

Então, conforme já estudamos, a tabela intermediária recebe as chaves primárias de ambas as tabelas originais, e ambas funcionam como chaves estrangeiras.

Porém aqui temos algo bem específico: na tabela intermediária, nenhuma das chaves primárias das entidades sozinhas é suficiente para ser sua chave primária, pois uma mesma tupla de qualquer dos lados pode participar de múltiplas associações (ou seja, pode aparecer mais de uma vez nas linhas da tabela).

Então, somente a combinação das chaves primárias das entidades identifica de forma única cada registro da tabela intermediária. Por isso, a sua chave primária é uma chave composta, já que é formada pelas duas chaves estrangeiras juntas.



Mas cuidado: se houver possibilidade de haver mais de uma combinação para a tabela intermediária, deve-se utilizar uma chave substituta (surrogate key), como uma "id_disciplina". Isso porque, nesse caso, as chaves dos atributos já não são mais suficientes para identificar cada tupla.

Por exemplo, se um professor puder dar a mesma disciplina mais de uma vez (em semestres diferentes), a combinação "id_professor + id_disciplina" se repetiria, e não conseguimos mais identificar cada registro de forma única.



(CEBRASPE (CESPE)/TRF 6/Apoio Especializado/Desenvolvimento de Sistemas de Informação/2025) A relação muitos-para-muitos pode ser representada diretamente entre duas tabelas sem a necessidade de tabelas intermediárias.

Comentários:

A criação de uma tabela intermediária é a única forma de mapear um relacionamento M:N, sendo, portanto, obrigatória. A razão disso é estrutural: sem a tabela intermediária, não há como representar o fato de que um mesmo registro de qualquer dos lados pode participar de múltiplas associações. Se tentássemos representar isso diretamente entre duas tabelas, inevitavelmente violaríamos princípios fundamentais do modelo relacional, como a atomicidade dos valores em cada célula.

Gabarito: Errado

(CEBRASPE (CESPE)/MCom/Tecnologia da Informação e de Engenharia Senior/2022) No banco de dados relacional, é permitido criar uma chave primária utilizando-se dois ou mais campos definidos como chaves estrangeiras.

Comentários:

A afirmativa está correta e é exatamente o que acontece ao mapear uma tabela intermediária no relacionamento M:N. Como nenhuma das chaves estrangeiras individualmente é suficiente para identificar de forma única cada registro da tabela intermediária, a solução é combinar as duas chaves estrangeiras em uma chave primária composta.

Gabarito: Certo

Visões (Views)

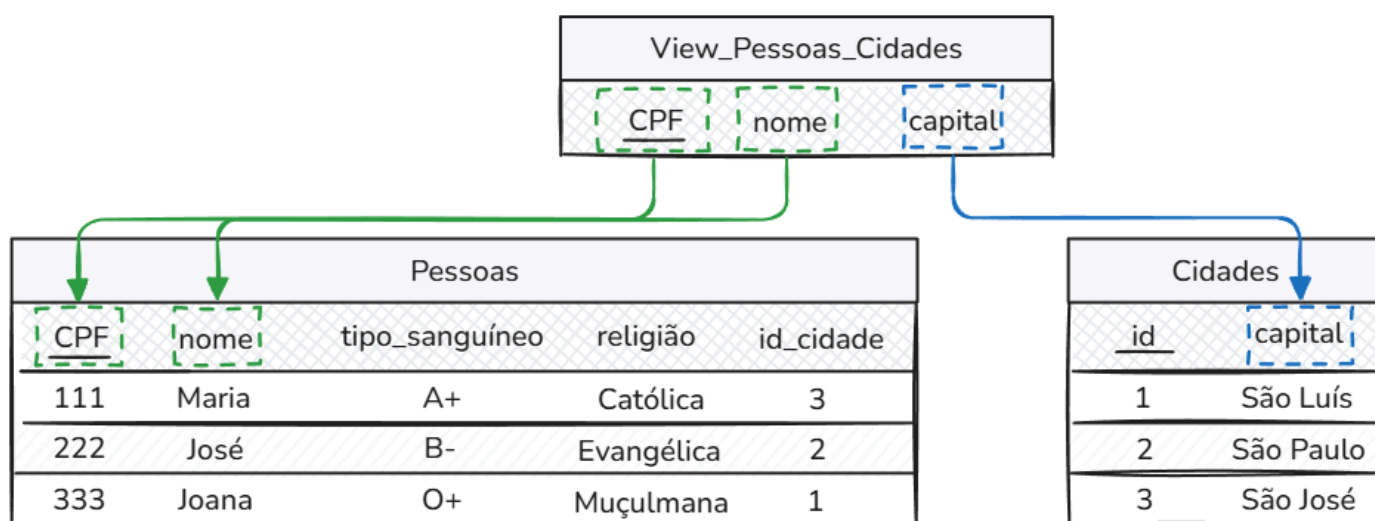
Uma **visão (view)** é uma **tabela derivada de outras tabelas do banco de dados, porém com a peculiaridade de não armazenar novamente os dados dessas outras tabelas.**

Por exemplo, vamos supor que em uma tabela "Cidades" com capitais, e em outra "Pessoas" temos dados pessoais delas, como "CPF" e "nome", e dados sensíveis, como "tipo_sanguíneo" e "religião".

Cidades	
<u>id</u>	nome
1	São Luís
2	São Paulo
3	São José

Pessoas			
<u>CPF</u>	nome	tipo_sanguíneo	religião
1	Maria	A+	Católica
2	José	B-	Evangélica
3	Joana	O+	Muçulmana

Queremos, então, disponibilizar uma view “View_Pessoas_Cidades” que forneça apenas as informações pessoais e as cidades das pessoas, escondendo dados sensíveis.



Percebam do nosso esquema que não há dados abaixo da tabela “View_Pessoas_Cidades”, já que ela não armazena dados fisicamente. Então, toda vez que consultamos os dados de uma view, o SGBD busca os dados nas tabelas originais que ela referencia.

Percebam então que os dados das **visões** são computados dinamicamente a cada requisição, por isso são **chamadas também de tabelas virtuais** (ou de tabelas derivadas). Por esse mesmo motivo, as **tabelas que de fato possuem os dados** para servir à visão são chamadas de **tabelas base**.



Uma visão pode ser derivada de tabelas base ou de outras visões, já que essas também são tabelas.

Como a visão não possui dados próprios, visto que sempre consulta as tabelas base no momento do acesso, qualquer alteração feita nas tabelas base se reflete automaticamente na visão.

Assim, uma view dinâmica sempre reflete o estado atual das tabelas, sem precisar de nenhuma ação explícita pelos usuários do banco de dados. Não há nenhum tipo de atualização manual ou qualquer coisa do tipo, porque não há o que atualizar, já que os dados não estão armazenados na view, estão nas tabelas originais.

Em resumo, **não há risco de defasagem entre as informações das tabelas bases e a view, a visão sempre mostra o estado atual dos dados.**



Por isso, visões “comuns” também são chamadas de visões dinâmicas.

Na prática, a visão é criada no banco de dados com um comando SQL chamado “CREATE VIEW ‘nome da view’” o atribuindo a uma consulta (comando “SELECT” do SQL). Para nosso exemplo, seria basicamente uma declaração em SQL que diz “Crie uma view com nome ‘View_Pessoas_Cidades’ que seja a consulta aos atributos ‘CPF’ e ‘nome’ da tabela ‘Pessoas’ e ‘capital’ da tabela ‘Cidades’”.



(ADAPTADA - FGV/Ana MPU/2025) No processo de realizar análises nos dados armazenados em um banco de dados relacional, o perito Charles está implementando diversas consultas SQL. Ele precisa acessar o resultado dessas consultas diversas vezes, como se fosse uma tabela, em um ciclo de refinamento analítico contínuo.

Em um banco de dados relacional, para criar uma tabela virtual que represente o resultado de uma consulta SQL, Charles deve implementar um(a) VIEW

Comentários:

Uma VIEW (visão) é exatamente uma tabela virtual cujo conteúdo é definido por uma consulta SQL. Charles pode criá-la com CREATE VIEW e, a partir daí, consultá-la repetidamente como se fosse uma tabela comum, sem necessidade de reescrever a query original.

Gabarito: Certa

(CEBRASPE (CESPE)/TRF 6/Apoio Especializado/Análise de Dados/2025) Uma view dinâmica é atualizada quando objetos relacionados ou objetos estendidos são criados ou alterados; essa atualização é manual por rotina específica do banco de dados.

Comentários:

A afirmativa erra em dois pontos. Primeiro, ao dizer que a atualização ocorre quando "objetos relacionados ou estendidos são criados ou alterados": a visão dinâmica não passa por nenhum processo de atualização, pois ela simplesmente não armazena dados. Toda vez que é acessada, o SGBD busca os dados diretamente nas tabelas base naquele momento, refletindo automaticamente qualquer alteração já existente nelas. Segundo, ao afirmar que essa atualização é "manual por rotina específica": isso contradiz diretamente o caráter dinâmico da visão comum, que é justamente a atualização automática e transparente a cada acesso, sem qualquer intervenção manual. Esse comportamento de atualização manual por rotina é, na verdade, característico das visões materializadas, não das dinâmicas.

Gabarito: Errado

(CEBRASPE (CESPE)/CTI/Inovação e Gestão de Infraestrutura de P&D/2024) Uma VIEW é uma representação virtual de dados que se comporta como uma tabela, formada a partir de uma consulta em tabelas relacionadas.

Comentários:

As visões de tabelas virtuais, não armazenam dados fisicamente, mas se apresentam ao usuário como se fossem tabelas normais.

Gabarito: Certo

Agora acredito que vocês devem estar pensando: *entendi, mas por que queremos criar uma visão no banco de dados, professor?*

Porque, como as visões funcionam como intermediárias entre os dados reais e os usuários que os pretendem acessar, podemos utilizar as visões para filtrar, simplificar, agregar, abstrair e organizar a informação conforme a necessidade de cada contexto, tudo isso sem duplicar dados e sem expor a complexidade interna do banco. Vamos explicar melhor isso.

Vantagem	Explicação
Segurança e controle de acesso	Uma visão expõe apenas os dados que deseja das tabelas que deseja. Assim, o administrador do banco pode conceder acesso à visão sem conceder acesso às tabelas base completas, restringindo o usuário a enxergar e manipular apenas o que lhe foi permitido, sem saber que existem outros dados por trás. Em resumo, a visão pode ser utilizada como uma espécie de “filtro de permissão”, definindo a fronteira entre o que é visível e o que permanece oculto para cada tipo de usuário.

Vantagem	Explicação
Simplificação de consultas complexas	Quando uma consulta envolve uma lógica muito complexa, a visão encapsula toda essa complexidade em uma única tabela virtual, acessível facilmente pelo usuário. Em outras palavras, para o usuário final tudo se comporta como uma consulta simples sobre a visão, sem que ele precise compreender de fato como as tabelas base estão relacionadas internamente.
Agregação de dados	Uma visão pode apresentar dados já processados, como somas, médias, contagens, etc. Dessa forma, o usuário recebe os resultados prontos, como um "relatório permanente", sempre atualizado e sempre disponível.
Abstração da estrutura real	Se as tabelas do banco de dados estão organizadas de forma pouco intuitiva, a visão permite apresentar esses mesmos dados de uma forma que seja natural e compreensível para quem os consome.
Economia de espaço	Como a visão armazena apenas a definição da consulta e não uma cópia dos dados, o custo de armazenamento é mínimo. Não importa se a consulta da visão retorna milhões de tuplas, o espaço ocupado pela visão em si é apenas o necessário para guardar o texto da consulta no catálogo do sistema.
Adequação a diferentes perfis de usuários	Diferentes tipos de usuários têm diferentes necessidades e diferentes formas de pensar sobre os dados. Então, uma mesma base de dados pode ter várias visões distintas, se isso fizer sentido para quem acessa.



(CEBRASPE (CESPE)/SEPLAN RR/Tecnologia da Informação/2023) Visões podem ser usadas para definir relações no esquema externo que exibem para os aplicativos as alterações feitas no esquema conceitual do banco de dados.

Comentários:

As visões são utilizadas exatamente para ocultar das aplicações as alterações feitas no esquema conceitual, e não para exibi-las. Na abstração da estrutura real, a visão permite apresentar os dados de forma natural e compreensível, independentemente de como as tabelas base estão organizadas internamente. Na simplificação de consultas complexas, o usuário não precisa compreender como as tabelas base estão relacionadas, pois a visão encapsula essa complexidade.

Gabarito: Errado

Views Materializadas

Como vimos, uma visão "comum" é apenas uma definição de consulta, executada toda vez que alguém acessa a view. Ou seja, ela não armazena dados em disco, sua tabela é gerada em memória cada vez que a consulta roda. Por isso mesmo, é chamada de virtual.

Pois bem, uma **visão materializada** faz o oposto disso: ela **executa a consulta e armazena no disco o resultado como uma tabela concreta**. Ou seja, os **dados ficam fisicamente gravados**, prontos para serem acessados diretamente, sem necessidade de recomputar a consulta a cada acesso.



O termo "materialização" vem justamente desse processo de tornar concreto algo que era virtual. A visão comum existe apenas como definição de uma consulta, sendo abstrata. Já a visão materializada roda essa consulta previamente definida, gerando dados reais, armazenados fisicamente. Ou seja, o virtual se torna material, real.

A **principal vantagem da view materializada sobre a comum se trata do desempenho de acesso**. Isso porque a visão materializada já tem os dados pré-computados armazenados fisicamente em tabela. Assim, quando consultada, o sistema simplesmente lê os dados da tabela, que já estão prontos. Enquanto isso, a visão comum precisa executar a consulta toda vez que é acessada, recalculando tudo do zero a partir das tabelas base. Dessa forma, quanto mais complexa a consulta (junções de dados de múltiplas tabelas, agregações pesadas, grandes volumes de dados), maior a diferença de desempenho entre as duas.

Em contrapartida, sofremos com **problemas de desatualização**. Enquanto uma **visão** comum está sempre atualizada, pois consulta as tabelas base no momento do acesso, **a visão materializada precisa passar por um processo de atualização periódica para sincronizá-la com as tabelas base**. Existe, portanto, uma janela de tempo em que os dados da tabela da view podem não refletir o estado mais recente do banco.

Assim sendo, utilizamos as visões materializadas como uma decisão de compromisso: queremos em cenários em que as consultas às tabelas base são pesadas e frequentes, enquanto os dados não mudam com tanta frequência (cenário típico de data warehouses). Nesses ambientes, reexecutar consultas complexas a cada acesso seria extremamente custoso, assim a visão materializada oferece respostas rápidas a um custo aceitável de possível defasagem de atualização.

Percebam então que, regra geral, **utilizamos a visão materializada essencialmente como uma forma de cache**. Queremos guardar o resultado de um cálculo “pesado” para não precisar refazê-lo toda vez, trocando deliberadamente essa velocidade de acesso contra mais espaço de armazenamento.



(CEBRASPE (CESPE)/APC (FUNPRESPE,EXE)/FUNPRESPE,EXE/Comercial/2022) Todas as views criadas especificamente pelo administrador de banco de dados são armazenadas fisicamente no banco de dados.

Comentários:

A visão comum não armazena dados fisicamente: ela é virtual, existe apenas como uma definição de consulta e gera sua tabela em memória a cada acesso. Já a visão materializada é justamente a exceção, pois executa a consulta previamente e armazena o resultado como uma tabela concreta em disco.

Gabarito: Errado

As 12 regras de Codd

No início dos anos 1980, o modelo relacional ganhou grande prestígio. Fabricantes de bancos de dados perceberam o valor comercial desse rótulo e começaram a chamar seus produtos de "relacionais" mesmo quando eles não seguiam verdadeiramente os princípios desse modelo.

Codd, então, criou 12 regras (que na verdade são 13, já que vão de 0 a 12) justamente para combater esse oportunismo, já que funcionam como um teste de conformidade: com elas, temos critérios objetivos para classificar os sistemas de banco de dados (SGBDs) em genuinamente relacionais, os separando daqueles que apenas aparentam ser.

#	Nome	Descrição
0	Regra fundamental	Se um SGBD se apresenta como relacional, então ele deve ser capaz de realizar suas atividades apenas usando os princípios do modelo relacional.

Essa é a regra que sustenta todas as outras, isto é, é a base da qual as outras se irradiam.

O modelo relacional deve ser o único meio que o SGBD relacional utiliza para gerenciar os dados. Ou seja, o banco de dados relacional deve tratar os dados como tabelas, as regras do banco devem ser expressas como restrições de integridade, as operações devem acontecer via a linguagem relacional (na prática, SQL) etc.

Consequentemente, se o sistema precisar recorrer a mecanismos não relacionais para realizar suas operações, ele não pode se intitular relacional.

1	Regra da informação	Toda informação é representada explicitamente no nível lógico e de exatamente uma forma: por valores em tabelas
---	----------------------------	---

Também chamado de “Princípio da Representação Uniforme”, porque estabelece que toda informação dentro de um banco de dados relacional deve ser representada de uma e somente uma forma: tabelas compostas por linhas e colunas, onde cada célula contém apenas um valor.

Consequentemente, nada fica escondido em estruturas externas ao modelo (como arquivos especiais, variáveis internas, etc.), sendo possível acessar e manipular a informação usando operações relacionais (na prática, SQL).

2	Regra do acesso garantido	Garante-se o acesso a qualquer dado pela combinação de nome da tabela, nome da coluna e valor da chave primária.
---	----------------------------------	--

Qualquer valor individual armazenado no banco de dados deve ser localizado de forma inequívoca se usarmos três informações: qual tabela, qual coluna e qual linha (identificada pela chave primária) contém o dado.

Consequentemente, com essas três informações, nenhum dado fica inacessível. Também não se pode exigir um caminho de acesso diferente desses três elementos.

3	Regra do tratamento sistemático de valores nulos	O SGBD relacional deve suportar valores nulos (NULL) para representar informação ausente e informação inaplicável.
---	---	--

#	Nome	Descrição
---	------	-----------

O banco de dados precisa ter uma forma padronizada de lidar com a ausência de informação: utilizando o NULL (valor nulo).

Mas cuidado, não confundam o valor nulo com outras coisas. O nulo não é zero, não é uma string vazia, não é um espaço em branco. *O que é então, professor?* O NULL significa que o valor não existe ou não se aplica.

Além disso, o tratamento do nulo deve ser uniforme, visto que não importa se o valor ausente ocorre em um atributo que é um número, um texto ou uma data, o comportamento do nulo deve sempre funcionar da mesma maneira.

Se assim não o fosse, cada situação de ausência de informação teria que ser tratada de forma única, com alto potencial de ocorrência de inconsistências de tratamento.

4	Regra do catálogo on-line baseado no modelo relacional	A descrição do banco de dados é representada no nível lógico da mesma forma que os dados "comuns".
---	---	--

O banco de dados não armazena apenas os dados dos usuários, porque também precisa armazenar informações sobre si mesmo: quais tabelas existem, quais colunas cada tabela tem, quais restrições estão definidas, etc. Esses dados são os chamados metadados, e ficam no catálogo do sistema.

Pois bem, essa regra exige que esses metadados sejam armazenados em tabelas comuns, acessíveis pela mesma linguagem relacional (na prática, SQL) usada para consultar os dados "normais".

5	Regra da sublinguagem ampla de dados	Deve existir pelo menos uma linguagem completa o suficiente para cobrir todas atividades essenciais com os dados sem depender de mecanismos externos.
---	---	---

Na prática, um sistema relacional pode oferecer suporte a várias formas de interação com os dados, com linguagens específicas, com interfaces gráficas etc.

Porém, sempre deve existir pelo menos uma linguagem com sintaxe clara e bem definida, abrangente o suficiente para conseguir fazer tudo que o banco precisa :

- Definir dados: criar e alterar a estrutura das tabelas
- Definir visões: criar visões sobre os dados

#	Nome	Descrição
---	------	-----------

- Manipular dados: inserir, consultar, atualizar e excluir dados, tanto de forma interativa quanto por programas
- Restrições de integridade: definir regras que os dados devem obedecer
- Autorização: controlar quem pode fazer o quê
- Limites de transação: demarcar o início (begin), a confirmação (commit) e o cancelamento de transações (rollback).

A regra não exige que seja a única forma de interação, mas exige que exista pelo menos uma linguagem completa o suficiente para cobrir todos esses aspectos.

Na prática, o SQL é a linguagem que cumpre esse papel nos SGBDs atuais.

Vale saber que essa regra também é chamada de regra da sublinguagem compreensiva de dados.

6	Regra de atualização de visões	Todas as visões que são teoricamente atualizáveis devem ser atualizáveis pelo sistema relacional.
---	---------------------------------------	---

Antes de explicarmos, vamos relembrar que visões são tabelas virtuais baseadas em consultas em tabelas base.

Pois bem, aqui estamos preocupados com a possibilidade de realizar operações de modificação de dados através da visão, o que inclui INSERT (inserir novas tuplas), UPDATE (alterar valores existentes) e DELETE (excluir tuplas).

Em resumo, essa regra diz que se há possibilidade teórica de realizarmos essas operações na tabela base utilizando a visão, então o sistema relacional deve permitir que a operação se realize pela visão.

Eu sei que parece uma regra óbvia, mas existem casos em que não é possível atingir os dados nas tabelas bases a partir da view, isso porque existem visões que perdem a rastreabilidade aos dados das tabelas bases. Há casos também que os SGBDs atuam de forma conservadora e simplesmente bloqueiam as atualizações em views para evitar possíveis erros, como quando uma visão atua sobre atributos de várias tabelas. Nesses casos, a modificação não é permitida.

7	Regra da inserção, atualização, e exclusão de Alto nível	A inserção, atualização, e exclusão de informações devem ser permitidas para qualquer conjunto de valores e não apenas para uma única linha em uma única tabela.
---	---	--

#	Nome	Descrição
---	------	-----------

Um sistema relacional deve ser capaz de inserir, atualizar ou excluir múltiplas tuplas em uma única operação, tratando um conjunto de informações da mesma forma que trataria uma única informação. Ou seja, não deve ser necessário processar os dados tupla por tupla para realizar operações com conjuntos de dados.

A lógica disso é que se o modelo relacional baseia-se na teoria de conjuntos, então todas suas operações (e não apenas a leitura) devem respeitar essa natureza.

8	Regra da independência física de dados	Não precisamos alterar os programas que acessam o banco de dados relacional quando mudanças são feitas na forma de armazenamento das suas informações ou em seus métodos de acesso.
---	---	---

Essa regra exige que os sistemas relacionais de fato respeitem a existência de dois mundos distintos e desacoplados: o lógico (como o usuário e os programas enxergam os dados) e o físico (como os dados estão realmente armazenados no disco).

Assim sendo, mudanças no nível físico (como reorganizar arquivos, mudar estruturas de indexação, migrar para outro tipo de armazenamento ou alterar métodos de acesso) não devem afetar em nada o funcionamento dos programas que usam o banco. Ou seja, o programa continua fazendo as mesmas consultas da mesma forma e obtendo os mesmos resultados.

Em resumo, a camada física é “invisível” para quem interage com o banco relacional no nível lógico.

9	Independência lógica de dados	Não precisamos alterar os programas que acessam o banco de dados relacional quando realizamos mudanças nas suas tabelas, desde que a informação original seja preservada após a alteração.
---	--------------------------------------	--

Essa regra exige que mudanças na estrutura lógica do banco relacional (como dividir uma tabela em duas, reorganizar colunas de uma tabela ou reestruturar o esquema) não afetem os programas que interagem com esse banco, desde que essas mudanças preservem a informação original (ou seja, nenhum dado é perdido).

Na prática, os sistemas relacionais respeitam essa regra permitindo a criação de visões. Isso porque, se mudarmos as tabelas base, podemos reescrever a visão para apresentar os dados no formato antigo aos programas, os mantendo em funcionamento sem alteração.

#	Nome	Descrição
10	Regra da independência de integridade	Restrições de integridade dos dados devem ser definíveis na linguagem relacional e armazenadas no catálogo do sistema, e não nos programas de aplicação.

Essa regra exige que as restrições de integridade sobre os dados pertencem ao banco de dados, não aos programas que o utilizam.

Por exemplo, se uma regra diz que o atributo “salário” não pode ser negativo, então essa restrição deve ser definida no próprio banco, usando linguagem relacional, e armazenada no catálogo do sistema. Consequentemente, não devemos esperar que cada programa que consoma o banco de dados relacional implemente essa verificação por conta própria.

11	Regra da independência de distribuição	O usuário final não deve ser capaz de perceber que os dados estão distribuídos em vários locais.
----	---	--

Essa regra aplica-se no caso de sistemas distribuídos, em que os dados podem estar fisicamente espalhados por vários servidores em locais diferentes.

Pois bem, nesses casos o usuário que interage com o banco deve perceber que os dados estão em um único lugar. Assim, ele faz suas consultas normalmente, sem precisar saber onde cada dado está armazenado, em qual servidor, em qual cidade, já que o próprio sistema relacional se encarrega de localizar, reunir e apresentar os dados juntos.

12	A não transposição (ou subversão) das regras	Se o sistema possui uma linguagem de baixo nível, essa linguagem não pode ser usada para contornar as regras de integridade definidas na linguagem relacional de alto nível.
----	---	--

Codd criou essa regra para barrar uma prática comum dos fabricantes de SGBDs “relacionais” da época: oferecer interface relacional de alto nível que operava sobre conjuntos de tuplas, mas cujas restrições eram passíveis de serem desrespeitada por uma interface de baixo nível que conseguia acessar os registros do banco diretamente.

Na prática, era como ter uma porta dos fundos de acesso direto aos dados, transpondo qualquer tipo de restrição imposta por outra interface. Assim, Codd exigiu que, mesmo que essa linguagem de baixo nível exista, ela deve respeitar as mesmas regras de integridade impostas pela linguagem de alto nível.

Em resumo, nenhum caminho alternativo pode ser usado para subverter as garantias do modelo relacional.



(FCC/SEAD PI/Tecnologia da Informação/2024) Um analista de uma Secretaria de Estado necessita modificar a estrutura (esquema) de um banco de dados relacional adicionando novas tabelas e novas colunas em tabelas já existentes. Os dados já utilizados permanecerão íntegros e disponíveis para as aplicações que os utilizam e sem necessitar que sejam modificadas.

Foram utilizadas as regras de Codd na estruturação do banco e, em especial, a regra de

- a) independência lógica de dados.
- b) independência de atualização de alto nível.
- c) persistência de dados.
- d) origem lógica de dados.
- e) permanência de dados.

Comentários:

a) **Correta.** A independência lógica de dados é exatamente a regra que permite modificar a estrutura lógica do banco — como adicionar novas tabelas ou colunas — sem que os programas que já utilizam os dados precisem ser alterados, desde que a informação original seja preservada. É precisamente o cenário descrito na questão.

b) **Errada.** Não existe nas 12 regras de Codd uma regra chamada "independência de atualização de alto nível". A regra de alto nível trata da capacidade de inserir, atualizar e excluir conjuntos de tuplas em uma única operação, sem relação com mudanças no esquema.

c) **Errada.** "Persistência de dados" não é uma das 12 regras de Codd, tratando-se de um conceito geral de banco de dados que se refere à capacidade de manter dados armazenados de forma durável.

d) **Errada.** "Origem lógica de dados" não corresponde a nenhuma das 12 regras de Codd, sendo uma alternativa fabricada para confundir.

e) **Errada.** "Permanência de dados" também não é uma das 12 regras de Codd, sendo igualmente uma alternativa sem correspondência no conjunto de regras definido por Codd.

Gabarito: A

(CEBRASPE (CESPE)/Ana TI (DATAPREV)/DATAPREV/Sustentação Tecnológica/2023) Views não inserem restrições à realização de consultas, inserções, atualizações e exclusões em tempo de execução, sendo, portanto, desnecessário traduzi-las para uma modificação nas relações reais no modelo lógico do banco de dados.

Comentários:

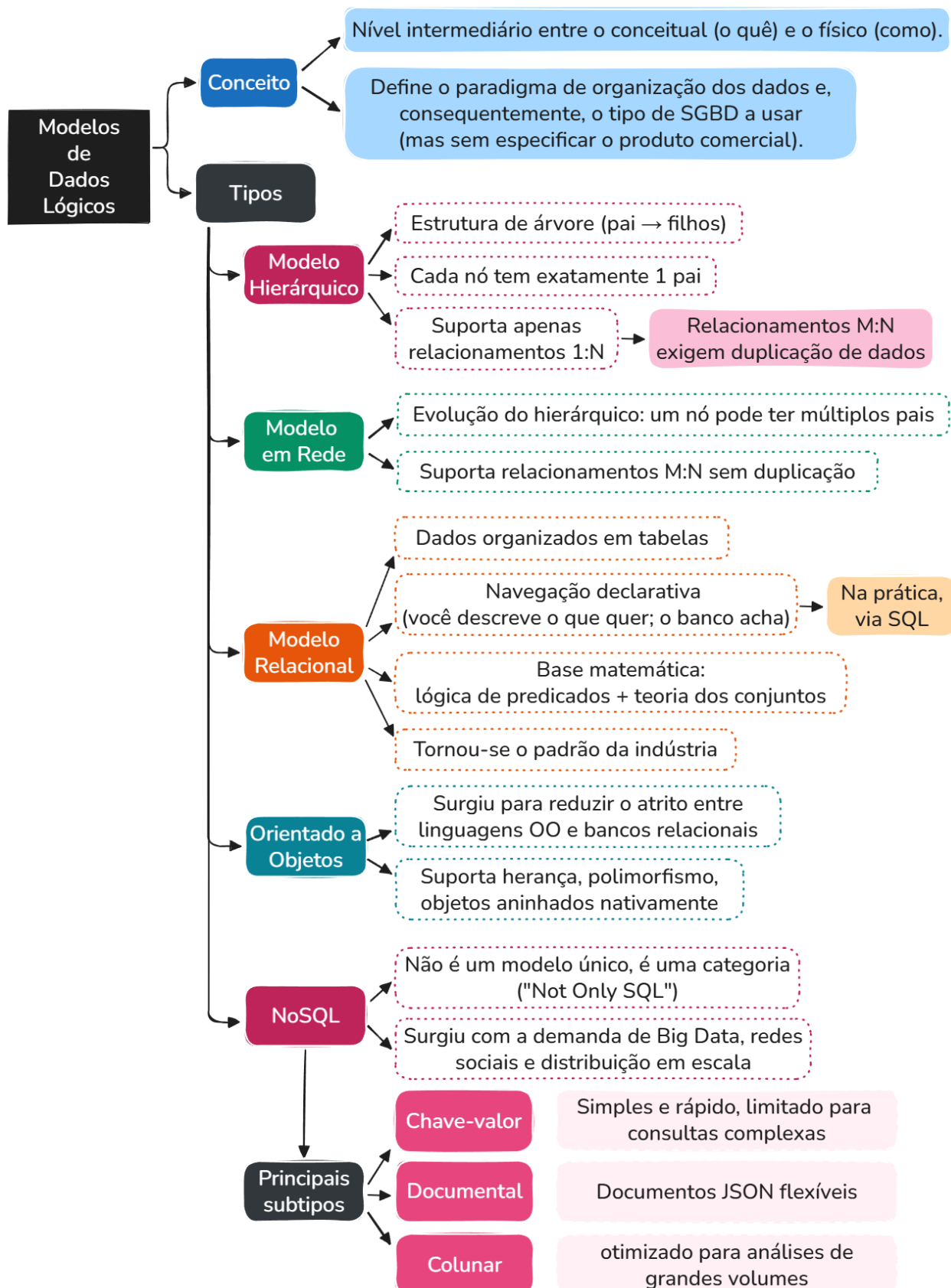
A Regra 6 (Atualização de Visões) estabelece que todas as visões que são teoricamente atualizáveis devem ser atualizáveis pelo sistema relacional, o que implica que operações de INSERT, UPDATE e DELETE realizadas por meio de uma visão precisam ser traduzidas para

modificações nas tabelas base (relações reais). Ou seja, a tradução para o modelo lógico subjacente é justamente o que se exige, não o que se dispensa.

Segundo, a própria regra reconhece que existem visões que perdem a rastreabilidade às tabelas base, e nesses casos, sim, a modificação não é exigida, mas porque ela é teoricamente impossível, não porque seja desnecessária. Portanto, a impossibilidade de modificação é uma exceção fundamentada, e não a regra geral.

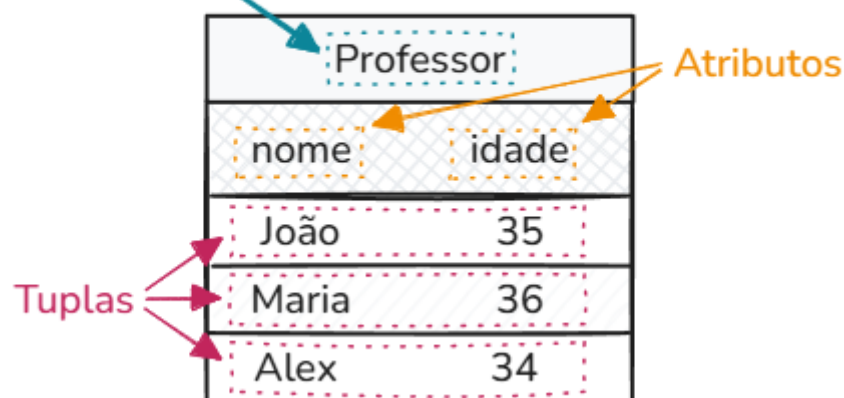
Gabarito: Errado

RESUMO

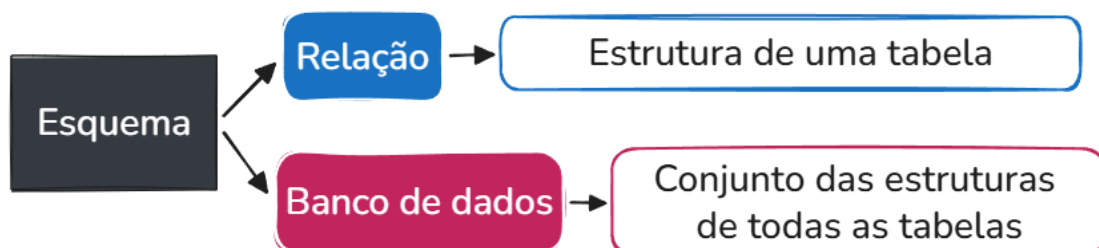
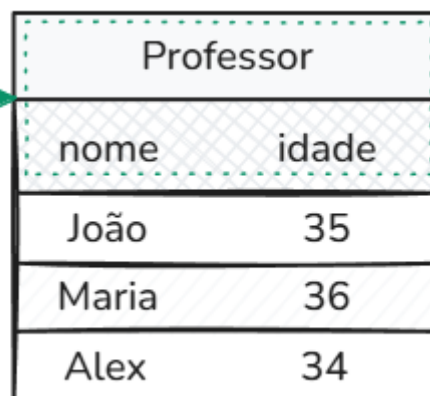


Modelo Relacional	Banco de Dados Relacional
Relação	Tabela
Tupla	Registro
Atributo	Campo

Nome da relação



Esquema da relação



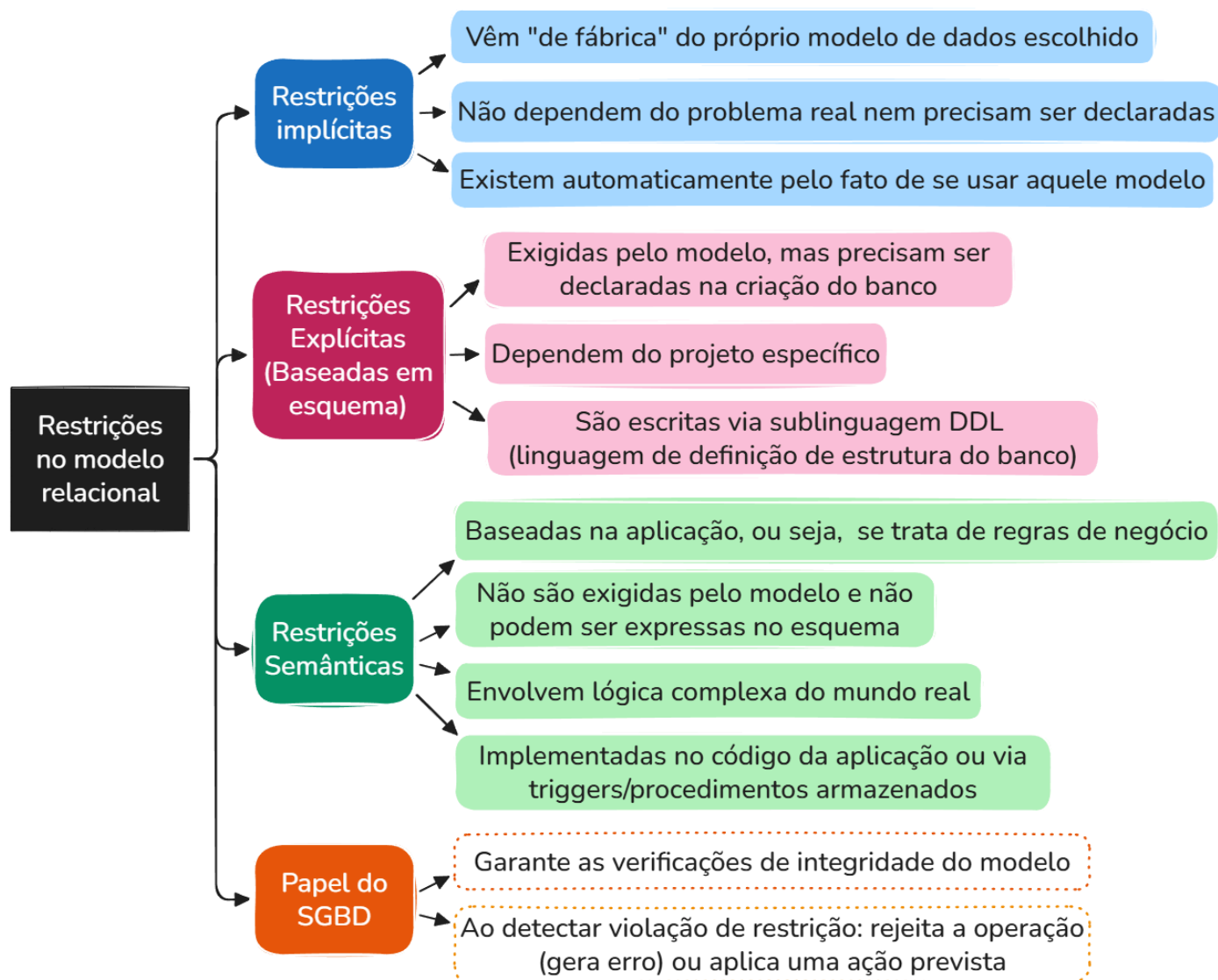
Cardinalidade = 2

Professor	
nome	idade
João	35
Maria	36
Alex	34

Aridade = 3



Também chamamos o tipo de dado, ou seja, o conjunto de valores permitidos para cada atributo (coluna), de **domínio**.



Restrições implícitas do modelo relacional

Restrição:	As tuplas de uma relação não possuem ordem.
Restrição:	Os atributos de uma tupla possuem ordem.
Restrição:	Cada valor de atributo deve ser atômico.

Restrição:

Um valor desconhecido, inexistente, indisponível, não aplicável ou indefinido deve ser representado por NULL.

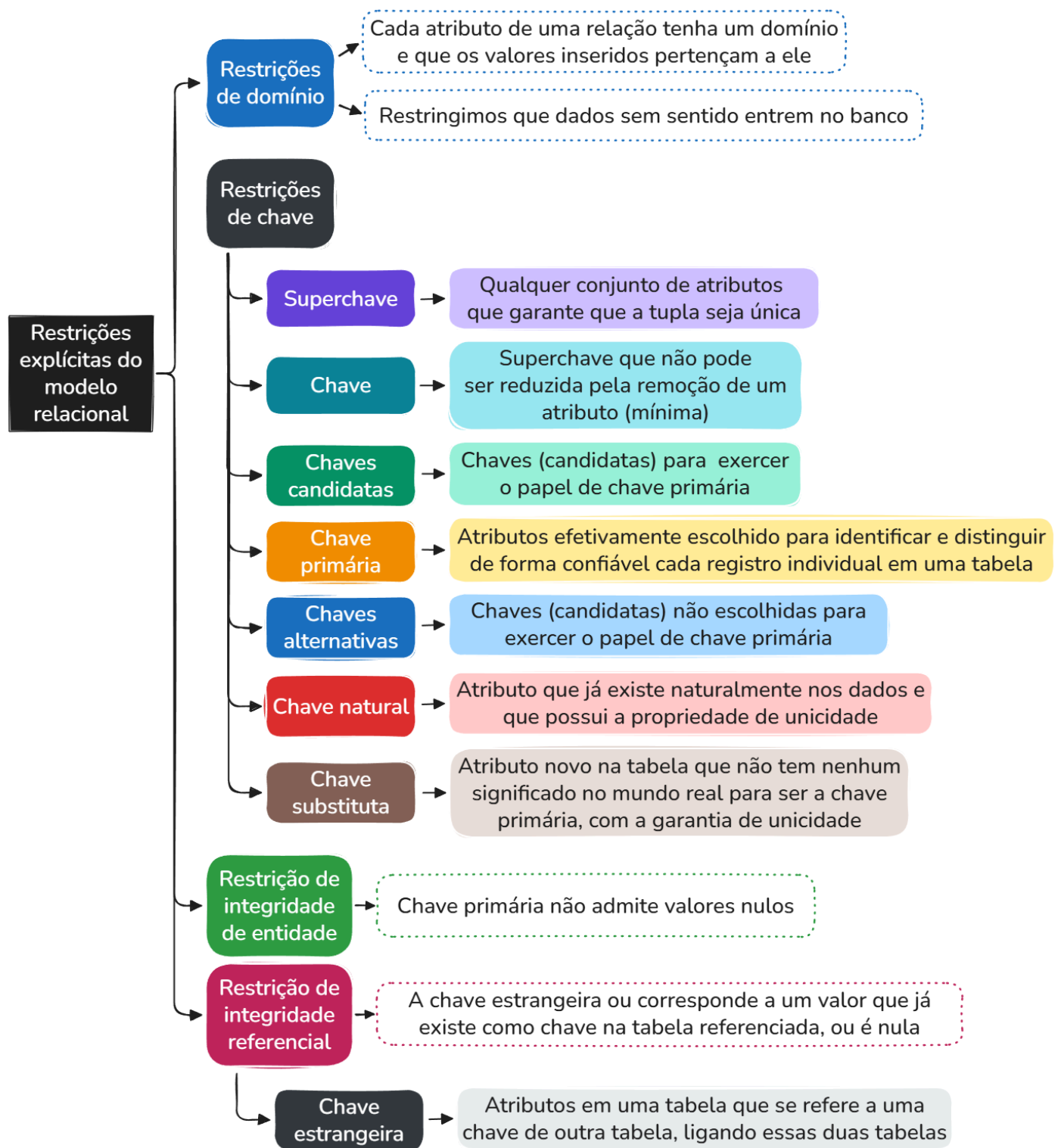
Restrição:

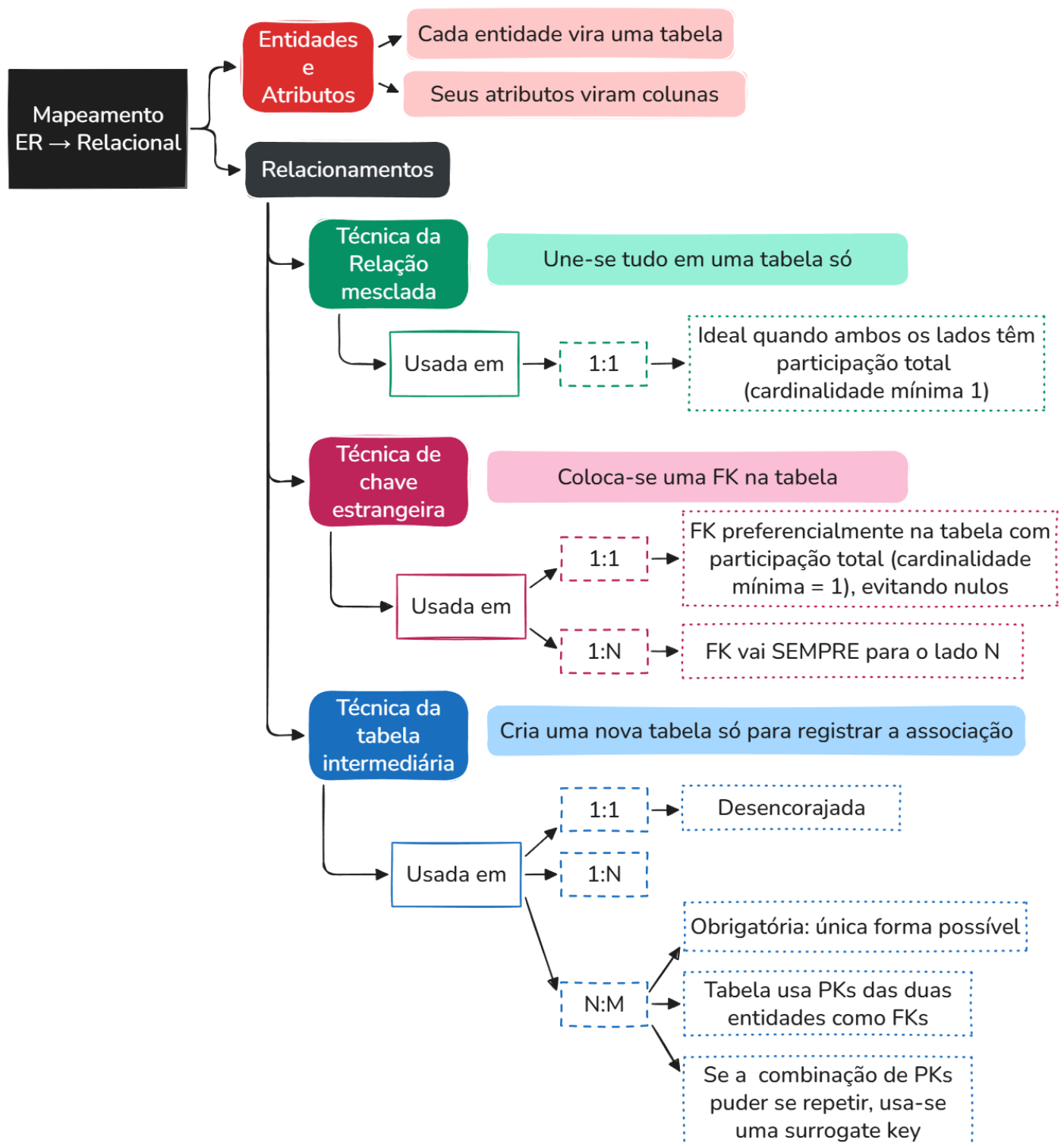
Não há tuplas duplicadas.

Restrição:

Uma relação representa fatos sobre entidades e sobre seus relacionamentos.

Propriedade	Chave primária	Chave única (alternativa)
Valores duplicados	Não permite	Não permite
Valores nulos	Não permite	Permite
Quantidade por tabela	Apenas uma	Várias
Papel	Identificador oficial da tupla	Garantia adicional de unicidade





Visões (Views)

O que é uma View?

- Tabela virtual/derivada: não armazena dados próprios
- É definida por uma consulta (SELECT) sobre tabelas base (ou outras views)
- Os dados são computados dinamicamente a cada acesso

Características principais

- Alterações nas tabelas base se refletem automaticamente na view — sem risco de defasagem
- Por isso são chamadas de visões dinâmicas
- Podem ser derivadas de tabelas base ou de outras views

Vantagem	Explicação
Segurança e controle de acesso	Uma visão expõe apenas os dados que deseja das tabelas que deseja. Assim, o administrador do banco pode conceder acesso à visão sem conceder acesso às tabelas base completas, restringindo o usuário a enxergar e manipular apenas o que lhe foi permitido, sem saber que existem outros dados por trás. Em resumo, a visão pode ser utilizada como uma espécie de “filtro de permissão”, definindo a fronteira entre o que é visível e o que permanece oculto para cada tipo de usuário.
Simplificação de consultas complexas	Quando uma consulta envolve uma lógica muito complexa, a visão encapsula toda essa complexidade em uma única tabela virtual, acessível facilmente pelo usuário. Em outras palavras, para o usuário final tudo se comporta como uma consulta simples sobre a visão, sem que ele precise compreender de fato como as tabelas base estão relacionadas internamente.
Agregação de dados	Uma visão pode apresentar dados já processados, como somas, médias, contagens, etc. Dessa forma, o usuário recebe os resultados prontos, como um “relatório permanente”, sempre atualizado e sempre disponível.
Abstração da estrutura real	Se as tabelas do banco de dados estão organizadas de forma pouco intuitiva, a visão permite apresentar esses mesmos dados de uma forma que seja natural e compreensível para quem os consome.

Vantagem	Explicação
Economia de espaço	Como a visão armazena apenas a definição da consulta e não uma cópia dos dados, o custo de armazenamento é mínimo . Não importa se a consulta da visão retorna milhões de tuplas, o espaço ocupado pela visão em si é apenas o necessário para guardar o texto da consulta no catálogo do sistema.
Adequação a diferentes perfis de usuários	Diferentes tipos de usuários têm diferentes necessidades e diferentes formas de pensar sobre os dados. Então, uma mesma base de dados pode ter várias visões distintas, se isso fizer sentido para quem acessa.

Views Materializadas

- Executa a consulta e armazena o resultado fisicamente em disco como uma tabela concreta
- Oposto da view comum: os dados existem de verdade, prontos para acesso direto

	View Comum	View Materializada
Armazenamento	Apenas a definição (virtual)	Dados físicos em disco
Acesso	Recomputa a consulta a cada acesso	Lê dados já prontos
Atualização	Sempre atualizada (dinâmica)	Pode ficar desatualizada
Desempenho	Menor (consulta roda do zero)	Maior (dados pré-computados)

As 12 regras de Codd

#	Nome	Descrição
0	Regra fundamental	Se um SGBD se apresenta como relacional, então ele deve ser capaz de realizar suas atividades apenas usando os princípios do modelo relacional.
1	Regra da informação	Toda informação é representada explicitamente no nível lógico e de exatamente uma forma: por valores em tabelas
2	Regra do acesso garantido	Garante-se o acesso a qualquer dado pela combinação de nome da tabela, nome da coluna e valor da chave primária.

#	Nome	Descrição
3	Regra do tratamento sistemático de valores nulos	O SGBD relacional deve suportar valores nulos (NULL) para representar informação ausente e informação inaplicável.
4	Regra do catálogo on-line baseado no modelo relacional	A descrição do banco de dados é representada no nível lógico da mesma forma que os dados "comuns".
5	Regra da sublinguagem ampla de dados	Deve existir pelo menos uma linguagem completa o suficiente para cobrir todas atividades essenciais com os dados sem depender de mecanismos externos.
6	Regra de atualização de visões	Todas as visões que são teoricamente atualizáveis devem ser atualizáveis pelo sistema relacional.
7	Regra da inserção, atualização, e exclusão de Alto nível	A inserção, atualização, e exclusão de informações devem ser permitidas para qualquer conjunto de valores e não apenas para uma única linha em uma única tabela.
8	Regra da independência física de dados	Não precisamos alterar os programas que acessam o banco de dados relacional quando mudanças são feitas na forma de armazenamento das suas informações ou em seus métodos de acesso.
9	Independência lógica de dados	Não precisamos alterar os programas que acessam o banco de dados relacional quando realizamos mudanças nas suas tabelas, desde que a informação original seja preservada após a alteração.
10	Regra da independência de integridade	Restrições de integridade dos dados devem ser definíveis na linguagem relacional e armazenadas no catálogo do sistema, e não nos programas de aplicação.
11	Regra da independência de distribuição	O usuário final não deve ser capaz de perceber que os dados estão distribuídos em vários locais.
12	A não transposição (ou subversão) das regras	Se o sistema possui uma linguagem de baixo nível, essa linguagem não pode ser usada para contornar as regras de integridade definidas na linguagem relacional de alto nível.

QUESTÕES COMENTADAS - FGV



1. (FGV/AAD (TCE,RR)/TCE RR/Tecnologia da Informação/Desenvolvimento de Sistemas/2025) Relacione as etapas de modelagem lógica e física de banco de dados às afirmativas a elas associadas

1. Modelagem Lógica.

2. Modelagem Física.

() É dependente do SGBD.

() Tem foco maior nas entidades e relacionamentos.

() Utiliza tipos de dados genéricos ou abstratos.

() Considera itens de desempenho tais como índices, particionamento, etc.

A relação correta, na ordem apresentada, é

a) 1 – 2 – 2 – 2.

b) 2 – 2 – 1 – 1.

c) 2 – 1 – 1 – 2.

d) 1 – 2 – 2 – 1.

e) 2 – 1 – 1 – 1.

Comentários:

(2) É dependente do SGBD.

A modelagem física é feita pensando em um banco específico (MySQL, Oracle, etc.), então depende do SGBD escolhido.

(1) Tem foco maior nas entidades e relacionamentos.

Na lógica, você define o quê existe: entidades (tabelas) e como elas se relacionam, sem se preocupar com detalhes técnicos.

(1) Utiliza tipos de dados genéricos ou abstratos.

Na lógica, há apenas "texto" ou "número", sem especificar VARCHAR(100) ou INT, por exemplo.

(2) Considera itens de desempenho tais como índices, particionamento, etc.

Isso é preocupação da física, que trata de como o banco vai funcionar de forma eficiente na prática.

Gabarito: C

2. (FGV/Tecno Jr (INPE)/INPE/Operação De Sistemas Espaciais Embarcados/2024)
Relativamente a Bancos de Dados Relacionais, analise os itens a seguir.

I. As tabelas são os componentes fundamentais de um banco de dados relacional e são usadas para organizar e armazenar dados de forma estruturada.

II. As tabelas podem estar relacionadas entre si por meio de chaves estrangeiras.

III. Os relacionamentos em um banco de dados relacional são conexões lógicas entre diferentes tabelas que permitem combinar dados e estão restritos a relacionamentos um-para-um entre tabelas.

IV. SQL (Structured Query Language) é uma linguagem usada exclusivamente para definir a estrutura dos dados, o que inclui a criação, alteração da estrutura e deleção de tabelas. Para manipular dados em um banco de dados relacional (consultar, inserir, deletar e alterar registros) é necessário utilizar outra linguagem.

V. A chave primária é uma coluna (ou conjunto de colunas) com valor único para cada linha de uma tabela e é usada para identificar registros de forma única.

Está correto o que se afirma em

a) I, II e III, apenas.

b) II, IV e V, apenas.

c) I, III e V, apenas.

d) I, II e V, apenas.

e) I, III e IV, apenas.

Comentários:

Item I: Certo

As tabelas são de fato a estrutura fundamental do modelo relacional, organizando os dados em linhas e colunas de forma estruturada. Toda informação em um banco de dados relacional é

representada por valores em tabelas, sendo essa a única forma de representação prevista pelo modelo.

Item II: Certo

As chaves estrangeiras são exatamente o mecanismo que estabelece vínculos entre tabelas distintas, garantindo a integridade referencial ao exigir que os valores referenciados existam na tabela pai.

Item III: Errado

Os relacionamentos em um banco de dados relacional não estão restritos a associações um-para-um. O modelo relacional suporta três tipos de razão de cardinalidade: 1:1, 1:N e M:N, sendo este último implementado por meio de uma tabela intermediária. Restringir os relacionamentos apenas ao tipo 1:1 seria uma limitação severa e incorreta do modelo.

Item IV: Errado

O SQL não é uma linguagem exclusiva de definição de estrutura. Ele é dividido em sublinguagens que cobrem todas as necessidades do banco: a DDL para definição de estruturas, a DML para manipulação de dados (consulta, inserção, atualização e exclusão), a DCL para controle de acesso e a DTL para controle de transações. Não é necessário usar outra linguagem para manipular dados.

Item V: Certo

A chave primária é o conjunto de um ou mais atributos escolhido para identificar de forma única cada tupla de uma tabela, sendo obrigatoriamente não nula e sem valores duplicados entre os registros.

Gabarito: D

3. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Governança/2024) Os principais sistemas de informação de uma organização armazenam seus dados transacionais em bancos de dados relacionais que são estruturados em tabelas que se relacionam entre si por meio da implementação de:

- a) chaves;
- b) índices;
- c) matrizes;
- d) métricas;
- e) nós e arestas.

Comentários:

- a) **Correta.** No modelo relacional, as tabelas se relacionam entre si por meio de chaves. Especificamente, a chave estrangeira é o mecanismo que cria o elo entre duas tabelas, referenciando a chave de outra tabela e permitindo assim que os dados se conectem.
- b) **Errada.** Índices não são o mecanismo de relacionamento entre tabelas. Esse tema vai além do conteúdo desta aula.
- c) **Errada.** Matrizes não fazem parte do vocabulário do modelo relacional, que trabalha com tabelas, tuplas e atributos.
- d) **Errada.** Métricas também não têm relação com a forma como tabelas se conectam no modelo relacional.
- e) **Errada.** Nós e arestas são os elementos do modelo em rede e do modelo hierárquico, não do modelo relacional. No modelo relacional, os dados são organizados em tabelas, não em estruturas de grafos.

Gabarito: A

4. (FGV/AFFC (STN)/STN/Tecnologia da Informação/Transformação Digital/2024) Com relação aos modelos de dados em aplicações de bancos de dados, a prática adotada em muitos casos é um modelo progressivo, baseado em níveis de abstração. Assinale esses níveis, na ordem em que são usualmente empregados nessa estratégia.

- a) Modelo conceitual – Modelo funcional – Modelo físico.
- b) Modelo conceitual – Modelo lógico – Modelo físico.
- c) Modelo funcional – Modelo conceitual – Modelo lógico.
- d) Modelo funcional – Modelo lógico – Modelo físico.
- e) Modelo lógico – Modelo físico – Modelo funcional.

Comentários:

- a) **Errada.** Não existe "modelo funcional" como nível de abstração na modelagem de dados. Os três níveis são conceitual, lógico e físico.
- b) **Correta.** A ordem correta vai do mais abstrato ao mais concreto: primeiro o modelo conceitual, que descreve o que o sistema precisa armazenar usando entidades e relacionamentos, sem se preocupar com tecnologia. Depois o modelo lógico, que traduz o conceitual para uma estrutura compatível com o tipo de banco escolhido, como o modelo relacional com suas tabelas. Por último o modelo físico, que define como os dados serão armazenados em um produto comercial específico, como MySQL ou Oracle.
- c) **Errada.** "Modelo funcional" não é um dos níveis de abstração de modelagem de dados.
- d) **Errada.** "Modelo funcional" não é um dos níveis de abstração de modelagem de dados.

e) **Errada.** A ordem está completamente errada, e "modelo funcional" não faz parte dos níveis de abstração de modelagem de dados.

Gabarito: B

5. (FGV/Res (TJ RJ)/TJ RJ/Tecnologia da Informação/2024) Com relação às três fases de um projeto de um novo banco de dados, avalie se as afirmativas a seguir são verdadeiras (V) ou falsas (F).

() O modelo conceitual pode ter a forma de um diagrama entidade-relacionamentos e captura as necessidades de uma organização em termos de armazenamento de dados independentemente da sua implementação.

() O projeto lógico tem como objetivo transformar o modelo conceitual obtido na primeira fase em um modelo lógico que definirá como o banco de dados será implementado em um SGBD.

() Na etapa do projeto físico, o modelo de banco de dados é enriquecido com detalhes que influenciam no desempenho do banco mas interferem em suas funcionalidades.

As afirmativas são, respectivamente,

a) V – F – F.

b) V – V – F.

c) F – F – F.

d) F – V – V.

e) F – F – V.

Comentários:

Item 1: Certo

O modelo conceitual é a primeira etapa do projeto de banco de dados e tem como objetivo representar as necessidades de informação de uma organização de forma abstrata, independentemente de qualquer tecnologia ou SGBD. O diagrama entidade-relacionamento é justamente a ferramenta mais utilizada nessa fase, permitindo representar entidades, atributos e relacionamentos do mundo real de forma visual e compreensível.

Item 2: Errado

O objetivo do projeto lógico está correto: transformar o modelo conceitual em um modelo lógico, como o relacional. Porém, a afirmativa erra ao dizer que o modelo lógico definirá *como o banco de dados será implementado em um SGBD*. O modelo lógico ainda é independente de um SGBD específico, ele define a estrutura dos dados em termos de tabelas, colunas e restrições, mas sem se preocupar com detalhes de implementação física. Quem define como o banco será de fato implementado em um SGBD é o projeto físico, etapa posterior.

Item 3: Errado

A primeira parte está correta: o projeto físico enriquece o modelo com detalhes que influenciam o desempenho, como estruturas de indexação, particionamento e métodos de armazenamento. Porém, a afirmativa erra ao afirmar que essas decisões *interferem nas funcionalidades* do banco. Na verdade, o princípio da independência física de dados garante exatamente o oposto: mudanças no nível físico não devem afetar as funcionalidades disponíveis no nível lógico.

Gabarito: B

6. (FGV/AFTRM (Pref Cuiabá)/Pref Cuiabá/Tecnologia da Informação/2024) Uma importante finalidade de um sistema gerenciador de banco de dados (SGBD) é fornecer aos usuários uma visão abstrata dos dados, classificada em níveis. Em relação ao tema, assinale a afirmativa correta.

a) O nível lógico de abstração descreve em detalhes estruturas complexas de como os dados são realmente armazenados nos dispositivos.

b) O nível lógico descreve o banco de dados inteiro em termos de um pequeno número de estruturas relativamente simples, que podem ser implementadas em estruturas complexas em nível físico, onde o usuário do nível lógico não precisa estar consciente desta complexidade.

c) O nível de abstração mais alto é o nível de visão (view). O SGBD pode fornecer várias visões para um mesmo banco de dados, definido de acordo com os mecanismos de busca dos dados.

d) Os administradores de banco de dados, que precisam decidir que informações armazenar no banco de dados, usam o nível físico de abstração.

e) Para ocultar detalhes do nível lógico do banco de dados, as visões fornecem um mecanismo de recuperação de modo a evitar que usuários acessem certas partes do banco de dados.

Comentários:

a) Errada. Descrever detalhes de como os dados são armazenados fisicamente é característica do nível físico, não do lógico.

b) Correta. O nível lógico descreve o banco de dados de forma mais simples, usando estruturas como tabelas e relacionamentos, sem que o usuário precise saber como os dados estão guardados fisicamente nos dispositivos.

c) Errada. O nível de abstração mais alto não é o de visão. A visão é um recurso que serve para filtrar e simplificar o acesso aos dados, mas o nível mais abstrato é justamente o conceitual.

d) Errada. Os administradores que decidem quais informações armazenar trabalham no nível conceitual, não no físico. O nível físico cuida de como os dados são guardados nos dispositivos.

e) **Errada.** As visões não servem para recuperação de dados no sentido descrito. Elas funcionam como um filtro que expõe apenas os dados permitidos para cada usuário, ocultando os demais, sem ser um mecanismo de recuperação.

Gabarito: B

7. (FGV/Ana (CVM)/CVM/Ciência de Dados/2024) O modelo relacional representa o banco de dados como uma coleção de relações. Considere a relação COLABORADOR apresentada a seguir, cuja chave primária é Matricula.

COLABORADOR

Nome	Matricula	Cpf	Celular	Endereco
Fernando Azul	1234	422.111.232-78	(21) 9888-9991	Rua das Flores, 8
Carla Vermelha	1357	444.222.333-77	(21) 9777-7474	Rua da Árvore, 4
Vivian Amarela	1458	147.854.111-12	NULL	Rua das Frutas, 2

Na relação COLABORADOR, o(a):

- a) superchave padrão é representada pelos atributos Matricula e Cpf;
- b) estado de relação atual reflete a cardinalidade dos domínios e o produto cartesiano;
- c) ordenação das tuplas faz parte da definição da relação sendo influenciada pela alteração da ordem das tuplas;
- d) domínio de cada atributo é um conjunto de valores atômicos, indivisível em se tratando de modelo relacional formal;
- e) restrição de integridade referencial garante que nenhum valor da chave primária será NULL.

Comentários:

a) **Errada.** A superchave padrão não é necessariamente a combinação de Matricula e Cpf. A superchave mais óbvia seria o conjunto de todos os atributos da relação, e uma chave candidata seria o atributo Matricula sozinho, já que ele é a chave primária.

b) **Errada.** O estado atual de uma relação é simplesmente o conjunto de tuplas existentes naquele momento, ou seja, os dados concretos armazenados. Não é definido pela cardinalidade dos domínios nem pelo produto cartesiano.

c) Errada. No modelo relacional, as tuplas de uma relação não possuem ordem. A ordenação não faz parte da definição formal da relação, e alterar a ordem das tuplas não afeta em nada a relação.

d) Correta. O modelo relacional exige que cada valor de atributo seja atômico, isto é, indivisível. Cada coluna guarda um único valor por linha, sem listas ou estruturas compostas.

e) Errada. A restrição que garante que nenhum valor da chave primária seja NULL é a integridade de entidade, não a referencial. A integridade referencial trata de outra coisa: garante que os valores de uma chave estrangeira correspondam a valores existentes na chave primária de outra tabela.

Gabarito: D

8. (FGV/Aud CE (TCE,PA)/TCE PA/Informática/Administrador de Banco de Dados/2024) Gerenciar a integridade de um banco de dados é fundamental para garantir a consistência, a precisão e a confiabilidade dos dados.

Sobre os diferentes modelos de gerência de integridade de um banco de dados, é correto afirmar que

a) o modelo de integridade semântica garante que os valores em uma coluna estejam dentro de um conjunto de valores permitidos.

b) o modelo de integridade de domínio assegura que cada linha de uma tabela seja única e identificável.

c) o modelo de integridade de entidade garante a integridade dos dados quando múltiplas transações são executadas simultaneamente.

d) o modelo de integridade de transações garante que as operações de uma transação não sejam executadas parcialmente.

e) o modelo de integridade construtiva garante a consistência das relações entre tabelas.

Comentários:

a) Errada. Garantir que os valores de uma coluna estejam dentro de um conjunto de valores permitidos é a definição de integridade de domínio, não de integridade semântica. As restrições semânticas são aquelas ligadas a regras de negócio do mundo real que não conseguem ser expressas no esquema do banco.

b) Errada. Garantir que cada linha de uma tabela seja única e identificável é o papel da integridade de chave, não da integridade de domínio. A integridade de domínio cuida dos valores válidos para cada coluna.

c) Errada. A integridade de entidade exige que a chave primária de cada tupla nunca seja nula, garantindo que todo registro possa ser identificado. Ela não tem relação com execução simultânea de transações.

d) **Correta.** A integridade de transações garante que as operações de uma transação sejam executadas por completo ou não sejam executadas de forma alguma, nunca de forma parcial.

e) **Errada.** Garantir a consistência das relações entre tabelas é o papel da integridade referencial. O termo "integridade construtiva" não corresponde a nenhum dos modelos estudados.

Gabarito: D

9. (FGV/Aud CE (TCE,PA)/TCE PA/Administrativa/Ciência de Dados/2024) Considere o Sistema de Concursos de uma empresa hipotética que adota um Sistema de Gerenciamento de Banco de Dados (SGDB) relacional.

A primeira tabela construída no banco foi Tabela_Candidato, que possui os seguintes campos:

candidato_ID;

candidato_Nome;

candidato_CPF.

A segunda tabela construída foi a Tabela_Concurso, que possui os seguintes campos:

concurso_ID;

candidato_ID;

concurso_Nome;

concurso_Cargo.

No contexto dos bancos de dados relacionais, é correto afirmar que

a) o campo candidato_ID é uma chave estrangeira na Tabela_Candidato.

b) o campo candidato_ID é uma chave primária na Tabela_Concurso.

c) a Tabela_Candidato está incompleta, haja vista que não possui informação do concurso que o candidato pode fazer.

d) dá forma como o banco de dados foi concebido, só é possível lançar um concurso por candidato.

e) a inserção da chave estrangeira candidato_ID na tabela Tabela_Concurso, permite o lançamento de um candidato em vários concursos.

Comentários:

a) **Errada.** O candidato_ID é a chave primária da Tabela_Candidato, não uma chave estrangeira. Chave estrangeira é um atributo que referencia a chave de outra tabela, e é exatamente o que acontece com o candidato_ID na Tabela_Concurso.

b) Errada. O candidato_ID na Tabela_Concurso é uma chave estrangeira, pois referencia a chave primária da Tabela_Candidato. A chave primária da Tabela_Concurso seria o concurso_ID.

c) Errada. A Tabela_Candidato não precisa ter informações do concurso dentro dela. Justamente para isso existe a chave estrangeira na Tabela_Concurso, criando a ligação entre as duas tabelas.

d) Errada. É o contrário: como o candidato_ID aparece como chave estrangeira na Tabela_Concurso, um mesmo candidato pode aparecer em várias linhas dessa tabela, ou seja, em vários concursos.

e) Correta. Ao colocar o candidato_ID como chave estrangeira na Tabela_Concurso, cada linha dessa tabela representa um candidato participando de um concurso. Como o mesmo candidato_ID pode aparecer em várias linhas diferentes, um candidato pode ser lançado em vários concursos distintos.

Gabarito: E

10. (FGV/AJ (TRF 1ª Região)/TRF 1/Apoio Especializado/Suporte em Tecnologia da Informação/2024) Pedro está implementando um banco de dados a partir de um projeto lógico que recebeu dos desenvolvedores de Sistemas de Informação do Tribunal. No projeto, há relacionamentos entre diversas tabelas.

No SQL, para implementar os relacionamentos de modo a prevenir que alguma transação de banco de dados destrua os relacionamentos entre as tabelas, Pedro deve utilizar a cláusula:

a) CHECK

b) UNION

c) DEFAULT

d) ORDER BY

e) FOREIGN KEY

Comentários:

a) Errada. CHECK é uma restrição usada para validar valores inseridos em uma coluna, não para criar relacionamentos entre tabelas.

b) Errada. UNION é uma operação para combinar resultados de consultas, não tem relação com a criação de vínculos entre tabelas.

c) Errada. DEFAULT define um valor padrão para uma coluna quando nenhum valor é informado, não serve para relacionar tabelas.

d) Errada. ORDER BY é usado para ordenar os resultados de uma consulta, não tem função de proteção de relacionamentos.

e) **Correta.** A cláusula FOREIGN KEY é justamente o que define uma chave estrangeira, que é o mecanismo do modelo relacional para criar ligações entre tabelas. Ao declarar uma chave estrangeira, o banco garante a integridade referencial, ou seja, impede que existam valores na tabela filha que não correspondam a nenhum registro na tabela pai, protegendo assim os relacionamentos entre as tabelas.

Gabarito: E

11. (FGV/APP GG (SEPLAG Niterói)/Pref Niterói/Gestão de Tecnologia da Informação e Comunicação/2024) As restrições de integridade são fundamentais para garantir a confiabilidade dos dados em um banco de dados. Elas definem regras que os dados devem obedecer, assegurando sua precisão e consistência.

Assinale a opção que indica a restrição de integridade que limita os valores que um atributo pode assumir.

- a) Restrições de Chave.
- b) Restrições de Valores.
- c) Restrições de Domínio.
- d) Restrições de Entidade.
- e) Restrições de Integridade Referencial.

Comentários:

a) **Errada.** Restrições de chave garantem que as tuplas sejam únicas dentro de uma relação, definindo atributos que identificam cada registro. Não têm relação com os valores permitidos para um atributo.

b) **Errada.** Esse nome não corresponde a nenhum tipo de restrição do modelo relacional.

c) **Correta.** A restrição de domínio exige que cada atributo tenha um conjunto de valores permitidos e que apenas valores pertencentes a esse conjunto possam ser inseridos. Por exemplo, definir que o atributo "idade" aceita apenas números inteiros.

d) **Errada.** A restrição de integridade de entidade exige que a chave primária nunca seja nula. Ela não trata dos valores permitidos para os atributos em geral.

e) **Errada.** A integridade referencial exige que o valor de uma chave estrangeira corresponda a um valor existente na tabela referenciada. Isso garante consistência entre tabelas, não limita os valores de um atributo qualquer.

Gabarito: C

12. (FGV/Ana Leg (ALETO)/ALETO/Suporte Técnico em Informática/2024) Na abordagem de sistemas de banco de dados, a arquitetura de três esquemas (ou arquitetura ANSI/SPARC) sustenta a abstração entre seus níveis.

Nesse contexto, o conceito que define a capacidade de alteração do esquema conceitual da arquitetura de três esquemas, sem repercussão nos esquemas externos ou nos programas de aplicação é definido como independência de dados

- a) conceitual.
- b) de usuário.
- c) física.
- d) lógica.
- e) visual.

Comentários:

- a) Errada.** Independência conceitual não é um tipo de independência definido nesse contexto.
- b) Errada.** Independência de usuário não é um tipo de independência definido nesse contexto.
- c) Errada.** A independência física trata de mudanças no nível de armazenamento em disco, como reorganizar arquivos ou mudar métodos de acesso, sem afetar os programas. O enunciado fala de mudanças no esquema conceitual, não no físico.
- d) Correta.** A independência lógica é justamente a capacidade de alterar a estrutura lógica do banco sem repercutir nos programas de aplicação, desde que a informação original seja preservada.
- e) Errada.** Independência visual não é um tipo de independência definido nesse contexto.

Gabarito: D

13. (FGV/Ana Leg (ALETO)/ALETO/Desenvolvimento de Sistema/2024) Visões em SQL possuem um papel relevante em bancos de dados relacionais. Ela oferece mecanismos importantes, tais como abstração de dados, segurança e simplificação das consultas. Nesse contexto, é correto afirmar que

- a) a cláusula WITH CHECK OPTION da SQL é demandada no final da especificação da visão, a fim de que a tornar uma asserção.
- b) a estratégia de modificação de consultas para a implementação de visões é eficaz para consultas complexas.
- c) a responsabilidade pela atualização do estado de uma visão é do usuário, que necessita enviar um comando refresh a cada execução.
- d) uma visão em linha (view em linha) consiste na definição de uma visão na cláusula HAVING de uma consulta SQL.
- e) visões materializadas criam uma tabela temporária, utilizando técnicas como a atualização

incremental.

Comentários:

- a) **Errada.** A cláusula WITH CHECK OPTION não é um tema abordado no conteúdo desta aula.
- b) **Errada.** A estratégia de modificação de consultas para visões complexas é justamente problemática, não eficaz. Visões complexas podem perder a rastreabilidade às tabelas base, tornando modificações impossíveis.
- c) **Errada.** A responsabilidade pela atualização não é do usuário. A visão comum se atualiza automaticamente a cada acesso, pois busca os dados diretamente nas tabelas base naquele momento, sem precisar de nenhum comando manual.
- d) **Errada.** View em linha e cláusula HAVING não são temas abordados no conteúdo desta aula.
- e) **Correta.** A visão materializada executa a consulta e armazena o resultado como uma tabela física em disco, diferente da visão comum que é virtual. Como os dados ficam gravados fisicamente, eles podem ficar desatualizados em relação às tabelas base e precisam passar por um processo de atualização periódica para sincronização, o que é compatível com a ideia de atualização incremental mencionada na alternativa.

Gabarito: E

14. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Na fase de projeto de um banco de dados relacional, é comum que seja construído um modelo E-R, pois isso permite uma compreensão lógica das entidades envolvidas e suas respectivas propriedades. Uma das vantagens desses modelos é a identificação imediata da lista de:

- a) chaves estrangeiras a serem estabelecidas;
- b) índices auxiliares para as eventuais buscas;
- c) permissões de acesso a serem concedidas;
- d) triggers a serem implementados;
- e) tipos de dados adequados a cada um dos atributos.

Comentários:

- a) **Correta.** O modelo E-R mostra as entidades e seus relacionamentos. Quando mapeamos esse modelo para o relacional, precisamos decidir como representar esses relacionamentos entre tabelas, e a principal forma de fazer isso é justamente por meio de chaves estrangeiras. Ou seja, ao olhar para o diagrama E-R, já conseguimos identificar quais tabelas precisarão se conectar e, portanto, quais chaves estrangeiras serão necessárias.
- b) **Errada.** Índices auxiliares são decisões do nível físico do banco de dados, não do modelo E-R, que opera no nível conceitual.

c) Errada. Permissões de acesso não fazem parte do modelo E-R, que se preocupa apenas com as entidades, atributos e relacionamentos do mundo real.

d) Errada. Triggers são mecanismos relacionados às restrições semânticas e regras de negócio, implementados fora do esquema do modelo, não sendo identificados no modelo E-R.

e) Errada. Os tipos de dados de cada atributo são definidos na modelagem lógica relacional, como as restrições de domínio, e não no modelo E-R, que é conceitual e abstrato, sem se preocupar com detalhes técnicos de armazenamento.

Gabarito: A

15. (FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/Banco de Dados/2023) O administrador de banco de dados do TJSE deverá projetar um banco de dados relacional para armazenar dados, oriundos do novo sistema de gestão de processos. O banco de dados armazenará dados sobre processos, autor, advogado, ocorrência, tipo da ação, município, situação, dentre outros. Para atender esses requisitos, o administrador deverá criar um banco de dados:

a) com uma única tabela para armazenar todos os dados;

b) separado para cada tabela, distribuindo os dados entre os bancos de dados;

c) com um esquema lógico das tabelas relacionadas, sem definir as chaves estrangeiras;

d) com todas as tabelas desnormalizadas para reduzir a redundância de dados;

e) com um esquema lógico das tabelas relacionadas, mantendo a integridade referencial.

Comentários:

a) Errada. Um banco de dados relacional é composto por várias tabelas relacionadas entre si, não por uma única tabela com tudo misturado. Isso causaria enorme redundância de dados.

b) Errada. As tabelas de um banco de dados relacional ficam juntas em um mesmo banco, relacionadas umas com as outras, e não espalhadas em bancos separados.

c) Errada. Sem definir chaves estrangeiras, as tabelas ficam sem ligação entre si, e é exatamente a chave estrangeira que cria o elo entre elas, garantindo que os dados referenciados existam de verdade.

d) Errada. Tabelas desnormalizadas aumentam a redundância de dados, não a reduzem. O objetivo do modelo relacional é justamente o contrário.

e) Correta. Um banco de dados relacional bem projetado possui um esquema lógico com várias tabelas relacionadas entre si. A integridade referencial garante que os valores das chaves estrangeiras correspondam a valores existentes nas tabelas referenciadas, mantendo os dados consistentes e sem referências inválidas.

Gabarito: E

16. (FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/Web designer/2023) Julia é uma WebDesigner e recebeu um modelo de dados com a seguinte especificação de um campo a ser armazenado no banco de dados DB_ProcessJud do sistema SIS_ProcessJud. NUM_PROCESSO, INTEGER, NOT NULL, PRIMARY KEY A partir do modelo de dados recebido, Julia pode concluir que NUM_PROCESSO é um campo:

- a) inteiro de acesso reservado;
- b) do tipo caractere com tamanho fixo;
- c) utilizado como chave de acesso ao sistema SIS_ProcessJud;
- d) que permite o armazenamento de valores nulos em determinadas circunstâncias;
- e) identificador único de um registro em uma tabela do banco de dados DB_ProcessJud.

Comentários:

a) Errada. PRIMARY KEY não tem nada a ver com acesso reservado ao sistema. Ela serve para identificar de forma única cada registro dentro de uma tabela.

b) Errada. O tipo definido é INTEGER, ou seja, número inteiro. Caractere com tamanho fixo seria outro tipo de dado, como CHAR.

c) Errada. A chave primária identifica registros dentro de uma tabela do banco de dados, não serve como chave de acesso ou login ao sistema SIS_ProcessJud.

d) Errada. A especificação traz duas restrições que juntas proíbem valores nulos: NOT NULL, que impede explicitamente o campo de ficar vazio, e PRIMARY KEY, que por definição nunca permite valores nulos.

e) Correta. PRIMARY KEY é a chave primária, que é exatamente o conjunto de atributos escolhido para identificar e distinguir de forma única cada registro individual em uma tabela. Como também está marcado como NOT NULL, nenhum registro pode ficar sem esse valor.

Gabarito: E

17. (FGV/FR (Pref RJ)/Pref RJ/2023) Em bancos de dados relacionais, os relacionamentos entre entidades identificadas na modelagem de dados são explicitados no SQL por meio de:

- a) Cláusulas Check;
- b) Cláusulas Unique;
- c) Foreign keys;
- d) Primary keys;
- e) Views.

Comentários:

- a) **Errada.** Cláusula Check serve para definir restrições de domínio, ou seja, limitar quais valores são aceitos em uma coluna. Não é usada para ligar tabelas entre si.
- b) **Errada.** Cláusula Unique define uma chave alternativa, garantindo que os valores de uma coluna não se repitam. Não representa relacionamentos entre tabelas.
- c) **Correta.** A chave estrangeira é justamente o mecanismo que cria a ligação entre duas tabelas. Ela armazena o valor da chave de outra tabela, representando o relacionamento entre as entidades no banco de dados relacional.
- d) **Errada.** A chave primária identifica de forma única cada linha dentro de uma mesma tabela. Ela é o ponto de referência para a chave estrangeira, mas por si só não representa o relacionamento entre tabelas.
- e) **Errada.** Views são tabelas virtuais que apresentam dados de outras tabelas de forma filtrada ou simplificada. Elas não criam nem representam relacionamentos entre entidades.

Gabarito: C

18. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Sobre a utilização de views, em bancos de dados como MySQL, SQL Server, analise as afirmativas a seguir.

- I. Views servem para construir e armazenar o conteúdo de consultas complexas de uso frequente.
- II. Os principais SGBD não permitem o uso de operações de update/delete sobre views.
- III. Em geral, as views não podem ser referenciadas na cláusula FROM em outras consultas SQL.

Está correto somente o que se afirma em:

- a) I;
- b) II;
- c) III;
- d) I e II;
- e) II e III.

Comentários:

- I. **Correta.** Uma view encapsula uma consulta complexa e a disponibiliza como uma tabela virtual de uso frequente, sem precisar reescrever essa consulta toda vez.
- II. **Errada.** Só em casos específicos que os SGBDs atuam de forma conservadora e simplesmente bloqueiam as atualizações em views para evitar possíveis erros, como quando uma visão atua sobre atributos de várias tabelas. Nesses casos, a modificação não é permitida.

I Errada. Uma view se comporta como uma tabela para quem a consulta. Nada impede que seja referenciada na cláusula FROM de outras consultas, inclusive uma visão pode ser derivada de outras visões.

Gabarito: A

19. (FGV/Ag TE (SEFAZ BA)/SEFAZ BA/Tecnologia da Informação/2022) Considerando as restrições do modelo de dados relacionais e os esquemas de bancos de dados relacionais, aquelas que não podem ser expressas diretamente nos modelos de dados e, portanto, devem ser expressas e impostas pelos programas de aplicação, denominam-se restrições

a) inerentes.

b) explícitas.

c) de atributo.

d) semânticas;

e) de domínio.

Comentários:

a) Errada. As restrições inerentes são aquelas que vêm de fábrica com o próprio modelo, ou seja, não precisam ser declaradas por ninguém porque já fazem parte do modelo por natureza.

b) Errada. As restrições explícitas são aquelas que o modelo exige, mas que precisamos declarar na criação da estrutura do banco usando linguagem como SQL, por exemplo, definindo o tipo de dado de cada coluna.

c) Errada. Restrição de atributo não é o nome utilizado para esse tipo de restrição descrita no enunciado.

d) Correta. As restrições semânticas, também chamadas de regras de negócio, são justamente aquelas que não conseguimos expressar no esquema do banco de dados. Elas envolvem lógica do mundo real que precisa ser implementada no código do programa que acessa o banco, como a regra de que um professor perde o ponto se atrasar mais de 5 minutos.

e) Errada. As restrições de domínio dizem respeito ao conjunto de valores permitidos para cada atributo, como definir que o campo idade aceita apenas números inteiros. Elas são um tipo de restrição explícita, pois são declaradas no esquema do banco.

Gabarito: D

20. (FGV/Tec Leg (CM Taubaté)/CM Taubaté/Informática/2022) No contexto dos bancos de dados relacionais, analise a declaração de uma chave estrangeira na criação de uma tabela T2, exibida a seguir.

CONSTRAINT FK_TESTE FOREIGN KEY (A) REFERENCES T1(A)

Considere ainda as seguintes afirmativas a respeito desse comando e suas consequências:

- I. A coluna A na tabela T1 deve ser definida como chave primária ou equivalente.
- II. A coluna A da tabela T2 não pode conter valores nulos.
- III. A coluna A da tabela T2 não pode ser declarada como chave primária ou equivalente.

Está correto somente o que se afirma em

- a) I.
- b) II.
- c) III.
- d) I e II.
- e) II e III.

Comentários:

Item I: Certo

Para que uma coluna possa ser referenciada por uma chave estrangeira, ela precisa ser uma chave primária ou possuir uma restrição de unicidade equivalente na tabela referenciada. Isso é necessário porque a integridade referencial exige que cada valor da chave estrangeira aponte para um único registro identificável na tabela pai. Se a coluna referenciada permitisse duplicatas, a associação seria ambígua.

Item II: Errado

A coluna A da tabela T2, por ser uma chave estrangeira, pode sim conter valores nulos. O NULL em uma chave estrangeira indica simplesmente que aquela tupla não está associada a nenhum registro da tabela referenciada, o que é uma situação válida e não viola a restrição de integridade referencial. A proibição de valores nulos é característica da chave primária, não da chave estrangeira.

Item III: Errado

Não há nenhuma restrição que impeça uma coluna de ser simultaneamente chave estrangeira e chave primária ou equivalente. Isso ocorre com frequência na prática, especialmente no mapeamento de relacionamentos 1:1, em que a chave primária de uma tabela referencia a chave primária da outra, e também em tabelas intermediárias de relacionamentos M:N, em que as chaves estrangeiras formam juntas a chave primária composta da tabela.

Gabarito: A

21. (FGV/Ana (MPE SC)/MPE SC/Dados e Pesquisas/2022) Num banco de dados relacional, considere as tabelas T1 e T2, criadas como descrito a seguir.

T1 tem duas colunas, intituladas A e B, do tipo inteiro; a coluna A é declarada como primary key, e não aceita valores nulos.

T2 tem duas colunas, intituladas C e A, do tipo inteiro; a coluna C é declarada como primary key, e não aceita valores nulos; a coluna A foi declarada como UNIQUE, não aceita valores nulos e ainda foi declarada como uma foreign key que referencia a coluna A da tabela T1.

À luz dessa estrutura, é correto afirmar que o relacionamento entre T1 e T2:

- a) é do tipo 1:1;
- b) é do tipo 1:N;
- c) é do tipo N:1;
- d) é do tipo M:N;
- e) não pode ser categorizado, pois isso depende das instâncias de cada tabela.

Comentários:

a) Correta. A coluna A em T2 é UNIQUE, o que significa que cada valor dela só pode aparecer uma vez em T2. Isso quer dizer que cada registro de T1 pode estar associado a no máximo um registro de T2, e cada registro de T2 aponta para exatamente um registro de T1. Dos dois lados, a associação é de no máximo 1 para 1, caracterizando um relacionamento 1:1.

b) Errada. O relacionamento 1:N ocorre quando um registro de um lado pode estar associado a vários do outro. Aqui isso não acontece porque a coluna A em T2 é UNIQUE, impedindo que dois registros de T2 apontem para o mesmo registro de T1.

c) Errada. N:1 seria o inverso do 1:N, o que também não se aplica pelo mesmo motivo: a restrição UNIQUE na coluna A de T2 impede múltiplas referências ao mesmo registro de T1.

d) Errada. O relacionamento M:N exigiria que vários registros de cada lado pudessem se associar a vários do outro lado. A restrição UNIQUE elimina completamente essa possibilidade.

e) Errada. A estrutura das tabelas já é suficiente para categorizar o relacionamento. A restrição UNIQUE na chave estrangeira define o tipo de relacionamento independentemente dos dados que serão inseridos.

Gabarito: A

22. (FGV/Ag TE (SEFAZ BA)/SEFAZ BA/Tecnologia da Informação/2022) Com relação aos conceitos de banco de dados relacionais, analise as afirmativas a seguir.

I. Instância do banco se refere à supressão de detalhes da organização e do armazenamento de dados, descartando para um melhor conhecimento desses dados os recursos essenciais.

II. Modelo de dados se refere a uma coleção de conceitos que podem ser utilizados para

descrever a estrutura de um banco de dados, oferecendo os meios necessários para alcançar essa abstração.

III. Abstração de dados refere aos conjuntos de dados e metadados e usuários presentes no servidor de dados em um determinado instante.

Está correto o que se afirma em

- a) I, somente.
- b) II, somente.
- c) III, somente.
- d) I e II, somente.
- e) I e III, somente.

Comentários:

Item I: Errado

A definição apresentada não corresponde ao conceito de instância do banco de dados. A instância refere-se ao conjunto de dados efetivamente armazenados no banco em um determinado momento, ou seja, uma "fotografia" do estado atual dos dados. O que o item descreve — supressão de detalhes de organização e armazenamento para destacar os aspectos essenciais — é o conceito de abstração de dados.

Item II: Certo

O modelo de dados é de fato uma coleção de conceitos utilizados para descrever a estrutura de um banco de dados, incluindo os tipos de dados, os relacionamentos entre eles e as restrições que devem ser respeitadas. É justamente por meio do modelo de dados que se alcança a abstração, permitindo representar a realidade de forma estruturada e compreensível.

Item III: Errado

A definição apresentada não corresponde ao conceito de abstração de dados. O que o item descreve, o conjunto de dados e metadados presentes no servidor em um determinado instante, é o conceito de instância do banco de dados. A abstração de dados, por sua vez, é o mecanismo que permite ocultar detalhes de implementação e armazenamento, expondo ao usuário apenas os aspectos essenciais e relevantes dos dados.

Gabarito: B

23. (FGV/Ana (FunSaúde CE)/FunSaúde CE/II e Infraestrutura de Informática/2021) Considere a definição e as instâncias das tabelas de bancos de dados CLUBE e JOGO exibidas a seguir.

nome
Barcelona
Boca Juniors
The Strongest

JOGO

mandante	visitante	golsM	golsV
Barcelona	Boca Juniors	1	0
Barcelona	The Strongest	NULL	NULL
Boca Juniors	Barcelona	0	0
Boca Juniors	The Strongest	3	0
The Strongest	Barcelona	2	0
The Strongest	Boca Juniors	2	0

Cada clube deve jogar quatro vezes, duas como mandante e duas como visitante.

As colunas golsM e golsV registram o número de gols dos times mandantes e visitantes, respectivamente, em cada jogo. Ambas são nulas enquanto o jogo não for realizado.

Em relação às instâncias das tabelas CLUBE e JOGO, definidas anteriormente, considere as constraints a seguir.

- I. constraint pk1 primary key (mandante, visitante)
- II. constraint fk1 foreign key (mandante) references clube
- III. constraint fk2 foreign key (visitante) references clube

Dessas constraints, podem ser aplicadas à tabela JOGO:

- a) I, apenas.
- b) I e II, somente.
- c) I e III, somente.

- d) II e III, somente.
- e) I, II e III.

Comentários:

As três constraints podem ser aplicadas. A constraint I define a chave primária composta pelas colunas mandante e visitante juntas, pois nenhuma delas sozinha identifica uma linha de forma única, mas a combinação das duas sim (não há jogos repetidos da mesma forma). A constraint II define mandante como chave estrangeira referenciando a tabela CLUBE, o que é válido porque todo valor de mandante existe na tabela CLUBE. A constraint III faz o mesmo para visitante, que também sempre corresponde a um clube existente na tabela CLUBE. Nenhuma das três viola qualquer regra do modelo relacional.

Gabarito: E

24. (FGV/Ana TI (BANESTES)/BANESTES/Desenvolvimento de Sistemas/2021) Na questão, considere as tabelas T1, T2 e T3, cujas estruturas e instâncias são exibidas a seguir. O valor NULL deve ser tratado como unknown (desconhecido).

Suponha que as tabelas T1 e T2, descritas anteriormente, tenham sido declaradas com as colunas A e C, respectivamente, definidas como chaves primárias.

Para tanto, a definição de chaves estrangeiras na tabela T3 deveria ter como base o script:

- a) foreign key (A) references T1(A),
foreign key (C) references T2(C)
- b) foreign key (A, C) references T1(A), T2(C)
- c) foreign key T1(A) for A,
foreign key T2(C) for C
- d) foreign key T1(A) for A, T2(C) for C
- e) foreign key T1(A) references (A),
foreign key T2(C) references (C)

Comentários:

a) **Correta.** A sintaxe correta para declarar chave estrangeira é: foreign key (coluna na tabela atual) references tabela_pai(coluna da tabela pai). Aqui, a tabela T3 possui a coluna A que referencia a chave primária A de T1, e a coluna C que referencia a chave primária C de T2. Cada chave estrangeira é declarada separadamente.

b) Errada. Não existe a sintaxe de referenciar duas tabelas diferentes em um único comando foreign key. Cada chave estrangeira deve ser declarada individualmente, apontando para uma única tabela referenciada.

c) Errada. A sintaxe está invertida e incorreta. O comando foreign key não usa a estrutura tabela(coluna) seguida de for coluna.

d) Errada. Assim como a alternativa anterior, a sintaxe está errada. Não se declara chave estrangeira colocando o nome da tabela logo após foreign key.

e) Errada. A sintaxe está invertida. No comando references, o que deve aparecer entre parênteses é a coluna da tabela pai, e o nome da tabela pai deve vir antes dos parênteses, não depois de references.

Gabarito: A

25. (FGV/Sup (IMBEL)/IMBEL/Tecnologia da Informação/2021) Na criação de esquemas de bancos de dados relacionais a partir de modelos/diagramas de entidades e relacionamentos, a implementação física de relacionamentos M:N é usualmente feita por meio de

a) chaves primárias.

b) índices não únicos.

c) restrições de validação.

d) tabelas.

e) triggers.

Comentários:

a) Errada. Chaves primárias servem para identificar de forma única cada linha de uma tabela, não para representar o relacionamento entre duas entidades.

b) Errada. Índices são estruturas que aceleram buscas no banco de dados, não têm relação com o mapeamento de relacionamentos M:N.

c) Errada. Restrições de validação servem para garantir que os dados inseridos obedeçam a certas regras, como tipos de dado ou valores permitidos, não para implementar relacionamentos.

d) Correta. No relacionamento M:N, a única forma de fazer o mapeamento é criando uma tabela intermediária. Essa tabela recebe as chaves primárias das duas tabelas originais, ambas funcionando como chaves estrangeiras, e a combinação dessas duas chaves forma a chave primária composta da tabela intermediária.

e) Errada. Triggers são códigos executados automaticamente em resposta a eventos no banco, não são o mecanismo usado para implementar relacionamentos M:N.

Gabarito: D

26. (FGV/SAD PE/Controle Interno/2009) Modelagem de Dados é uma parte importante do desenho de um sistema de informação, que especifica as estruturas de dados e as regras necessárias para suportar uma área de negócios e que representa um conjunto de requerimentos de informações desse negócio.

A respeito dos modelos existentes, analise as afirmativas a seguir:

I. trata de regras de normalização das estruturas de dados, derivação de relacionamentos e de estruturas de agregação e generalização-especialização, restrição de domínio, de Integridade e de Implementação;

II. trata da análise das características e dos recursos necessários para armazenamento real e manipulação das estruturas de dados.

Os modelos I e II são conhecidos, respectivamente, por:

- a) conceitual e hierárquico.
- b) relacional e conceitual.
- c) hierárquico e lógico.
- d) físico e relacional.
- e) lógico e físico.

Comentários:

a) Errada. O modelo conceitual é o mais abstrato, focado em entidades e relacionamentos do mundo real, sem falar em normalização ou armazenamento físico.

b) Errada. O modelo relacional organiza dados em tabelas e não é o mesmo que modelo lógico, e o conceitual não trata de armazenamento real.

c) Errada. O modelo hierárquico organiza dados em estrutura de árvore com nós e arestas, e não se relaciona com armazenamento físico como descrito na afirmativa II.

d) Errada. O modelo físico descreve como os dados são armazenados em um produto específico, e não trata das regras descritas na afirmativa I.

e) Correta. O modelo lógico é o intermediário que traduz o modelo conceitual para uma estrutura compatível com o paradigma escolhido, definindo regras como restrições de domínio e integridade, conforme a afirmativa I. Já o modelo físico descreve como os dados serão de fato armazenados e manipulados em um produto comercial específico, exatamente como descreve a afirmativa II.

Gabarito: E

LISTA DE QUESTÕES - FGV



1. (FGV/AAD (TCE,RR)/TCE RR/Tecnologia da Informação/Desenvolvimento de Sistemas/2025) Relacione as etapas de modelagem lógica e física de banco de dados às afirmativas a elas associadas

1. Modelagem Lógica.

2. Modelagem Física.

() É dependente do SGBD.

() Tem foco maior nas entidades e relacionamentos.

() Utiliza tipos de dados genéricos ou abstratos.

() Considera itens de desempenho tais como índices, particionamento, etc.

A relação correta, na ordem apresentada, é

a) 1 – 2 – 2 – 2.

b) 2 – 2 – 1 – 1.

c) 2 – 1 – 1 – 2.

d) 1 – 2 – 2 – 1.

e) 2 – 1 – 1 – 1.

2. (FGV/Tecno Jr (INPE)/INPE/Operação De Sistemas Espaciais Embarcados/2024) Relativamente a Bancos de Dados Relacionais, analise os itens a seguir.

I. As tabelas são os componentes fundamentais de um banco de dados relacional e são usadas para organizar e armazenar dados de forma estruturada.

II. As tabelas podem estar relacionadas entre si por meio de chaves estrangeiras.

III. Os relacionamentos em um banco de dados relacional são conexões lógicas entre diferentes tabelas que permitem combinar dados e estão restritos a relacionamentos um-para-um entre tabelas.

IV. SQL (Structured Query Language) é uma linguagem usada exclusivamente para definir a estrutura dos dados, o que inclui a criação, alteração da estrutura e deleção de tabelas. Para manipular dados em um banco de dados relacional (consultar, inserir, deletar e alterar registros) é necessário utilizar outra linguagem.

V. A chave primária é uma coluna (ou conjunto de colunas) com valor único para cada linha de uma tabela e é usada para identificar registros de forma única.

Está correto o que se afirma em

- a) I, II e III, apenas.
- b) II, IV e V, apenas.
- c) I, III e V, apenas.
- d) I, II e V, apenas.
- e) I, III e IV, apenas.

3. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Governança/2024) Os principais sistemas de informação de uma organização armazenam seus dados transacionais em bancos de dados relacionais que são estruturados em tabelas que se relacionam entre si por meio da implementação de:

- a) chaves;
- b) índices;
- c) matrizes;
- d) métricas;
- e) nós e arestas.

4. (FGV/AFFC (STN)/STN/Tecnologia da Informação/Transformação Digital/2024) Com relação aos modelos de dados em aplicações de bancos de dados, a prática adotada em muitos casos é um modelo progressivo, baseado em níveis de abstração.

Assinale esses níveis, na ordem em que são usualmente empregados nessa estratégia.

- a) Modelo conceitual – Modelo funcional – Modelo físico.
- b) Modelo conceitual – Modelo lógico – Modelo físico.
- c) Modelo funcional – Modelo conceitual – Modelo lógico.
- d) Modelo funcional – Modelo lógico – Modelo físico.
- e) Modelo lógico – Modelo físico – Modelo funcional.

5. (FGV/Res (TJ RJ)/TJ RJ/Tecnologia da Informação/2024) Com relação às três fases de um projeto de um novo banco de dados, avalie se as afirmativas a seguir são verdadeiras (V) ou falsas (F).

() O modelo conceitual pode ter a forma de um diagrama entidade-relacionamentos e captura as necessidades de uma organização em termos de armazenamento de dados independentemente da sua implementação.

() O projeto lógico tem como objetivo transformar o modelo conceitual obtido na primeira fase em um modelo lógico que definirá como o banco de dados será implementado em um SGBD.

() Na etapa do projeto físico, o modelo de banco de dados é enriquecido com detalhes que influenciam no desempenho do banco mas interferem em suas funcionalidades.

As afirmativas são, respectivamente,

a) V – F – F.

b) V – V – F.

c) F – F – F.

d) F – V – V.

e) F – F – V.

6. (FGV/AFTRM (Pref Cuiabá)/Pref Cuiabá/Tecnologia da Informação/2024) Uma importante finalidade de um sistema gerenciador de banco de dados (SGBD) é fornecer aos usuários uma visão abstrata dos dados, classificada em níveis. Em relação ao tema, assinale a afirmativa correta.

a) O nível lógico de abstração descreve em detalhes estruturas complexas de como os dados são realmente armazenados nos dispositivos.

b) O nível lógico descreve o banco de dados inteiro em termos de um pequeno número de estruturas relativamente simples, que podem ser implementadas em estruturas complexas em nível físico, onde o usuário do nível lógico não precisa estar consciente desta complexidade.

c) O nível de abstração mais alto é o nível de visão (view). O SGBD pode fornecer várias visões para um mesmo banco de dados, definido de acordo com os mecanismos de busca dos dados.

d) Os administradores de banco de dados, que precisam decidir que informações armazenar no banco de dados, usam o nível físico de abstração.

e) Para ocultar detalhes do nível lógico do banco de dados, as visões fornecem um mecanismo de recuperação de modo a evitar que usuários acessem certas partes do banco de dados.

7. (FGV/Ana (CVM)/CVM/Ciência de Dados/2024) O modelo relacional representa o banco de dados como uma coleção de relações. Considere a relação COLABORADOR apresentada a

seguir, cuja chave primária é Matricula.

COLABORADOR

Nome	Matricula	Cpf	Celular	Endereco
Fernando Azul	1234	422.111.232-78	(21) 9888-9991	Rua das Flores, 8
Carla Vermelha	1357	444.222.333-77	(21) 9777-7474	Rua da Árvore, 4
Vivian Amarela	1458	147.854.111-12	NULL	Rua das Frutas, 2

Na relação COLABORADOR, o(a):

- a) superchave padrão é representada pelos atributos Matricula e Cpf;
- b) estado de relação atual reflete a cardinalidade dos domínios e o produto cartesiano;
- c) ordenação das tuplas faz parte da definição da relação sendo influenciada pela alteração da ordem das tuplas;
- d) domínio de cada atributo é um conjunto de valores atômicos, indivisível em se tratando de modelo relacional formal;
- e) restrição de integridade referencial garante que nenhum valor da chave primária será NULL.

8. (FGV/Aud CE (TCE,PA)/TCE PA/Informática/Administrador de Banco de Dados/2024) Gerenciar a integridade de um banco de dados é fundamental para garantir a consistência, a precisão e a confiabilidade dos dados.

Sobre os diferentes modelos de gerência de integridade de um banco de dados, é correto afirmar que

- a) o modelo de integridade semântica garante que os valores em uma coluna estejam dentro de um conjunto de valores permitidos.
- b) o modelo de integridade de domínio assegura que cada linha de uma tabela seja única e identificável.
- c) o modelo de integridade de entidade garante a integridade dos dados quando múltiplas transações são executadas simultaneamente.
- d) o modelo de integridade de transações garante que as operações de uma transação não sejam executadas parcialmente.

e) o modelo de integridade construtiva garante a consistência das relações entre tabelas.

9. (FGV/Aud CE (TCE,PA)/TCE PA/Administrativa/Ciência de Dados/2024) Considere o Sistema de Concursos de uma empresa hipotética que adota um Sistema de Gerenciamento de Banco de Dados (SGDB) relacional.

A primeira tabela construída no banco foi Tabela_Candidato, que possui os seguintes campos:

candidato_ID;

candidato_Nome;

candidato_CPF.

A segunda tabela construída foi a Tabela_Concurso, que possui os seguintes campos:

concurso_ID;

candidato_ID;

concurso_Nome;

concurso_Cargo.

No contexto dos bancos de dados relacionais, é correto afirmar que

a) o campo candidato_ID é uma chave estrangeira na Tabela_Candidato.

b) o campo candidato_ID é uma chave primária na Tabela_Concurso.

c) a Tabela_Candidato está incompleta, haja vista que não possui informação do concurso que o candidato pode fazer.

d) dá forma como o banco de dados foi concebido, só é possível lançar um concurso por candidato.

e) a inserção da chave estrangeira candidato_ID na tabela Tabela_Concurso, permite o lançamento de um candidato em vários concursos.

10. (FGV/AJ (TRF 1ª Região)/TRF 1/Apoio Especializado/Suporte em Tecnologia da Informação/2024) Pedro está implementando um banco de dados a partir de um projeto lógico que recebeu dos desenvolvedores de Sistemas de Informação do Tribunal. No projeto, há relacionamentos entre diversas tabelas.

No SQL, para implementar os relacionamentos de modo a prevenir que alguma transação de banco de dados destrua os relacionamentos entre as tabelas, Pedro deve utilizar a cláusula:

a) CHECK

b) UNION

- c) DEFAULT
- d) ORDER BY
- e) FOREIGN KEY

11. (FGV/APP GG (SEPLAG Niterói)/Pref Niterói/Gestão de Tecnologia da Informação e Comunicação/2024) As restrições de integridade são fundamentais para garantir a confiabilidade dos dados em um banco de dados. Elas definem regras que os dados devem obedecer, assegurando sua precisão e consistência.

Assinale a opção que indica a restrição de integridade que limita os valores que um atributo pode assumir.

- a) Restrições de Chave.
- b) Restrições de Valores.
- c) Restrições de Domínio.
- d) Restrições de Entidade.
- e) Restrições de Integridade Referencial.

12. (FGV/Ana Leg (ALETO)/ALETO/Suporte Técnico em Informática/2024) Na abordagem de sistemas de banco de dados, a arquitetura de três esquemas (ou arquitetura ANSI/SPARC) sustenta a abstração entre seus níveis.

Nesse contexto, o conceito que define a capacidade de alteração do esquema conceitual da arquitetura de três esquemas, sem repercussão nos esquemas externos ou nos programas de aplicação é definido como independência de dados

- a) conceitual.
- b) de usuário.
- c) física.
- d) lógica.
- e) visual.

13. (FGV/Ana Leg (ALETO)/ALETO/Desenvolvimento de Sistema/2024) Visões em SQL possuem um papel relevante em bancos de dados relacionais. Ela oferece mecanismos importantes, tais como abstração de dados, segurança e simplificação das consultas.

Nesse contexto, é correto afirmar que

- a) a cláusula WITH CHECK OPTION da SQL é demandada no final da especificação da visão, a fim de que a tornar uma asserção.
- b) a estratégia de modificação de consultas para a implementação de visões é eficaz para

consultas complexas.

c) a responsabilidade pela atualização do estado de uma visão é do usuário, que necessita enviar um comando refresh a cada execução.

d) uma visão em linha (view em linha) consiste na definição de uma visão na cláusula HAVING de uma consulta SQL.

e) visões materializadas criam uma tabela temporária, utilizando técnicas como a atualização incremental.

14. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Na fase de projeto de um banco de dados relacional, é comum que seja construído um modelo E-R, pois isso permite uma compreensão lógica das entidades envolvidas e suas respectivas propriedades. Uma das vantagens desses modelos é a identificação imediata da lista de:

a) chaves estrangeiras a serem estabelecidas;

b) índices auxiliares para as eventuais buscas;

c) permissões de acesso a serem concedidas;

d) triggers a serem implementados;

e) tipos de dados adequados a cada um dos atributos.

15. (FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/Banco de Dados/2023) O administrador de banco de dados do TJSE deverá projetar um banco de dados relacional para armazenar dados, oriundos do novo sistema de gestão de processos. O banco de dados armazenará dados sobre processos, autor, advogado, ocorrência, tipo da ação, município, situação, dentre outros. Para atender esses requisitos, o administrador deverá criar um banco de dados:

a) com uma única tabela para armazenar todos os dados;

b) separado para cada tabela, distribuindo os dados entre os bancos de dados;

c) com um esquema lógico das tabelas relacionadas, sem definir as chaves estrangeiras;

d) com todas as tabelas desnormalizadas para reduzir a redundância de dados;

e) com um esquema lógico das tabelas relacionadas, mantendo a integridade referencial.

16. (FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/Web designer/2023) Julia é uma WebDesigner e recebeu um modelo de dados com a seguinte especificação de um campo a ser armazenado no banco de dados DB_ProcessJud do sistema SIS_ProcessJud. NUM_PROCESSO, INTEGER, NOT NULL, PRIMARY KEY A partir do modelo de dados recebido, Julia pode concluir que NUM_PROCESSO é um campo:

a) inteiro de acesso reservado;

- b) do tipo caractere com tamanho fixo;
- c) utilizado como chave de acesso ao sistema SIS_ProcessJud;
- d) que permite o armazenamento de valores nulos em determinadas circunstâncias;
- e) identificador único de um registro em uma tabela do banco de dados DB_ProcessJud.

17. (FGV/FR (Pref RJ)/Pref RJ/2023) Em bancos de dados relacionais, os relacionamentos entre entidades identificadas na modelagem de dados são explicitados no SQL por meio de:

- a) Cláusulas Check;
- b) Cláusulas Unique;
- c) Foreign keys;
- d) Primary keys;
- e) Views.

18. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Sobre a utilização de views, em bancos de dados como MySQL, SQL Server, analise as afirmativas a seguir.

I. Views servem para construir e armazenar o conteúdo de consultas complexas de uso frequente.

II. Os principais SGBD não permitem o uso de operações de update/delete sobre views.

III. Em geral, as views não podem ser referenciadas na cláusula FROM em outras consultas SQL.

Está correto somente o que se afirma em:

- a) I;
- b) II;
- c) III;
- d) I e II;
- e) II e III.

19. (FGV/Ag TE (SEFAZ BA)/SEFAZ BA/Tecnologia da Informação/2022) Considerando as restrições do modelo de dados relacionais e os esquemas de bancos de dados relacionais, aquelas que não podem ser expressas diretamente nos modelos de dados e, portanto, devem ser expressas e impostas pelos programas de aplicação, denominam-se restrições

- a) inerentes.
- b) explícitas.

- c) de atributo.
- d) semânticas;
- e) de domínio.

20. (FGV/Tec Leg (CM Taubaté)/CM Taubaté/Informática/2022) No contexto dos bancos de dados relacionais, analise a declaração de uma chave estrangeira na criação de uma tabela T2, exibida a seguir.

CONSTRAINT FK_TESTE FOREIGN KEY (A) REFERENCES T1(A)

Considere ainda as seguintes afirmativas a respeito desse comando e suas consequências:

- I. A coluna A na tabela T1 deve ser definida como chave primária ou equivalente.
- II. A coluna A da tabela T2 não pode conter valores nulos.
- III. A coluna A da tabela T2 não pode ser declarada como chave primária ou equivalente.

Está correto somente o que se afirma em

- a) I.
- b) II.
- c) III.
- d) I e II.
- e) II e III.

21. (FGV/Ana (MPE SC)/MPE SC/Dados e Pesquisas/2022) Num banco de dados relacional, considere as tabelas T1 e T2, criadas como descrito a seguir.

T1 tem duas colunas, intituladas A e B, do tipo inteiro; a coluna A é declarada como primary key, e não aceita valores nulos.

T2 tem duas colunas, intituladas C e A, do tipo inteiro; a coluna C é declarada como primary key, e não aceita valores nulos; a coluna A foi declarada como UNIQUE, não aceita valores nulos e ainda foi declarada como uma foreign key que referencia a coluna A da tabela T1.

À luz dessa estrutura, é correto afirmar que o relacionamento entre T1 e T2:

- a) é do tipo 1:1;
- b) é do tipo 1:N;
- c) é do tipo N:1;
- d) é do tipo M:N;

e) não pode ser categorizado, pois isso depende das instâncias de cada tabela.

22. (FGV/Ag TE (SEFAZ BA)/SEFAZ BA/Tecnologia da Informação/2022) Com relação aos conceitos de banco de dados relacionais, analise as afirmativas a seguir.

I. Instância do banco se refere à supressão de detalhes da organização e do armazenamento de dados, descartando para um melhor conhecimento desses dados os recursos essenciais.

II. Modelo de dados se refere a uma coleção de conceitos que podem ser utilizados para descrever a estrutura de um banco de dados, oferecendo os meios necessários para alcançar essa abstração.

III. Abstração de dados refere aos conjuntos de dados e metadados e usuários presentes no servidor de dados em um determinado instante.

Está correto o que se afirma em

- a) I, somente.
- b) II, somente.
- c) III, somente.
- d) I e II, somente.
- e) I e III, somente.

23. (FGV/Ana (FunSaúde CE)/FunSaúde CE/TI e Infraestrutura de Informática/2021) Considere a definição e as instâncias das tabelas de bancos de dados CLUBE e JOGO exibidas a seguir.

nome
Barcelona
Boca Juniors
The Strongest

JOGO

mandante	visitante	golsM	golsV
Barcelona	Boca Juniors	1	0
Barcelona	The Strongest	NULL	NULL
Boca Juniors	Barcelona	0	0
Boca Juniors	The Strongest	3	0
The Strongest	Barcelona	2	0
The Strongest	Boca Juniors	2	0

Cada clube deve jogar quatro vezes, duas como mandante e duas como visitante.

As colunas golsM e golsV registram o número de gols dos times mandantes e visitantes, respectivamente, em cada jogo. Ambas são nulas enquanto o jogo não for realizado.

Em relação às instâncias das tabelas CLUBE e JOGO, definidas anteriormente, considere as constraints a seguir.

- I. constraint pk1 primary key (mandante, visitante)
- II. constraint fk1 foreign key (mandante) references clube
- III. constraint fk2 foreign key (visitante) references clube

Dessas constraints, podem ser aplicadas à tabela JOGO:

- a) I, apenas.
- b) I e II, somente.
- c) I e III, somente.
- d) II e III, somente.
- e) I, II e III.

24. (FGV/Ana TI (BANESTES)/BANESTES/Desenvolvimento de Sistemas/2021) Na questão, considere as tabelas T1, T2 e T3, cujas estruturas e instâncias são exibidas a seguir. O valor NULL deve ser tratado como unknown (desconhecido).

Suponha que as tabelas T1 e T2, descritas anteriormente, tenham sido declaradas com as colunas A e C, respectivamente, definidas como chaves primárias.

Para tanto, a definição de chaves estrangeiras na tabela T3 deveria ter como base o script:

- a) foreign key (A) references T1(A),
foreign key (C) references T2(C)
- b) foreign key (A, C) references T1(A), T2(C)
- c) foreign key T1(A) for A,
foreign key T2(C) for C
- d) foreign key T1(A) for A, T2(C) for C
- e) foreign key T1(A) references (A),
foreign key T2(C) references (C)

25. (FGV/Sup (IMBEL)/IMBEL/Tecnologia da Informação/2021) Na criação de esquemas de bancos de dados relacionais a partir de modelos/diagramas de entidades e relacionamentos, a implementação física de relacionamentos M:N é usualmente feita por meio de

- a) chaves primárias.
- b) índices não únicos.
- c) restrições de validação.
- d) tabelas.
- e) triggers.

26. (FGV/SAD PE/Controle Interno/2009) Modelagem de Dados é uma parte importante do desenho de um sistema de informação, que especifica as estruturas de dados e as regras necessárias para suportar uma área de negócios e que representa um conjunto de requerimentos de informações desse negócio.

A respeito dos modelos existentes, analise as afirmativas a seguir:

I. trata de regras de normalização das estruturas de dados, derivação de relacionamentos e de estruturas de agregação e generalização-especialização, restrição de domínio, de Integridade e de Implementação;

II. trata da análise das características e dos recursos necessários para armazenamento real e manipulação das estruturas de dados.

Os modelos I e II são conhecidos, respectivamente, por:

- a) conceitual e hierárquico.
- b) relacional e conceitual.

- c) hierárquico e lógico.
- d) físico e relacional.
- e) lógico e físico.

GABARITO

1. C	10. E	19. D
2. D	11. C	20. A
3. A	12. D	21. A
4. B	13. E	22. B
5. B	14. A	23. E
6. B	15. E	24. A
7. D	16. E	25. D
8. D	17. C	26. E
9. E	18. A	

IDEF1X

O **IDEF1X** é uma **linguagem visual utilizada para representar informações**, sendo que, no contexto de banco de dados, a utilizamos para desenhar modelos lógicos de dados.



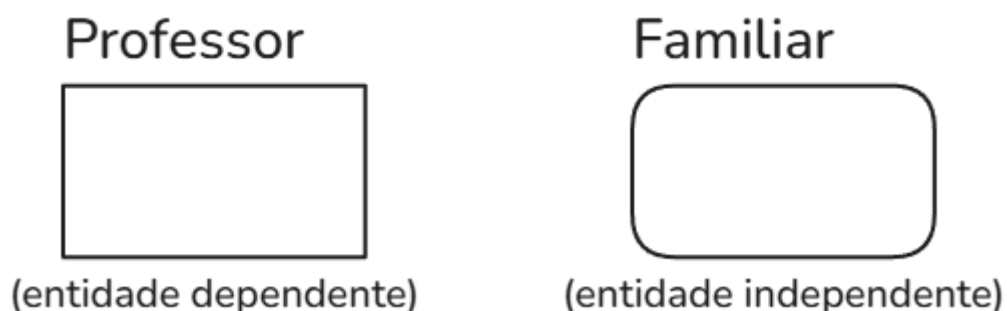
A sigla IDEF1X significa Integration DEFinition for information modeling, em tradução livre ao português seria algo como "Definição de Integração para Modelagem de Informações".

Assim sendo, com o IDEF1X temos uma notação gráfica capaz de representar modelos relacionais, organizando suas informações de forma padronizada. Para ficar mais claro, a ideia aqui é a mesma dos modelos entidade relacionamento, só que, enquanto nesses utilizamos o Diagrama Entidade Relacionamento (DER) para os representar graficamente, utilizamos a IDEF1X para representar modelos de dados lógicos relacionais.

Vamos iniciar nosso estudo explorando o conceito de "Entidades", que são as "coisas" do mundo (real ou abstrato) que queremos representar no banco. Podem ser pessoas Professor, Disciplina, Aluno etc.

Graficamente o IDEF1X representa entidades de duas formas, a depender se a **entidade** é independente ou dependente (fraca). Se for **independente**, temos **retângulos com pontas retas**, se for **dependente**, os retângulos ganham **pontas arredondadas**. Em ambos casos, o nome da entidade fica no topo deles.

Por exemplo, a seguir vamos supor uma entidade independente "Professor" e uma entidade dependente "Familiar".



Os **atributos** são as características de uma entidade. Por exemplo, uma entidade "Professor" (e a "Familiar" também) pode ter os atributos "CPF", "RG" e "nome", enquanto uma "Disciplina"

pode ter "id" e "matéria". Para os representarmos, **basta inserirmos seus nomes dentro dos retângulos**.

Professor

CPF
RG
nome

Familiar

CPF
RG
nome

Disciplina

id
matéria

Chaves são quaisquer atributos que identificam um registro de forma única. Ao escolhermos uma de fato, obtemos a chave primária. No IDEF1X, **identificamos uma chave primária o separando dos demais por uma linha horizontal**. Além disso, também podemos associar a expressão "(PK)", de primary key, ao atributo.

Dentre as chaves possíveis de Professor (RG e CPF), vamos transformar o atributo "CPF" em chave primária.

Professor

CPF
RG
nome

Familiar

CPF
RG
nome

Disciplina

id
matéria

Já a chave estrangeira é quando a chave primária de uma entidade "migra" para outra entidade, criando uma ligação entre elas. Por exemplo, podemos migrar a chave primária de "Professor" (CPF) para aparecer na tabela "Disciplina" como chave estrangeira, indicando qual professor leciona determinada disciplina. Essa "migração" da chave primária para chave estrangeira é justamente o que cria o relacionamento entre as tabelas.

Além disso, podemos ter **chaves alternativas** (únicas), isto é, as chaves que não viraram primárias, sendo que as **representamos por "(AK)"** (alternate key).

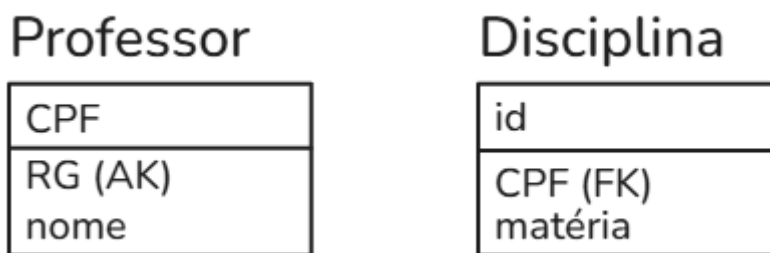
Professor

CPF
RG (AK)
nome

Disciplina

id
CPF
matéria

No diagrama, indicamos que há uma **chave estrangeira** na entidade associada “(FK)” (foreign key) **ao atributo**.

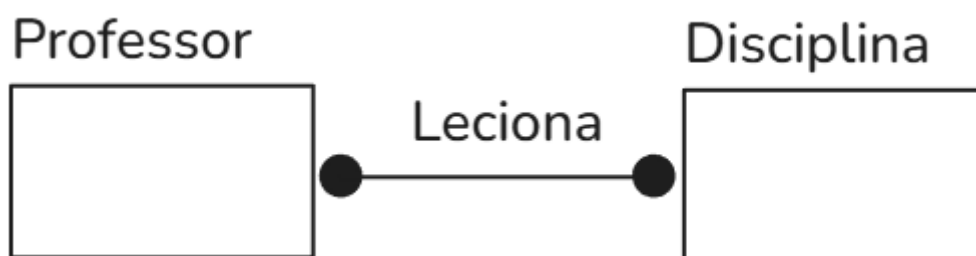


Então, podemos ter os relacionamentos, que exibem as associações entre instâncias das entidades. Passemos a estudá-los.

Um **relacionamento não-específico** ocorre quando, dos dois lados, existem várias ocorrências associadas às entidades, configurando-se no “**muitos para muitos**” (N:M). Aqui não é possível apontar “uma ocorrência específica” pois são várias de ambos os lados.

Por exemplo, isso ocorre se nossa regra for “professores podem lecionar várias disciplinas e disciplinas podem ser lecionadas por vários professores”, já que não dá para especificar uma ocorrência específica de nenhum dos lados.

Representamos esse tipo de relacionamento como uma linha com dois círculos nas pontas.



Consequentemente, um **relacionamento específico** é quando, **pelo menos de um dos lados, conseguimos apontar uma ocorrência específica associada**. Assim, aqui abarcamos as seguintes razões de relacionamento:

- **Um para um (1:1)**: uma ocorrência de cada lado
- **Um para muitos (1:N)**: de um lado é específico (o “1” do lado esquerdo)

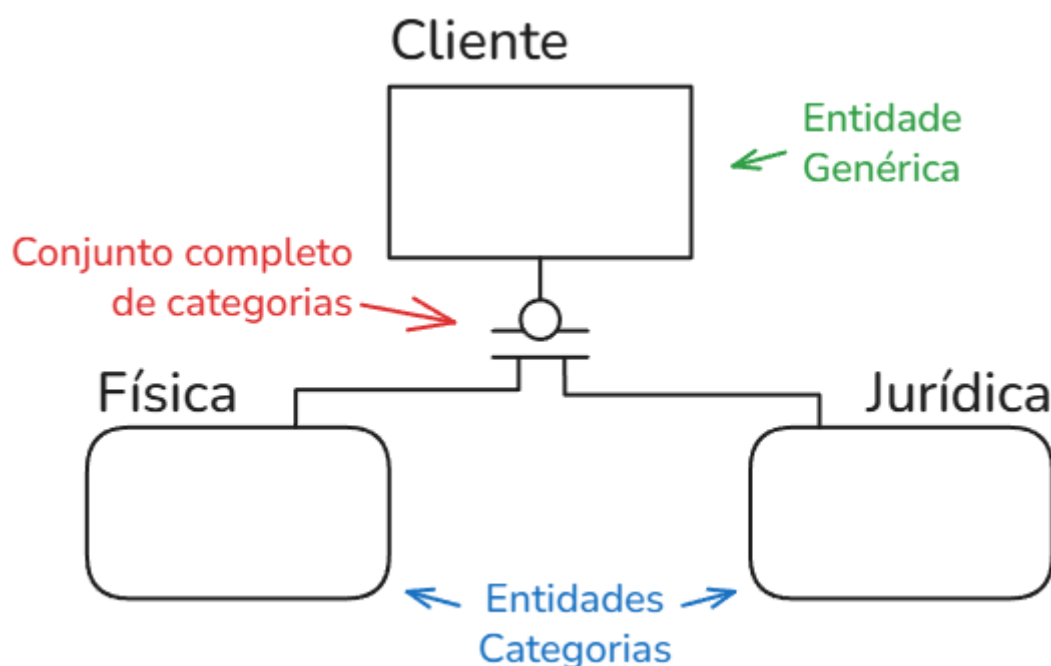
Por exemplo, supondo a regra “cada departamento é coordenado por um e apenas um professor”, temos um relacionamento específico.

Saibam que relacionamentos específicos se dividem em dois subtipos, os de categorização e conexão.

Ao utilizar **relacionamentos específicos de categorização** estabelecemos uma **herança (ou generalização) entre entidades**. Assim, temos uma entidade genérica, que concentra atributos comuns, e entidades categorias, que concentram os atributos específicos de cada especialização.

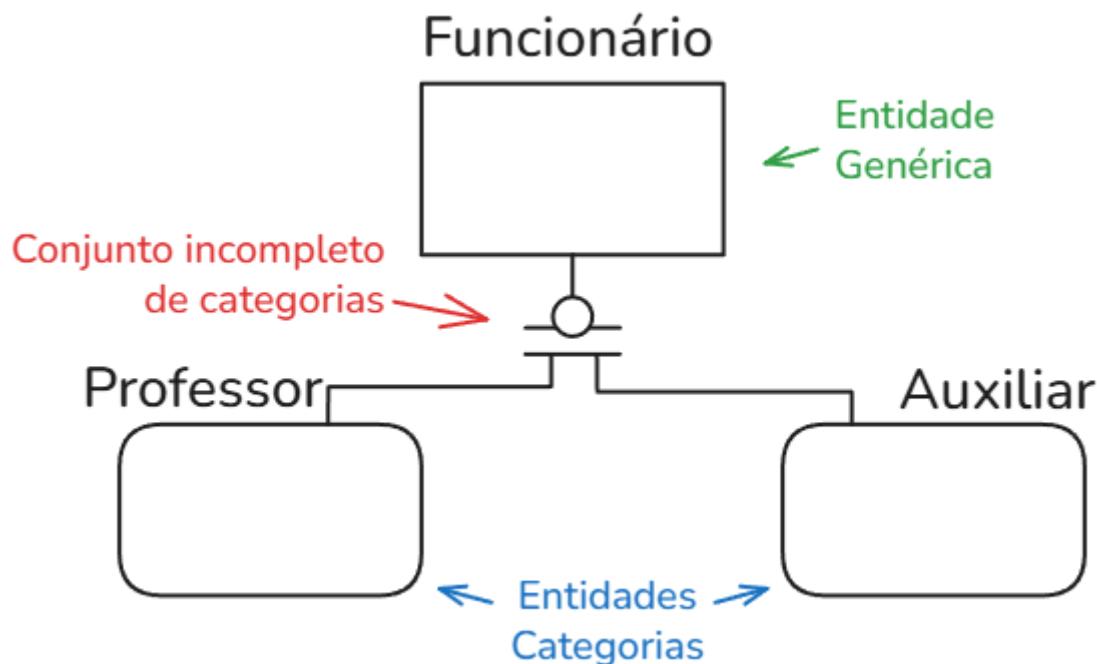
Então, aqui podemos ter grupos de entidades categorias como **grupos completos**, em que **toda ocorrência da entidade genérica obrigatoriamente pertence a uma das categorias** (Não existe ocorrência "sem categoria"), situação em que representamos o relacionamento por linha dupla sob uma circunferência.

Por exemplo, "todo Cliente deve ser Pessoa Física ou Pessoa Jurídica", não existindo Cliente sem classificação.



Além disso, também podemos ter entidades categorias como **grupo incompleto**, em que **uma ocorrência da entidade genérica pode ou não estar associada a uma categoria específica**. Nessa situação, representamos o relacionamento por linha simples sob a circunferência.

Por exemplo, temos funcionários que podem ser "Professor" ou "Auxiliar de Professor", mas também podem existir funcionários que estão fora dessas duas categorias (sendo apenas funcionários mesmo).

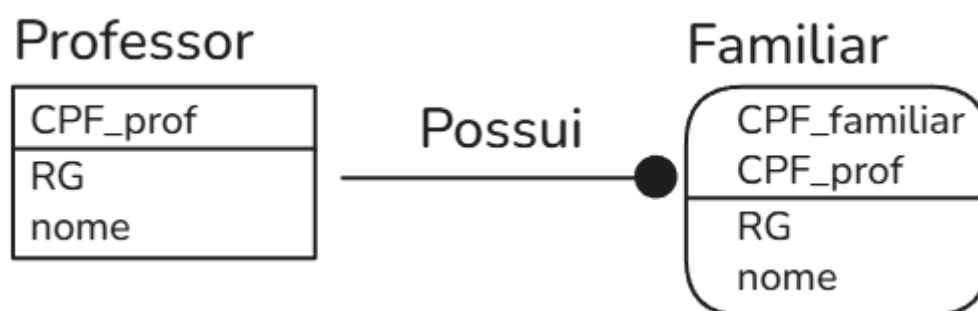


Também podemos ter **relacionamentos específicos de conexão**, o caso “clássico”, em que há uma associação entre entidades, chamadas de genitora ou filha. Porém, aqui apenas com a cardinalidade 1:N (já que as N:M foram tratadas como “relacionamento não-específico”), onde:

- **Entidade Genitora - lado do "1"**: há no máximo 1 genitora por filha
- **Entidade Filha - lado do "N"**: pode haver várias filhas para uma genitora

Nos **relacionamentos de conexão específica (ou relacionamento identificador)**, a **chave primária da entidade genitora compõe a chave primária da entidade filha**, ou seja, a entidade filha depende da genitora para ser identificada (é entidade fraca).

Por exemplo, os registros de pessoas em “Familiar”, que dependem da estarem associados a um “Professor” para existirem. No diagrama, identificamos o lado “1” excluindo o círculo do segmento de reta, e o “N” o mantendo.

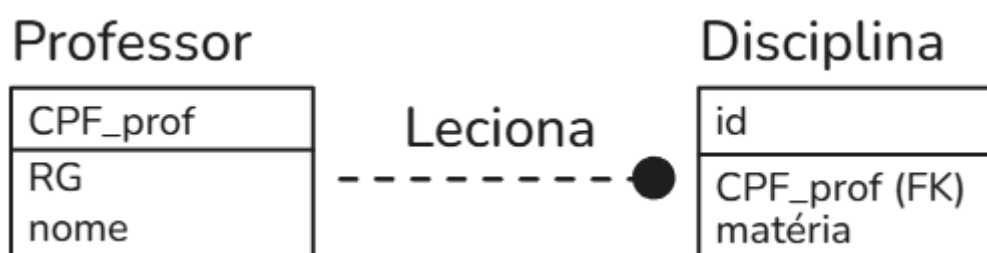


Já no **relacionamento não-identificador** ocorre quando a **chave da genitora não faz parte da chave da filha, ou seja, migra como uma chave estrangeira “comum”**. Nessa situação, o relacionamento é representado por linha pontilhada.

Como não poderia deixar de ser, temos subdivisões.

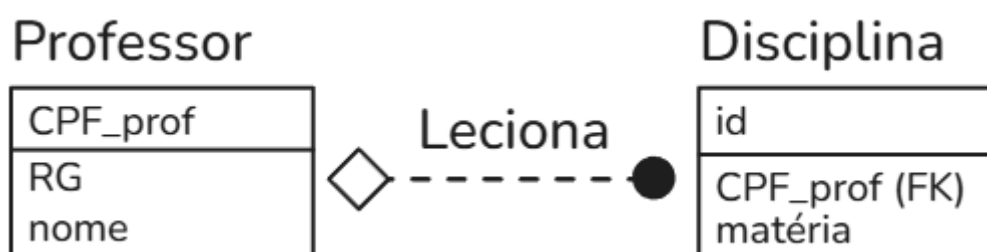
No **relacionamento não-identificador obrigatório**, **toda ocorrência da entidade filha deve necessariamente estar associada a uma genitora** (cardinalidade mínima 1). Não existindo, assim, filha "sem pai".

Por exemplo, “toda disciplina é lecionada por, necessariamente, um professor”.



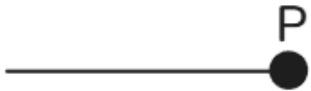
Enquanto no **relacionamento não-identificador opcional**, **uma ocorrência da entidade filha pode ou não estar associada a uma genitora** (cardinalidade mínima 0). Aqui utilizamos uma notação pouco usual: desenhamos um pequeno losango junto à entidade genitora.

Por exemplo, com a regra “uma disciplina pode não ser lecionada por nenhum professor” (situação que um Auxiliar assume).



Pra finalizar, saibam que nos relacionamentos podemos representar a exata cardinalidade entre as entidades, ou seja, podemos definir quantas ocorrências de uma entidade podem estar exatamente associadas a uma ocorrência da outra. Para isso, basta adicionar uma informação ao círculo. Vejamos uma tabela para explicar o tema.

Cardinalidade	Notação	Descrição
Zero, uma ou várias		Trata-se da notação padrão, não é necessário adicionar nenhuma informação extra ao círculo

Cardinalidade	Notação	Descrição
Uma ou várias		Adicionamos um "P" de números positivos (isto é, maiores que 1).
Zero ou uma		Adicionamos um "Z" (zero), indicando que podemos estar associados a no mínimo 0 ocorrência (consequentemente, no máximo 1).
Exatamente N		Nossa ocorrência está associada a exatamente "N" ocorrências da outra entidade.
N até M		Nossa ocorrência está associada de "N" a "M" ocorrências da outra entidade.
Nota N		Há uma nota explicativa em algum local do diagrama IDEF1X explicando essa cardinalidade.

QUESTÕES COMENTADAS



1. (Instituto Verbena/TJ AC/Analista de Banco de Dados/2024) A modelagem da informação trata da concepção da estrutura da informação em determinado universo de discurso, sendo uma etapa fundamental no projeto de bancos de dados. IDEF1X é uma linguagem para modelagem da informação, baseada no modelo entidade-relacionamento que tem como construções sintáticas básicas a entidade, o relacionamento e o atributo. A entidade dependente é representada, na notação IDEF1X, por:

- a) um retângulo com os cantos em 90 graus.
- b) uma elipse rotulada com um verbo ou uma frase verbal.
- c) uma elipse com linha pontilhada.
- d) um retângulo com os cantos arredondados.

Comentários:

- a) **Errada.** O retângulo com cantos em 90 graus (pontas retas) representa a entidade independente, não a dependente.
- b) **Errada.** Elipse com verbo não é uma construção do IDEF1X descrita para representar entidades.
- c) **Errada.** Elipse com linha pontilhada também não corresponde a nenhuma representação de entidade no IDEF1X.
- d) **Correta.** A entidade dependente (fraca) é representada por um retângulo com os cantos arredondados, enquanto a entidade independente usa cantos retos.

Gabarito: D

2. (CEBRASPE (CESPE)/AIS (EMPREL)/EMPREL/Banco de Dados/2023) Na notação IDEF1X, uma entidade é dependente sempre que sua chave (também conhecida como chave primária) contém a chave completa de pelo menos outra entidade. Nesse contexto, a entidade dependente é representada por

- a) retângulos com os cantos vivos (90 graus).

- b) retângulos com os cantos arredondados.
- c) uma linha pontilhada.
- d) linhas rotuladas com um verbo ou uma frase verbal.
- e) uma linha dupla.

Comentários:

a) **Errada.** Retângulos com cantos vivos (90 graus) representam entidades independentes, não dependentes.

b) **Correta.** Entidades dependentes (fracas) são representadas por retângulos com cantos arredondados no IDEF1X.

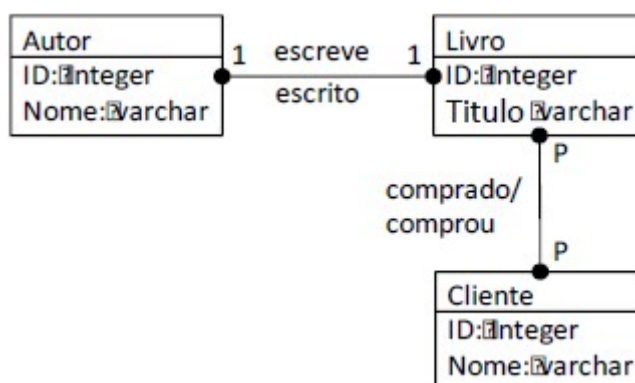
c) **Errada.** A linha pontilhada representa um tipo de relacionamento não-identificador, não uma entidade dependente.

d) **Errada.** Isso não é uma forma de representar entidades no IDEF1X conforme o conteúdo estudado.

e) **Errada.** A linha dupla aparece nos relacionamentos de categorização com grupo completo, não para representar entidades dependentes.

Gabarito: B

3. (FGV/AJ (TJ RN)/TJ RN/Apoio Especializado/Análise de Suporte/2023) Observe o seguinte Modelo Entidade Relacionamento a seguir.



O modelo ilustrado atende a regra de negócio:

- a) um autor pode escrever diversos livros;
- b) cada livro pode ser escrito por diversos autores;
- c) para se tornar cliente, é necessário pelo menos uma compra;
- d) cliente e autor são especializações da entidade pessoa;

e) os atributos ID podem armazenar letras, números e símbolos.

Comentários:

a) Errada. Para que um autor pudesse escrever vários livros, o relacionamento entre Autor e Livro teria que indicar cardinalidade de um para muitos no lado de Autor. O modelo não representa isso isoladamente como a regra central.

b) Errada. Para que cada livro pudesse ser escrito por vários autores, teríamos um relacionamento do tipo muitos para muitos (não-específico), representado por linha com dois círculos nas pontas. Isso não é o que o modelo mostra.

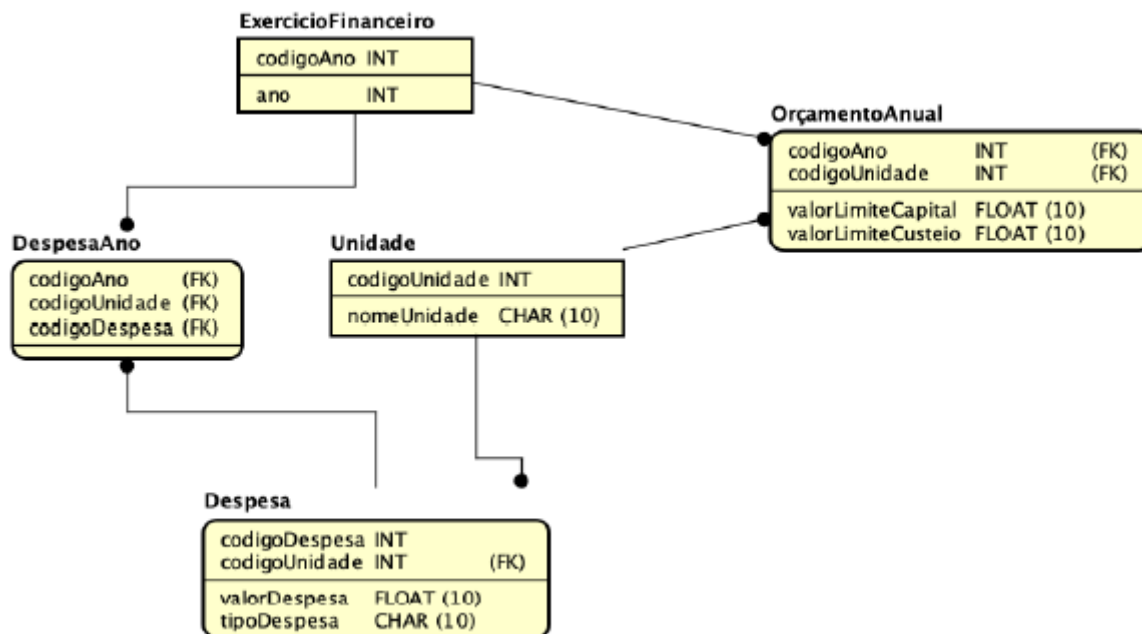
c) Correta. O modelo mostra um relacionamento não-identificador mandatório entre Compra e Cliente, o que significa que toda ocorrência da entidade filha deve obrigatoriamente estar associada a uma genitora. Aqui, isso se lê ao contrário: um Cliente só existe se tiver ao menos uma Compra associada, atendendo exatamente a regra de que é necessária pelo menos uma compra para se tornar cliente.

d) Errada. Para que Cliente e Autor fossem especializações de Pessoa, o modelo teria que usar um relacionamento de categorização, com uma entidade genérica Pessoa e entidades categoria Cliente e Autor, representado por uma circunferência no diagrama. Isso não é o que foi modelado.

e) Errada. O tipo de dado que um atributo pode armazenar não é definido pelo diagrama IDEF1X em si. O modelo apenas representa a estrutura das entidades e seus relacionamentos, não o formato dos valores armazenados.

Gabarito: C

4. (CEPS UFPA/UFPA/Governança TI/2022) Considere o diagrama Entidade Relacionamento a seguir (notação IDEF1X), representando um modelo parcial de informações sobre o registro de despesas em unidades acadêmicas.



Com base no diagrama fornecido, analise as afirmativas a seguir.

- I. O relacionamento entre a entidade 'Despesa' e 'DespesaAno' pode ser suprimido deste diagrama sem alterar o significado do modelo.
- II. O atributo codigoUnidade (FK) em DespesaAno pode ser suprimido deste diagrama por representar uma informação redundante com os relacionamentos estabelecidos.
- III. Os atributos valorLimiteCapital e valorLimiteCusteio podem ser suprimidos deste diagrama sem alterar o significado do modelo.

Está(ão) correta(s)

- a) I, somente.
- b) II, somente.
- c) III, somente.
- d) I e II, somente.
- e) II e III, somente.

Comentários:

- I. Errada.** O relacionamento entre Despesa e DespesaAno não pode ser suprimido, pois ele representa uma associação real entre as entidades e é justamente ele que cria a ligação por meio da migração de chave.
- II. Correta.** Quando uma chave estrangeira (FK) já está representada pelo relacionamento no diagrama, manter o atributo codigoUnidade (FK) explicitamente em DespesaAno é redundante.

O próprio relacionamento já indica essa ligação, então o atributo pode ser removido sem perder informação.

III. Errada. Os atributos valorLimiteCapital e valorLimiteCusteio são características próprias de uma entidade, e removê-los alteraria o significado do modelo, pois essas informações deixariam de existir no diagrama.

Gabarito: B

5. (FGV/ALERO/Tecnologia da Informação/2018) Na representação de esquemas para bancos de dados relacionais por meio da notação IDEF1X, os relacionamentos podem ser identificadores ou não identificadores.

Sobre a consequência do uso de relacionamentos identificadores, assinale a afirmativa correta.

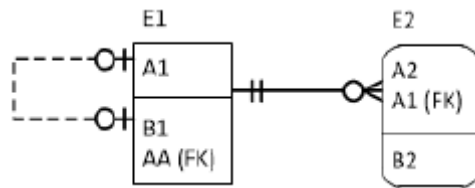
- a) Uma tabela não pode participar em mais de um relacionamento identificador.
- b) A cardinalidade de um relacionamento identificador deve ser 1:1.
- c) A tabela do lado N do relacionamento deve possuir uma chave primária que independa da chave estrangeira decorrente do relacionamento.
- d) A chave estrangeira decorrente do relacionamento deve fazer parte da chave primária da tabela.
- e) A chave estrangeira decorrente do relacionamento deve permitir a preenchimento com valores nulos.

Comentários:

- a) Errada.** O IDEF1X não impõe esse limite. Uma entidade filha pode participar de mais de um relacionamento identificador.
- b) Errada.** O relacionamento identificador (de conexão específica) tem cardinalidade 1:N, não 1:1.
- c) Errada.** É exatamente o contrário. No relacionamento identificador, a chave primária da entidade filha depende sim da chave estrangeira vinda da genitora.
- d) Correta.** No relacionamento identificador, a chave primária da entidade genitora migra para a entidade filha e passa a compor a chave primária dela. Ou seja, a chave estrangeira vira parte da chave primária da tabela filha, fazendo com que a filha dependa da genitora para ser identificada.
- e) Errada.** Se a chave estrangeira faz parte da chave primária da tabela filha, ela não pode ser nula, pois chave primária não admite valores nulos.

Gabarito: D

6. (FGV/ALERO/Tecnologia da Informação/2018) Com relação ao diagrama IDEF1X, considere as afirmativas a seguir sobre um eventual esquema relacional, com tabelas E1 e E2, que implemente aquele diagrama.



- I. A coluna A1, de E1, deve permitir valores nulos.
- II. A coluna AA de E1 não pode conter repetições de valores não nulos.
- III. A coluna A1, de E2, deve permitir valores nulos.

Está correto o que se afirma em

- a) I, somente.
- b) II, somente.
- c) III, somente.
- d) I e II, somente.
- e) II e III, somente.

Comentários:

I. A coluna A1, de E1, deve permitir valores nulos.

Errada. A1 está acima da linha horizontal, portanto é chave primária e não aceita valores nulos.

II. A coluna AA de E1 não pode conter repetições de valores não nulos.

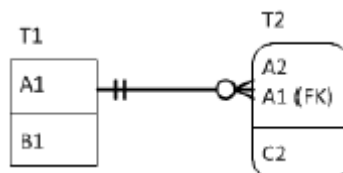
Correta. Chaves estrangeiras são únicas, portanto não aceitam valores repetidos.

III. A coluna A1, de E2, deve permitir valores nulos.

Errada. A1 está acima da linha horizontal com A2, portanto compõe a chave primária (além de ser chave estrangeira também), não aceitando valores nulos.

Gabarito: B

7. (FGV/2016) Analise o diagrama IDEF1X (pé de galinha) mostrado a seguir.



Sobre esse diagrama e sua implementação relacional correspondente, assinale a afirmativa correta.

- a) A1 em T2 pode ser nulo, independentemente de A2 em T2 ser nulo ou não.

- b) A1 em T2 pode ser nulo somente se A2 em T2 for nulo.
- c) O relacionamento entre T1 e T2 é "não identificador".
- d) Um registro qualquer de T2 pode não estar relacionado a algum registro de T1.
- e) Em T2, A2 e A1, concatenados, constituem um identificador para T2.

Comentários:

a) Errada. Se A1 migrou como parte da chave primária de T2, ela não pode ser nula, pois chave primária nunca aceita nulo.

b) Errada. A1 não pode ser nula em nenhuma situação em T2, pois faz parte da chave primária, independente de qualquer condição.

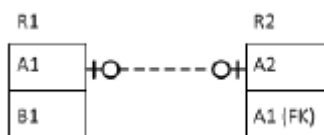
c) Errada. O relacionamento é identificador, não não-identificador. No relacionamento não-identificador, a chave da genitora migraria como chave estrangeira comum, sem compor a chave primária da filha.

d) Errada. No relacionamento identificador, a entidade filha depende da genitora para existir, então todo registro de T2 obrigatoriamente está relacionado a um registro de T1.

e) Correta. A chave primária de T1 (A1) migra para T2 e, concatenada com A2, forma a chave primária de T2, identificando cada registro de T2 de forma única.

Gabarito: E

8. (FGV/COMPESA/Administrador de Banco de Dados/2016) Analise o diagrama ER, exibido a seguir com a notação IDEF1X pé-de-galinha.



Analise, ainda, a lista de possíveis requisitos para uma implementação relacional desse modelo.

- I. A1 em R2 deve ter uma restrição do tipo UNIQUE.
- II. A1 em R2 não deve permitir valores nulos.
- III. A1 em R1 deve constituir a chave primária.
- IV. A2 em R2 deve permitir valores nulos.

Assinale a opção que indica a quantidade de requisitos corretamente estabelecidos.

- a) Zero.
- b) Um.
- c) Dois.

d) Três.

e) Quatro.

Comentários:

I. A1 em R2 deve ter uma restrição do tipo UNIQUE.

Errado. Deve ser uma chave primária, pois está acima da linha horizontal.

II. A1 em R2 não deve permitir valores nulos.

Errado. Chaves estrangeiras admitem valores nulos.

III. A1 em R1 deve constituir a chave primária.

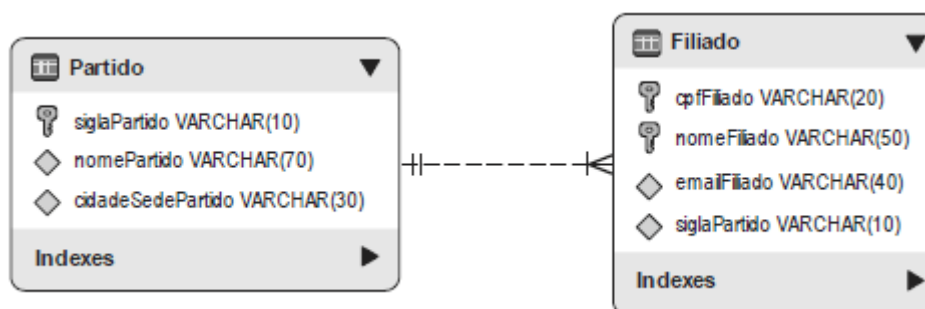
Certo. Atributos acima da linha horizontal são chaves primárias.

IV. A2 em R2 deve permitir valores nulos.

Certo. Atributos acima da linha horizontal são chaves primárias.

Gabarito: C

9. (FCC/ALMS/Informática/2016) Instrução: Para responder à questão, considere o modelo mostrado na imagem abaixo, oriundo de uma situação hipotética:



Após criadas as tabelas Partido e Filiado, foram incluídos, respectivamente, os seguintes registros:

sigla	Partido nomePartido	cidadeSedePartido
PDT	Partido Democrático Trabalhista	Brasília
PMDB	Partido do Movimento Democrático Brasileiro	Brasília
PSDB	Partido da Social Democracia Brasileira	São Paulo

cpfFiliado	nomeFiliado	emailFiliado	siglaPartido
------------	-------------	--------------	--------------

124.179.156-10	André Braga	braga@hotmail.com	PMDB
147.189.237-18	Marcos Pereira	mpereira@hotmail.com	PDT
154.496.172-14	Pedro Silva	pedro@gmail.com	PDT
192.345.176-01	Maria Souza	maria@ig.com.br	PSDB

Conclui-se, observando o modelo, que

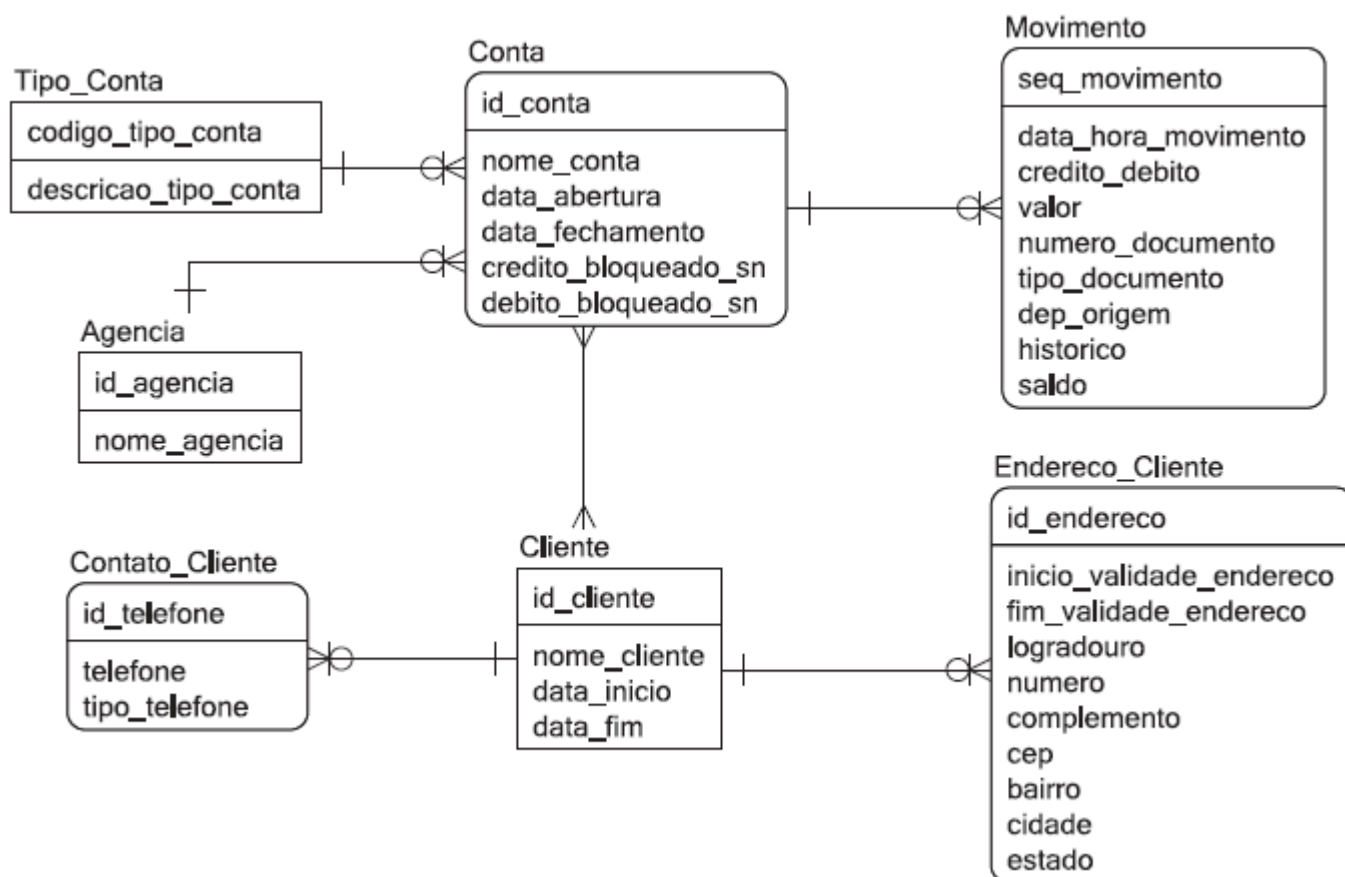
- a) será necessário cadastrar, no mínimo, dois partidos com pelo menos um filiado cada.
- b) está sendo utilizada a notação IDEF1X para relacionar as duas entidades.
- c) se trata de um relacionamento 1:n não identificado.
- d) siglaPartido deveria fazer parte da chave primária na entidade Filiado.
- e) todo partido cadastrado precisará ter, no mínimo, um filiado cadastrado.

Comentários:

- a) **Errada.** Não há nenhuma regra no modelo que exija cadastrar dois partidos com pelo menos um filiado cada. Um partido pode existir sem nenhum filiado associado.
- b) **Errada.** Esse tema vai além do conteúdo desta aula.
- c) **Correta.** O relacionamento entre Partido e Filiado é 1:N, pois um partido pode ter vários filiados, mas cada filiado pertence a apenas um partido. Nesse tipo de relacionamento, a chave primária do lado 1 vai para o lado N como chave estrangeira, que é exatamente o que acontece com siglaPartido na tabela Filiado. Como siglaPartido é apenas uma chave estrangeira e não faz parte da chave primária de Filiado, o relacionamento é chamado de não identificado.
- d) **Errada.** A siglaPartido na tabela Filiado é uma chave estrangeira, não parte da chave primária. No mapeamento 1:N, a chave estrangeira vai para o lado N apenas para registrar a associação, sem precisar compor a chave primária daquela tabela.
- e) **Errada.** Olhando os dados de exemplo, o partido PDT tem dois filiados e o PSDB tem um, mas o modelo não exige que todo partido tenha pelo menos um filiado. Um partido pode existir sem nenhum filiado cadastrado.

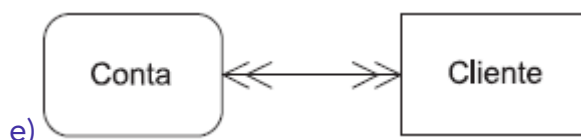
Gabarito: C

10. (CESGRANRIO/BASA/Tecnologia da Informação/2014) Para responder à questão, tenha como referência o diagrama de entidades e relacionamentos, apresentado abaixo, que representa parte do modelo de dados de uma instituição financeira.



Que representação gráfica do modelo ER proposta pela notação IDEF1X representa relacionamento existente entre Conta e Cliente?

- a)
- b)
- c)
- d)



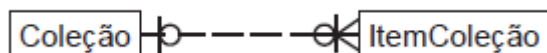
Comentários:

Entre conta e cliente, razão de cardinalidade "N:M" para as duas entidades, pois temos um pé de galinha de cada lado. Assim, devemos expressar em IDEF1X com dois círculos e uma linha os ligando.



Gabarito: B

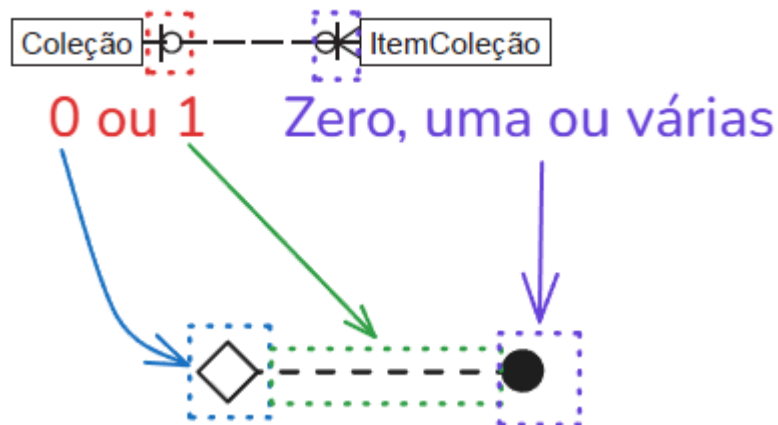
11. (CESGRANRIO/PETROBRAS/Informática/2014) O diagrama a seguir apresenta um modelo de entidades e relacionamentos segundo a notação da Engenharia de Informação.



A notação equivalente em IDEF1X é

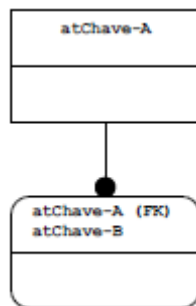
- a)
- b)
- c)
- d)
- e)

Comentários:



Gabarito: A

12. (NCE e FUJB (UFRJ)/2008) Na metodologia de modelagem IDEF1X, o relacionamento entre as entidades A e B, na figura a seguir, é do tipo:



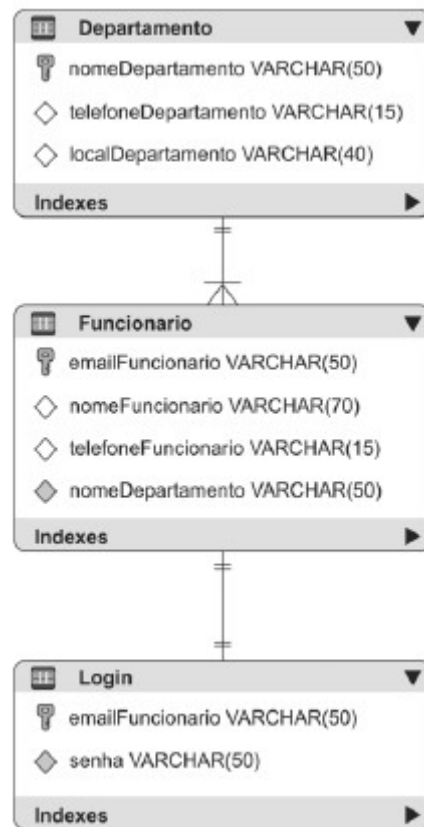
- a) A para B;
- b) migratório;
- c) ortogonal;
- d) pai para filho;
- e) supertipo para subtipo.

Comentários:

Se trata de um relacionamentos de conexão específica, ou seja, a entidade filha depende da genitora (pai) para ser identificada (é entidade fraca).

Gabarito: D

13. (FCC/AJ TRT5/TRT 5/Apoio Especializado/Tecnologia da Informação/2022) Atenção: O Modelo Entidade-Relacionamento abaixo deve ser utilizado para responder a questão.



Observando o Modelo Entidade-Relacionamento apresentado, é possível concluir corretamente que

- a) o atributo nomeDepartamento é chave estrangeira na entidade Departamento.
- b) a entidade Login não tem chave estrangeira nem primária.
- c) a cardinalidade do relacionamento entre as entidades Funcionario e Login é um-para-um.
- d) o atributo telefoneFuncionario é adequado para ser chave primária na entidade Funcionario.
- e) a chave primária da entidade Login é composta, pois emailFuncionario é chave primaria e chave estrangeira ao mesmo tempo.

Comentários:

- a) Errada.** Uma chave estrangeira serve para ligar duas tabelas diferentes. O nomeDepartamento dentro da própria tabela Departamento não faz esse papel, seria um atributo comum ou, no máximo, uma chave candidata da própria entidade.
- b) Errada.** Para uma entidade existir no modelo relacional, ela precisa ter pelo menos uma forma de identificar suas tuplas de forma única, ou seja, uma chave primária. Além disso, se Login se relaciona com Funcionario, é esperado que tenha uma chave estrangeira ligando as duas.

c) **Correta.** No relacionamento 1:1, cada tupla de um lado se associa a no máximo uma tupla do outro lado. Se cada funcionário possui no máximo um login e cada login pertence a no máximo um funcionário, a cardinalidade é um-para-um.

d) **Errada.** Para ser chave primária, um atributo precisa garantir que não haja valores duplicados. Um telefone não é uma boa escolha porque pode haver funcionários com o mesmo número, além de poder mudar ao longo do tempo, o que tornaria a chave instável.

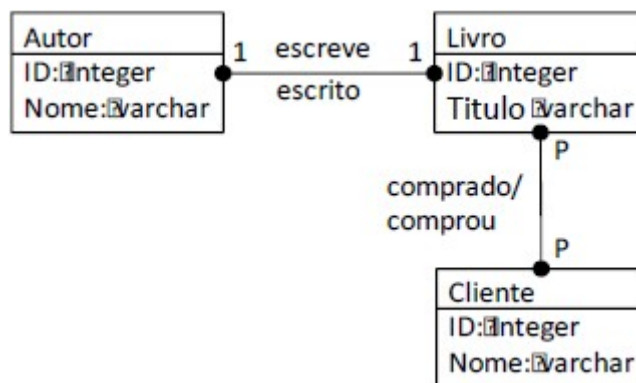
e) **Errada.** Uma chave primária composta é formada pela combinação de dois ou mais atributos. Se emailFuncionario sozinho já identifica de forma única cada tupla de Login, a chave primária não é composta, é simples. O fato de ser também chave estrangeira não a torna composta.

Gabarito: C

LISTA DE QUESTÕES - BANCAS DIVERSAS



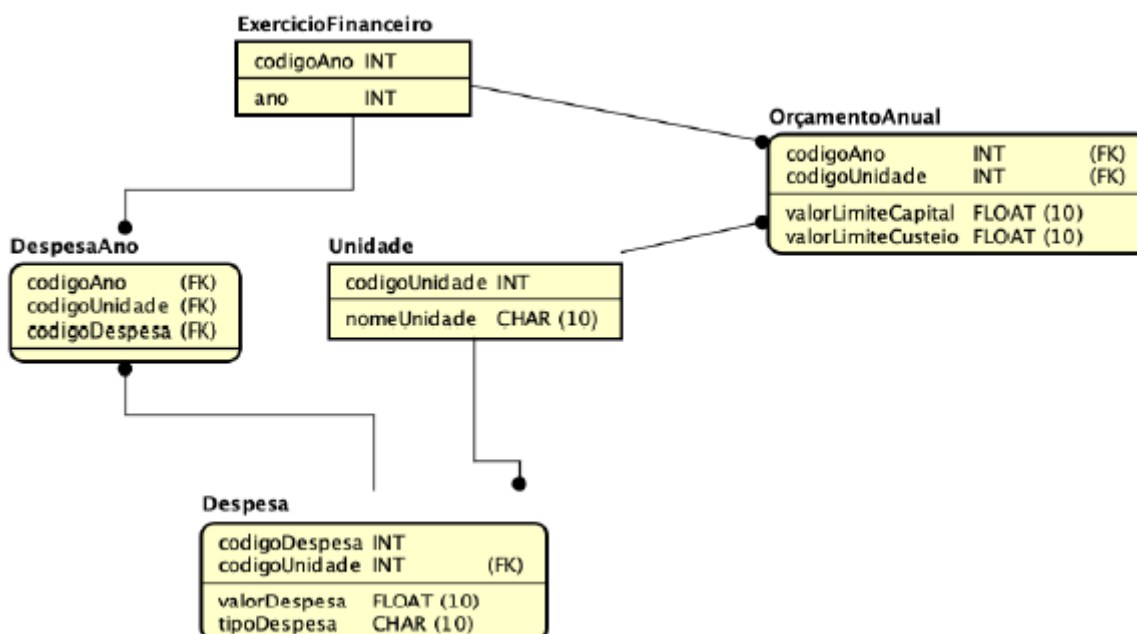
1. (Instituto Verbena/TJ AC/Analista de Banco de Dados/2024) A modelagem da informação trata da concepção da estrutura da informação em determinado universo de discurso, sendo uma etapa fundamental no projeto de bancos de dados. IDEF1X é uma linguagem para modelagem da informação, baseada no modelo entidade-relacionamento que tem como construções sintáticas básicas a entidade, o relacionamento e o atributo. A entidade dependente é representada, na notação IDEF1X, por:
 - a) um retângulo com os cantos em 90 graus.
 - b) uma elipse rotulada com um verbo ou uma frase verbal.
 - c) uma elipse com linha pontilhada.
 - d) um retângulo com os cantos arredondados.
2. (CEBRASPE (CESPE)/AIS (EMPREL)/EMPREL/Banco de Dados/2023) Na notação IDEF1X, uma entidade é dependente sempre que sua chave (também conhecida como chave primária) contém a chave completa de pelo menos outra entidade. Nesse contexto, a entidade dependente é representada por
 - a) retângulos com os cantos vivos (90 graus).
 - b) retângulos com os cantos arredondados.
 - c) uma linha pontilhada.
 - d) linhas rotuladas com um verbo ou uma frase verbal.
 - e) uma linha dupla.
3. (FGV/AJ (TJ RN)/TJ RN/Apoio Especializado/Análise de Suporte/2023) Observe o seguinte Modelo Entidade Relacionamento a seguir.



O modelo ilustrado atende a regra de negócio:

- a) um autor pode escrever diversos livros;
- b) cada livro pode ser escrito por diversos autores;
- c) para se tornar cliente, é necessário pelo menos uma compra;
- d) cliente e autor são especializações da entidade pessoa;
- e) os atributos ID podem armazenar letras, números e símbolos.

4. (CEPS UFPA/UFPA/Governança TI/2022) Considere o diagrama Entidade Relacionamento a seguir (notação IDEF1X), representando um modelo parcial de informações sobre o registro de despesas em unidades acadêmicas.



Com base no diagrama fornecido, analise as afirmativas a seguir.

I. O relacionamento entre a entidade 'Despesa' e 'DespesaAno' pode ser suprimido deste diagrama sem alterar o significado do modelo.

II. O atributo codigoUnidade (FK) em DespesaAno pode ser suprimido deste diagrama por representar uma informação redundante com os relacionamentos estabelecidos.

III. Os atributos valorLimiteCapital e valorLimiteCusteio podem ser suprimidos deste diagrama sem alterar o significado do modelo.

Está(ão) correta(s)

- a) I, somente.
- b) II, somente.
- c) III, somente.
- d) I e II, somente.
- e) II e III, somente.

5. (FGV/ALERO/Tecnologia da Informação/2018) Na representação de esquemas para bancos de dados relacionais por meio da notação IDEF1X, os relacionamentos podem ser identificadores ou não identificadores.

Sobre a consequência do uso de relacionamentos identificadores, assinale a afirmativa correta.

- a) Uma tabela não pode participar em mais de um relacionamento identificador.
- b) A cardinalidade de um relacionamento identificador deve ser 1:1.
- c) A tabela do lado N do relacionamento deve possuir uma chave primária que independa da chave estrangeira decorrente do relacionamento.
- d) A chave estrangeira decorrente do relacionamento deve fazer parte da chave primária da tabela.
- e) A chave estrangeira decorrente do relacionamento deve permitir a preenchimento com valores nulos.

6. (FGV/ALERO/Tecnologia da Informação/2018) Com relação ao diagrama IDEF1X, considere as afirmativas a seguir sobre um eventual esquema relacional, com tabelas E1 e E2, que implemente aquele diagrama.



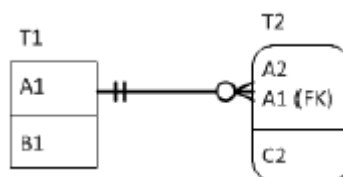
- I. A coluna A1, de E1, deve permitir valores nulos.
- II. A coluna AA de E1 não pode conter repetições de valores não nulos.

III. A coluna A1, de E2, deve permitir valores nulos.

Está correto o que se afirma em

- a) I, somente.
- b) II, somente.
- c) III, somente.
- d) I e II, somente.
- e) II e III, somente.

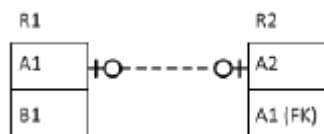
7. (FGV/2016) Analise o diagrama IDEF1X (pé de galinha) mostrado a seguir.



Sobre esse diagrama e sua implementação relacional correspondente, assinale a afirmativa correta.

- a) A1 em T2 pode ser nulo, independentemente de A2 em T2 ser nulo ou não.
- b) A1 em T2 pode ser nulo somente se A2 em T2 for nulo.
- c) O relacionamento entre T1 e T2 é "não identificador".
- d) Um registro qualquer de T2 pode não estar relacionado a algum registro de T1.
- e) Em T2, A2 e A1, concatenados, constituem um identificador para T2.

8. (FGV/COMPESA/Administrador de Banco de Dados/2016) Analise o diagrama ER, exibido a seguir com a notação IDEF1X pé-de-galinha.



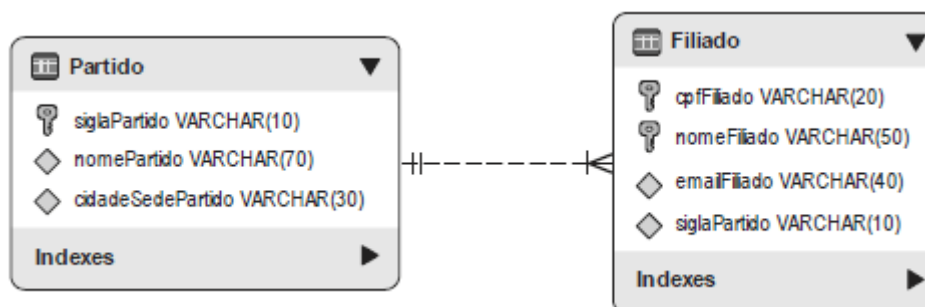
Analise, ainda, a lista de possíveis requisitos para uma implementação relacional desse modelo.

- I. A1 em R2 deve ter uma restrição do tipo UNIQUE.
- II. A1 em R2 não deve permitir valores nulos.
- III. A1 em R1 deve constituir a chave primária.
- IV. A2 em R2 deve permitir valores nulos.

Assinale a opção que indica a quantidade de requisitos corretamente estabelecidos.

- a) Zero.
- b) Um.
- c) Dois.
- d) Três.
- e) Quatro.

9. (FCC/ALMS/Informática/2016) Instrução: Para responder à questão, considere o modelo mostrado na imagem abaixo, oriundo de uma situação hipotética:



Após criadas as tabelas Partido e Filiado, foram incluídos, respectivamente, os seguintes registros:

sigla	Partido nomePartido	cidadeSedePartido
PDT	Partido Democrático Trabalhista	Brasília
PMDB	Partido do Movimento Democrático Brasileiro	Brasília
PSDB	Partido da Social Democracia Brasileira	São Paulo

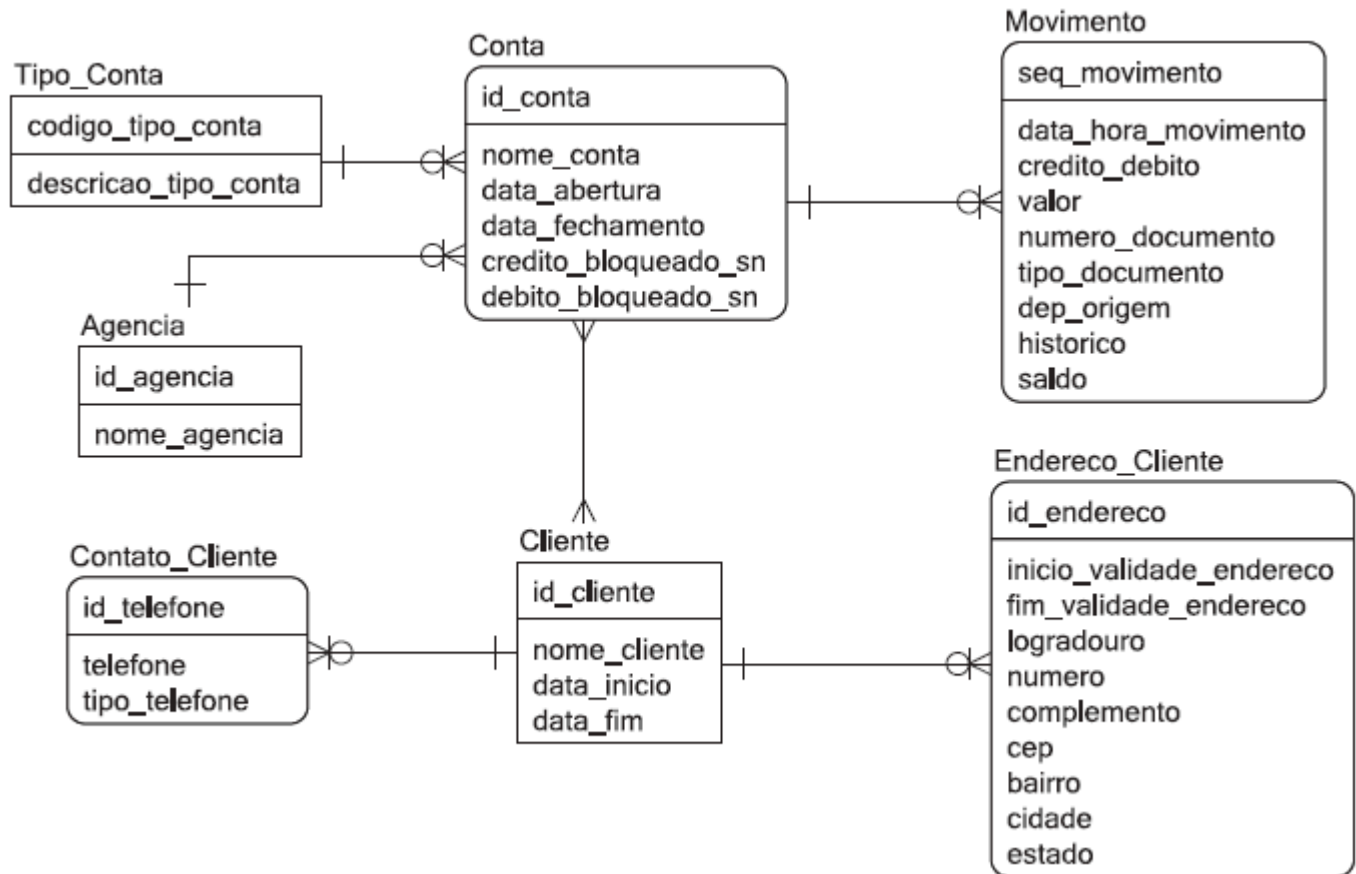
cpfFiliado	nomeFiliado	emailFiliado	siglaPartido
124.179.156-10	André Braga	braga@hotmail.com	PMDB
147.189.237-18	Marcos Pereira	mpereira@hotmail.com	PDT
154.496.172-14	Pedro Silva	pedro@gmail.com	PDT
192.345.176-01	Maria Souza	maria@ig.com.br	PSDB

Conclui-se, observando o modelo, que

- a) será necessário cadastrar, no mínimo, dois partidos com pelo menos um filiado cada.
- b) está sendo utilizada a notação IDEF1X para relacionar as duas entidades.

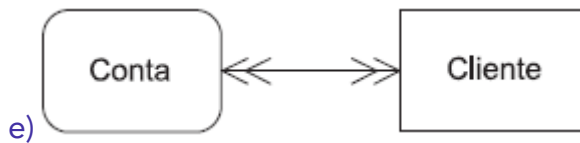
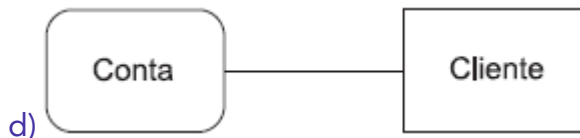
- c) se trata de um relacionamento 1:n não identificado.
- d) siglaPartido deveria fazer parte da chave primária na entidade Filiado.
- e) todo partido cadastrado precisará ter, no mínimo, um filiado cadastrado.

10. (CESGRANRIO/BASA/Tecnologia da Informação/2014) Para responder à questão, tenha como referência o diagrama de entidades e relacionamentos, apresentado abaixo, que representa parte do modelo de dados de uma instituição financeira.

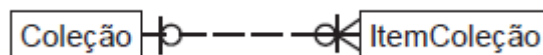


Que representação gráfica do modelo ER proposta pela notação IDEF1X representa relacionamento existente entre Conta e Cliente?

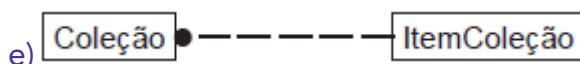
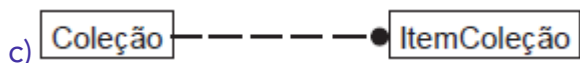
- a)
- b)
- c)



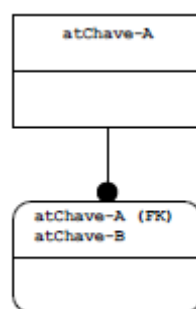
11. (CESGRANRIO/PETROBRAS/Informática/2014) O diagrama a seguir apresenta um modelo de entidades e relacionamentos segundo a notação da Engenharia de Informação.



A notação equivalente em IDEF1X é

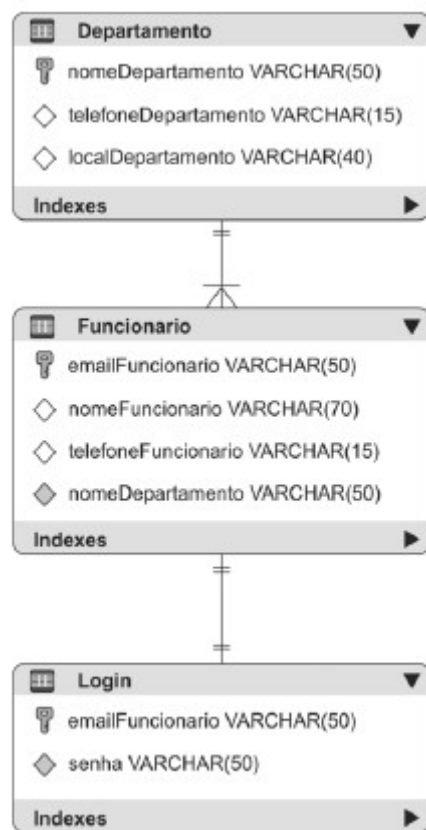


12. (NCE e FUJB (UFRJ)/2008) Na metodologia de modelagem IDEF1X, o relacionamento entre as entidades A e B, na figura a seguir, é do tipo:



- a) A para B;
- b) migratório;
- c) ortogonal;
- d) pai para filho;
- e) supertipo para subtipo.

13. (FCC/AJ TRT5/TRT 5/Apoio Especializado/Tecnologia da Informação/2022) Atenção: O Modelo Entidade-Relacionamento abaixo deve ser utilizado para responder a questão.



Observando o Modelo Entidade-Relacionamento apresentado, é possível concluir corretamente que

- a) o atributo nomeDepartamento é chave estrangeira na entidade Departamento.
- b) a entidade Login não tem chave estrangeira nem primária.
- c) a cardinalidade do relacionamento entre as entidades Funcionario e Login é um-para-um.
- d) o atributo telefoneFuncionario é adequado para ser chave primária na entidade Funcionario.
- e) a chave primária da entidade Login é composta, pois emailFuncionario é chave primaria e chave estrangeira ao mesmo tempo.

GABARITO

- | | | |
|------|------|-------|
| 1. D | 5. D | 9. C |
| 2. B | 6. B | 10. B |
| 3. C | 7. E | 11. A |
| 4. B | 8. C | 12. D |
| | | 13. C |

BIBLIOGRAFIA

- Ramakrishnan, R., & Gehrke, J. (2011). Sistemas de gerenciamento de banco de dados. AMGH Editora.
- Elmasri, R., Navathe, S. B., & Pinheiro, M. G. (2005). Sistemas de banco de dados.
- Garcia-Molina, H. (2008). Database systems: the complete book. Pearson Education India.
- Date, C. J. (2004). Introdução a sistemas de bancos de dados. Elsevier Brasil.
- Coronel, C., Morris, S., & Rob, P. (2016). Database systems: design, implementation, and management. Boston: Cengage learning.
- Debastiani, C. A. (2016). *Definindo Escopo em Projetos de Software*. Novatec Editora.

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.